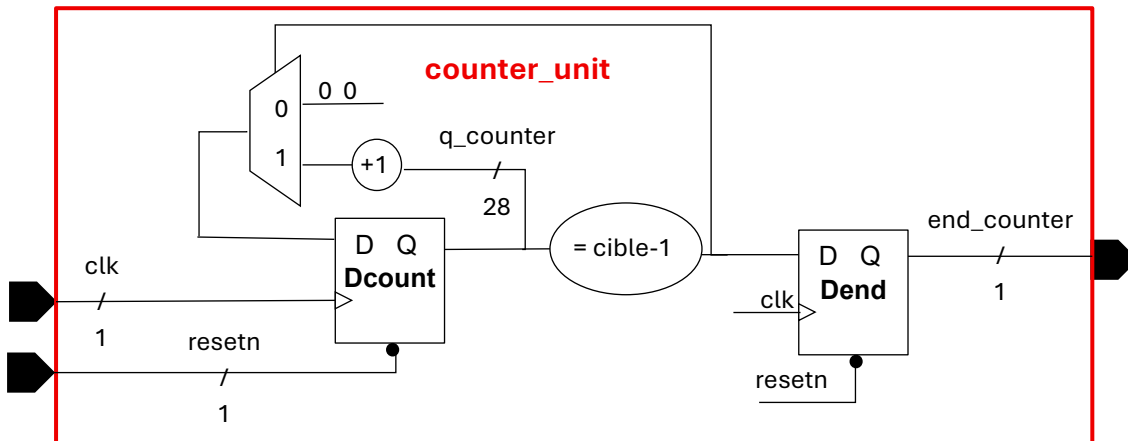


TP5 Domaines d'horloge

⇒ Architecture et code vhdl du compteur unitaire utilisé dans le TP5

Note: Après avoir déroulé le TP5 jusqu'à la fin, j'ai eu des problèmes de timings lors de l'implémentation du circuit avec la PLL et les deux horloges. Les chemins critiques se trouvaient dans le compteur unitaire du led_driver à 250Mhz. J'ai donc revu le code VHDL du compteur unit.



Dans le code VHDL initial, la logique combinatoire (additionneur, multiplexeur et comparaison) était décrite en dehors du process (resetn, clk)

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter_unit is
    generic (
        cible : integer := 20
    );
    port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          end_counter : out STD_LOGIC
    );
end counter_unit;

architecture Behavioral of counter_unit is
    signal d_counter : unsigned ( 27 downto 0 ) := (others => '0');
    signal q_counter : unsigned ( 27 downto 0 ) := (others => '0');
    signal comp : std_logic := '0'; -- sortie du comparateur

begin
    process(clk,resetn)
    begin
        if (resetn = '0') then
            q_counter <= (others => '0');
        elsif(rising_edge(clk)) then
            q_counter <= d_counter;
        end if;
    end process;

    comp <= '1' when q_counter = to_unsigned(cible - 1, q_counter'length) else
    d_counter <= q_counter + 1 when comp = '0' else (others => '0'); -- on remet le compteur à 0
    end_counter <= comp; -- rajout d'un buffer sur la sortie
end Behavioral;
```

Code initial : seulement la bascule D est décrite dans le process séquentiel

Code initial : logique combinatoire en dehors du process

Le code de compteur_unit a été simplifié en décrivant l'additionneur, le multiplexage et la comparaison dans le process séquentiel.

```
library IEEE;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter_unit is
    generic (
        cible : integer := 200000000
    );
    port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          end_counter : out STD_LOGIC
        );
end counter_unit;

architecture Behavioral of counter_unit is
    signal q_counter : unsigned ( 27 downto 0) := (others => '0');
begin
    process(clk, resetn)
    begin
        if (resetn = '0') then
            q_counter <= (others => '0');
            end_counter <= '0';
        elsif(rising_edge(clk)) then
            if q_counter = to_unsigned(cible - 1, q_counter'length) then
                q_counter <= (others => '0');
                end_counter <= '1';
            else
                q_counter <= q_counter+1;
                end_counter <= '0';
            end if;
        end if;
    end process;
end Behavioral;
```

Multiplexage

Addition

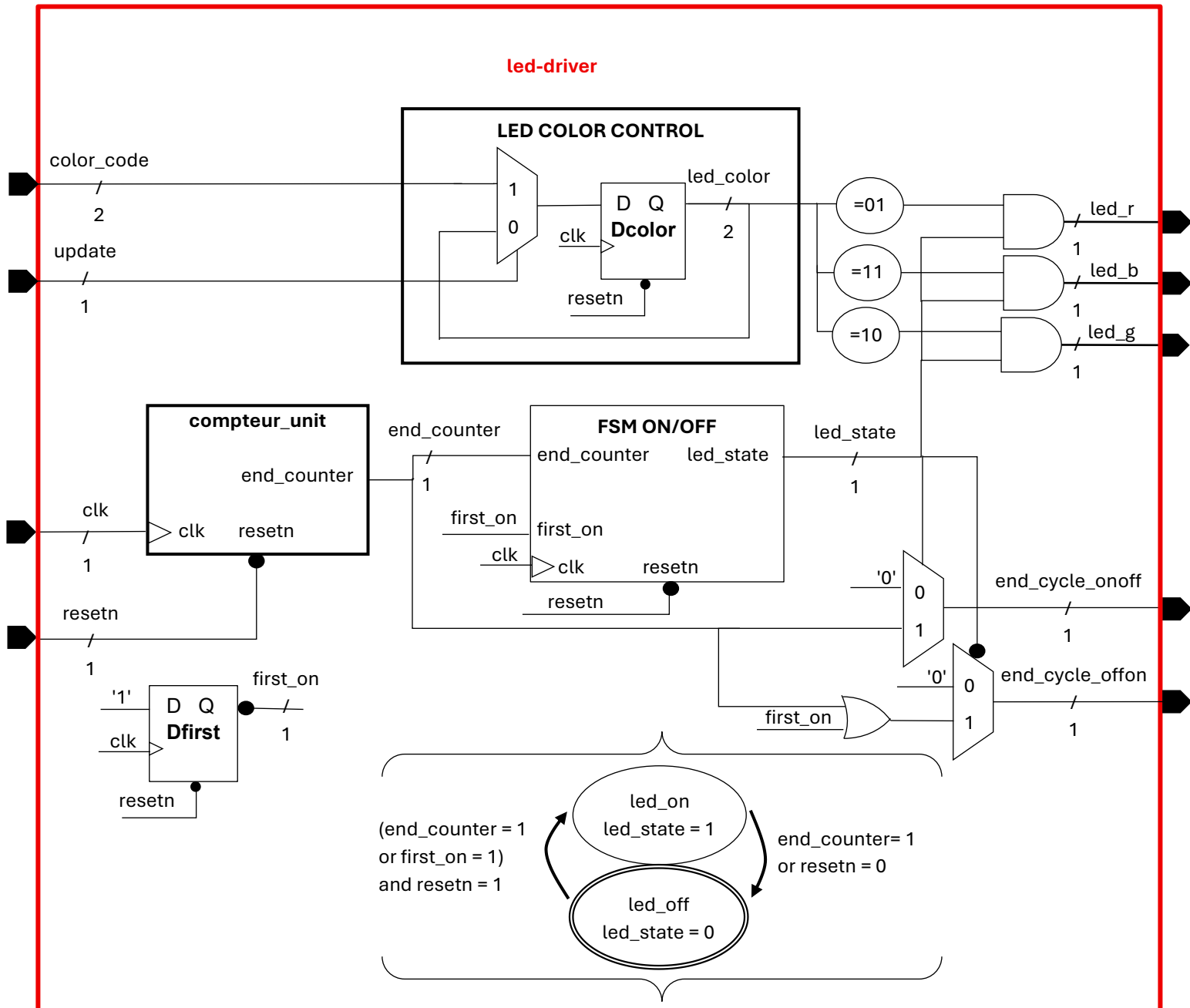
Comparaison

Registre sur end_counter

La suite du TP sera faite avec ce code pour counter_unit. Le fichier est counter_unit.vhd

⇒ Architecture et code vhdl du led_driver utilisé dans le TP5

Suite aux remarques de la correction du TP3 sur la confusion entre les états des machines d'état et les signaux, j'ai revu le schéma RTL et le code vhdl du led_driver



Le structure du led_driver a été revue notamment au niveau du décodage du color_code.

Le signal led_color est un std_logic_vector de 2 bits sur lequel on fait un comparaison pour allumer le led en rouge, en bleu ou en vert . Le signal led_color est mis à jour si update = 1 et synchronisé avant de contrôler les leds

La variable de type state n'est plus utilisée dans ce décodage.

Voici le tableau de décodage du color_code

color_code	couleur
00	off
01	rouge
10	verte
11	bleue

Voici le code vhdl du led_driver prenant en compte ces modifications. le fichier est led_driver.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity led_driver is
    generic (
        cible : integer := 20
    );
    Port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          color_code : in STD_LOGIC_VECTOR (1 downto 0);
          update : in STD_LOGIC;
          end_cycle_offon : out STD_LOGIC;
          end_cycle_onoff : out std_logic;
          led_r : out STD_LOGIC;
          led_g : out STD_LOGIC;
          led_b : out STD_LOGIC);
end led_driver;
```

```
architecture Behavioral of led_driver is
    type onoff_state is (led_on,led_off);
    signal current_onoff_state : onoff_state;
    signal next_onoff_state : onoff_state;
    signal led_state: std_logic := '0';
    signal first_on : std_logic := '0';
    signal led_color : std_logic_vector (1 downto 0):= "00";
    signal end_counter: std_logic:= '0';
begin
```

Utilisation d'un signal de décodage des couleur de type std_logic_vector 2bits

```
    cpt: entity work.counter_unit(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk,
            resetn => resetn,
            end_counter => end_counter
        );
```

```
    process(clk,resetn)
```

```
    begin
```

```
        if resetn = '0' then
```

```
            current_onoff_state <= led_off;
```

```
            first_on <= '1';
```

```
        elsif rising_edge(clk) then
```

```
            current_onoff_state <= next_onoff_state;
```

```
            first_on <= '0';
```

```
        end if ;
```

```
    end process;
```

```
    -- FSM ON OFF
```

```
    process(current_onoff_state, end_counter, first_on)
```

```
    begin
```

```
        next_onoff_state <= current_onoff_state;
```

```
        case current_onoff_state is
```

```
            when led_off =>
```

```
                if (end_counter = '1' or first_on = '1')then
```

```
                    next_onoff_state <= led_on;
```

```
                end if;
```

```
            when led_on =>
```

```
                if end_counter = '1' then
```

```
                    next_onoff_state <= led_off;
```

```
                end if;
```

```
            end case;
```

```
    end process;
```

```
    -- LED COLOR CONTROL
```

```
    process( clk, resetn)
```

```
    begin
```

```
        if resetn = '0' then
```

```
            led_color <= "00";
```

```
        elsif rising_edge(clk) then
```

```
            if update = '1' then
```

```
                led_color <= color_code;
```

```
            end if ;
```

```
        end if;
```

```
    end process;
```

```
    led_state <= '1' when current_onoff_state = led_on else '0';
```

```
    end_cycle_offon <= end_counter or first_on when led_state = '0' else '0';
```

```
    end_cycle_onoff <= end_counter when led_state = '1' else '0';
```

```
    led_r <= '1' when led_color = "01" and led_state = '1' else '0';
```

```
    led_b <= '1' when led_color = "11" and led_state = '1' else '0';
```

```
    led_g <= '1' when led_color = "10" and led_state = '1' else '0';
```

```
end Behavioral;
```

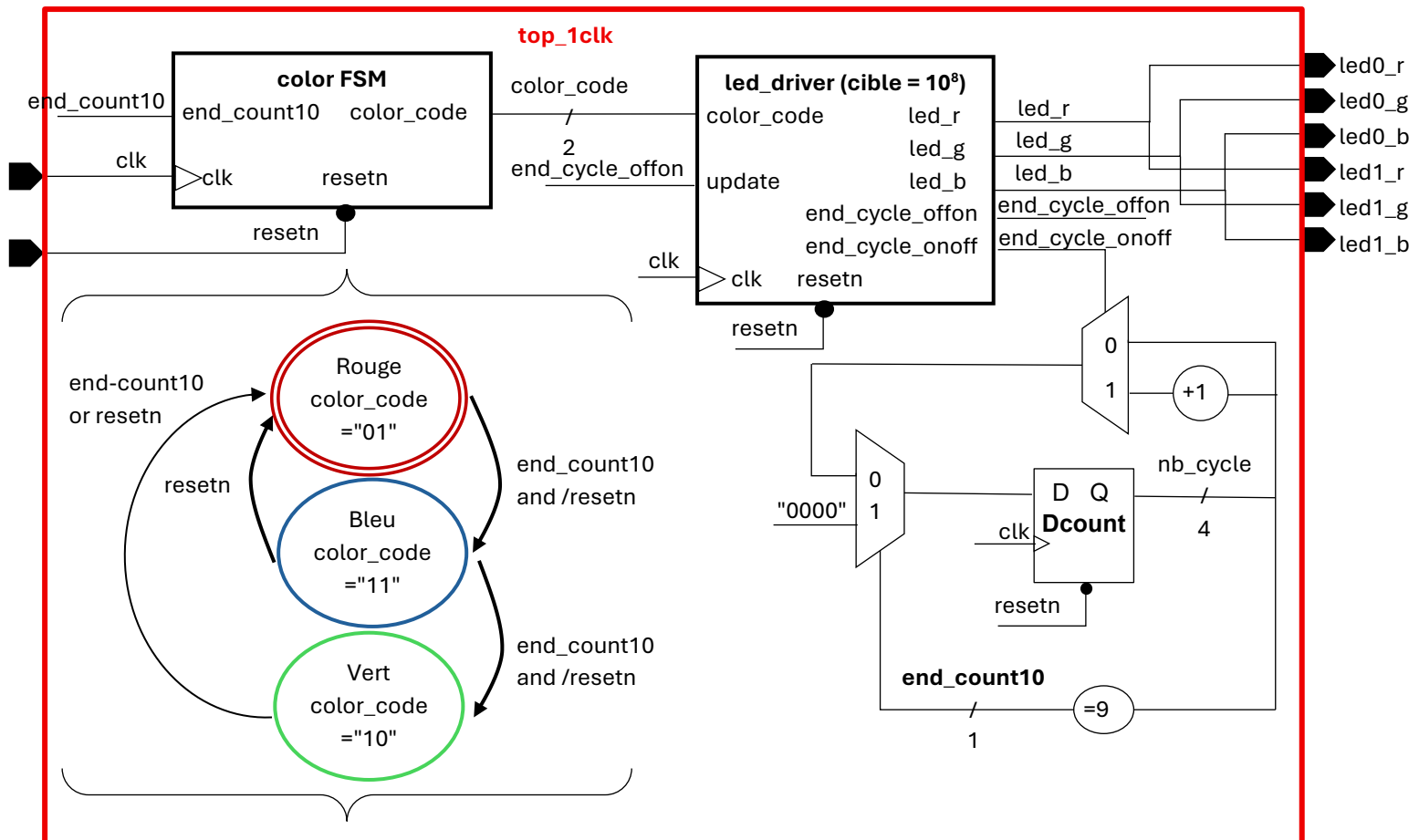
Machine d'état onoff

Mise à jour du code couleur sur le signal update

combinatoire: génération des signaux
-de gestion des fins de cycle
-de contrôle de la led en fonction du code couleur

1. A l'aide du module Led_driver du TP4, créez une architecture RTL permettant de piloter les deux LED RGB. Les LEDs RGB devront clignoter 10 fois en rouge puis 10 fois en bleu et 10 fois en vert avant de recommencer à partir du rouge. En entrée des modules Led_driver le signal update ne devra pas être à 1 pendant plus d'un coup d'horloge. Il doit s'agir d'une impulsion.

Le compteur de temporisation devra compter 100 000 000 coups d'horloge (pour une fréquence d'horloge à 100MHz, cela correspond à 1s). Dans la suite de ce TP, la valeur du compteur de temporisation ne sera pas modifiée, même lorsque les fréquences d'horloges changeront.



- On réutilise les led_driver du TP4 avec une cible à 100 000 000 pour contrôler les 3 signaux r,g,b des 2 leds led0 et led1.
- Le code couleur color_code est fourni par une machine d'état color FSM qui permet de passer de rouge à bleu à vert puis reboucler sur rouge. Rouge étant l'état initial.
- Le passage d'un état à l'autre se fait lorsque 10 cycles d'éclairage ont eu lieu (end_count10 =1)
- Le resetn est prioritaire et remet la machine d'état sur rouge.
- Le comptage de 10 cycles se fait en comptant les passages à 1 du signal end_cycle_onoff.
- La mise à jour du led_driver se fait sur le signal end_cycle_offon.

Note : end_cycle_onoff détecte une fin de cycle d'allumage de led sur le passage ON vers OFF.
end_cycle_offon détecte une fin de cycle d'allumage de led sur le passage OFF vers ON.

Le changement de la machine d'état color FSM et la mise à jour du led_driver ne se font pas sur le même signal de détection de fin de cycle afin d'éviter d'avoir des couleurs de led parasites au moment des transitions sur le led_driver.

2. Rédigez le code VHDL correspondant à votre architecture.

Le fichier est top_1clk.vhd

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_1clk is
    generic (
        cible : integer := 1000000000
    );
    port (
        clk      : in std_logic;
        resetn   : in std_logic;
        led0_r    : out std_logic;
        led0_b    : out std_logic;
        led0_g    : out std_logic;
        led1_r    : out std_logic;
        led1_b    : out std_logic;
        led1_g    : out std_logic
    );
end top_1clk;

architecture behavioral of top_1clk is
    type state is (rouge, bleu, vert);
    signal current_state : state;
    signal next_state : state;
    signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal end_cycle_onoff : std_logic:= '0';
    signal end_cycle_offon : std_logic:= '0';
    signal nb_cycle : unsigned (3 downto 0) := (others => '0');
    signal end_count10 : std_logic:= '0';
    signal update : std_logic:= '0';
    signal led_r : std_logic:= '0';
    signal led_b : std_logic:= '0';
    signal led_g : std_logic:= '0';

    begin
        led_driver: entity work.led_driver(behavioral)
            generic map (
                cible => cible
            )
            port map (
                clk => clk,
                resetn => resetn,
                color_code => color_code,
                update => end_cycle_offon,
                end_cycle_offon => end_cycle_offon,
                end_cycle_onoff => end_cycle_onoff,
                led_r => led_r,
                led_g => led_g,
                led_b => led_b
            );
    end;
```

Appel de led_driver
et connexion des signaux
d'entrée sortie

```

process(clk, resetn)
begin
    if resetn = '0' then
        nb_cycle <= (others => '0');
    elsif rising_edge(clk) then
        end_count10 <= '0';
        if end_cycle_onoff = '1' then
            if nb_cycle = 9 then
                end_count10 <= '1';
                nb_cycle <= "0000";
            else
                nb_cycle <= nb_cycle + 1;
            end if;
        end if;
    end if;
end process;

```

Compteur du nombre de cycle et
génération du signal end_count10

```

-- color fsm
process ( resetn, clk)
begin
    if resetn = '0' then
        current_state <= rouge;
    elsif rising_edge(clk) then
        current_state <= next_state;
    end if;
end process;
process (current_state, end_count10)
begin
    next_state <= current_state;
    case current_state is
        when rouge =>
            if end_count10 = '1' then
                next_state <= bleu;
            end if;
        when bleu =>
            if end_count10 = '1' then
                next_state <= vert;
            end if;
        when vert =>
            if end_count10 = '1' then
                next_state <= rouge;
            end if;
    end case;
end process;

```

machine d'état color FSM
génération du code couleur color_code

```

color_code <= "01" when current_state = rouge else
               "11" when current_state = bleu else
               "10" when current_state = vert else
               "00";

```

```

led0_r <= led_r;
led0_g <= led_g;
led0_b <= led_b;
led1_r <= led_r;
led1_g <= led_g;
led1_b <= led_b;

```

Affectation des signaux de couleur
à chacune des leds led0 et led1

```

end behavioral;

```

3. Rédigez le testbench et simulez votre design. Vérifiez que les modules réceptionnent correctement le signal update.

- Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn

Liste des signaux de sortie à vérifier : led0_r, led0_b, led0_g, led1_r, led1_b, led1_g,

Liste des signaux internes à vérifier : color_code, end_cycle_onoff, end_cycle_offon, nb_cycle, end_count10, led_driver: led_color; led_driver: end_counter

Listes des états à vérifier: current_state, led_driver: current_onoff_state

L'horloge clk est définie à 100Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 200ns

- Déroulé de la validation et du test bench

1. Test du reset au démarrage

1.1. Les entrées seront définies comme suit :

- **resetn = 0**

1.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g = 0
- led1_r = 0 , led1_b = 0 , led1_g = 0

1.3. Les signaux et états internes à vérifier

- current_state = rouge car au reset l'état de départ de color FSM est rouge
- color_code = "01" car l'état au reset de color FSM est rouge
- end_cycle_onoff = 0 , led_driver: end_counter car led_driver resetté
- end_cycle_offon = 1 car il suit led_driver: first_on = 1 quand current_onoff_state = led_off
- led_driver: led_color = "00" car led_driver resetté
- led_driver: current_onoff_state = led_off car led_driver resetté
- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée

1.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0 , led0_g = 0 après $2 \times \text{cible} \times \text{periode}(\text{clk}) + 1\text{ns}$
- on teste les 3 sorties de led1 : led1_r=0, led1_b=0 , led1_g = 0 après $2 \times \text{cible} \times \text{periode}(\text{clk}) + 1\text{ns}$
- on lance la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **resetn = 1**

2.2. Les sorties seront vérifiées

- **led0_r = 1** , led0_b = 0 , led0_g = 0 au front de clk suivant
- **led1_r = 1** , led1_b = 0 , led1_g = 0 au front de clk suivant

2.3. Les signaux et états internes à vérifier

- current_state = rouge car l'état de départ de color FSM est rouge tant que end_count10 = 0
- color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0
- **led_driver: current_onoff_state = led_on** au front de clk suivant
- led_driver: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver
- end_cycle_onoff = 0 suit end_counter quand current_onoff_state = led_on
- **end_cycle_offon = 0** au front de clk suivant car il est à 0 quand current_onoff_state = led_on
- **led_driver: led_color = "01"** au front de clk suivant
- nb_cycle = 0 et commence à s'incrémenter sur end_cycle_onoff
- end_count10 = 0 tant que end_cycle < 9

2.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=1, led0_b=0 , led0_g = 0 après $\text{periode}(\text{clk}) + 1\text{ns}$
- on teste les 3 sorties de led1 : led1_r=1, led1_b=0 , led1_g = 0 après $\text{periode}(\text{clk}) + 1\text{ns}$
- on attend periode(clk) pour lancer la suite

3. Test du fonctionnement après démarrage

3.1. Les entrées seront définies comme suit :

- **resetn = 1**

3.2. Les sorties seront vérifiées

- les 2 leds led 0 et led 1 vont clignoter 10 fois sur une couleur avant de passer à la couleur suivante

3.3. Les signaux et états internes à vérifier

- **current_state** = rouge / bleu/ vert tous les $10 \times 2 \times \text{cible} \times \text{periode}(\text{clk})$ avec une mise à jour quand **end_cycle_onoff** = 1
- **color_code** = 01/11/10 tous les $10 \times 2 \times \text{cible} \times \text{periode}(\text{clk})$ avec une mise à jour quand **end_cycle_onoff** = 1
- **led_driver: current_onoff_state** = led_on/led_off tous les $\text{cible} \times \text{periode}(\text{clk})$
- **led_driver: end_counter** = 1 pendant $\text{periode}(\text{clk})$ tous les $\text{cible} \times \text{periode}(\text{clk})$
- **end_cycle_onoff** = 1 pendant $\text{periode}(\text{clk})$ tous les $2 \times \text{cible} \times \text{periode}(\text{clk})$ quand **current_onoff_state** = led_on
- **end_cycle_offon** = 1 pendant $\text{periode}(\text{clk})$ tous les $2 \times \text{cible} \times \text{periode}(\text{clk})$ quand **current_onoff_state** = led_off
- **led_driver: led_color** = "01"/"11"/"10" tous les $10 \times 2 \times \text{cible} \times \text{periode}(\text{clk})$ avec une mise à jour quand **end_cycle_offon** = 1
- **nb_cycle** compte de 0 à 9 il s'incrémente tous les $2 \times \text{cible} \times \text{periode}(\text{clk})$ quand **end_cycle_onoff** = 1
- **end_count10** = 1 tant que **end_cycle** = 9 pendant $\text{periode}(\text{clk})$

3.4. Sur le test bench

Note: durant les test automatiques, on ne peut tester que les signaux top level. Je n'ai pas trouvé si on pouvait remonter des signaux de la hiérarchie sans créer des nouveaux port qui ne seraient utilisés que pour le debug/test.

- on attend le premier allumage de la led en rouge
- on teste les 3 sorties de led0 : **led0_r** = 0/1, **led0_b** = 0/1, **led0_g** = 0/1 après chaque période de $2 \times \text{cible} \times \text{periode}(\text{clk})$
- on teste les 3 sorties de led1 : **led1_r** = 0/1, **led1_b** = 0/1, **led1_g** = 0/1 après chaque période de $2 \times \text{cible} \times \text{periode}(\text{clk})$
- on vérifie successivement les états en rebouclant le test 10 fois pour chaque état .
- Le test bench testera 7 changement de couleur avant de continuer.

4. Test du reset en fonctionnement

4.1. Les entrées seront définies comme suit :

- **resetn = 0**

4.2. Les sorties seront vérifiées

- **led0_r** = 0, **led0_b** = 0, **led0_g** = 0
- **led1_r** = 0, **led1_b** = 0, **led1_g** = 0

4.3. Les signaux et états internes à vérifier

- **current_state** = rouge car au reset l'état de départ de color FSM est rouge
- **color_code** = "01" car l'état au reset de color FSM est rouge
- **end_cycle_onoff** = 0, **led_driver: end_counter** car **led_driver** resetté
- **end_cycle_offon** = 1 car il suit **fisrt_on** = 1 quand **current_onoff_state** = led_off
- **led_driver: led_color** = "00" car **led_driver** resetté
- **led_driver: current_onoff_state** = led_off car **led_driver** resetté
- **nb_cycle** = 0, **end_count10** = 0 car la bascule Dcount est resettée

4.4. Sur le test bench

- on teste les 3 sorties de led0 : **led0_r** = 0, **led0_b** = 0, **led0_g** = 0 après $2 \times \text{cible} \times \text{periode}(\text{clk}) + 1\text{ns}$
- on teste les 3 sorties de led1 : **led1_r** = 0, **led1_b** = 0, **led1_g** = 0 après $2 \times \text{cible} \times \text{periode}(\text{clk}) + 1\text{ns}$
- on attend $2 \times \text{cible} \times \text{periode}(\text{clk}) + 1\text{ns}$ pour continuer

5. Test du démarrage après reset

5.1. Les entrées seront définies comme suit :

- **resetn = 1**

5.2. Les sorties seront vérifiées

- **led0_r** = 1, **led0_b** = 0, **led0_g** = 0 au front de clk suivant
- **led1_r** = 1, **led1_b** = 0, **led1_g** = 0 au front de clk suivant

5.3. Les signaux et états internes à vérifier

- **current_state** = rouge car l'état de départ de color FSM est rouge tant que **end_count10** = 0
- **color_code** = "01" car l'état de départ de color FSM est rouge tant que **end_count10** = 0
- **led_driver: current_onoff_state** = **led_on** au front de clk suivant
- **led_driver: end_counter** = 0 jusqu'à la fin d' cycle de comptage de **counter_unit** du **led_driver**

- end_cycle_onoff = 0 suit end_counter quand current_onoff_state = led_on
- **end_cycle_offon = 0** au front de clk suivant car il est à 0 quand current_onoff_state = led_on
- **led_driver: led_color = "01"** au front de clk suivant
- nb_cycle = 0 et commence à s'incrémenter sur end_cycle_onoff
- end_count10 = 0 tant que end_cycle < 9

5.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=1, led0_b=0, led0_g=0 après period(clk)+1ns
- on teste les 3 sorties de led1 : led1_r=1, led1_b=0, led1_g=0 après period(clk)+1ns
- on attend 10*2*cible*periode(clk) pour terminer le test bench

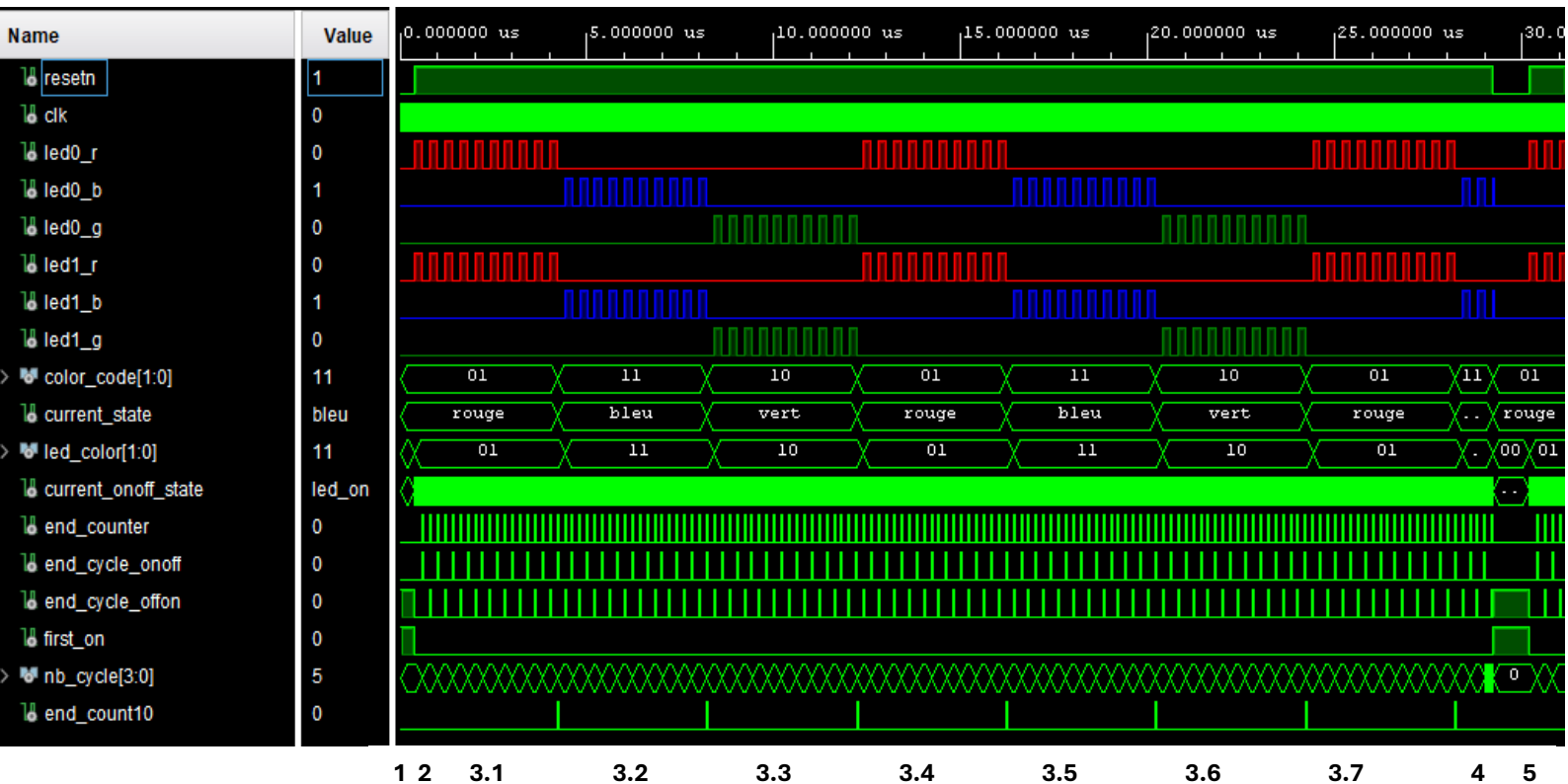
• Test bench

Le fichier de test bench est tb_top_1clk.vhd

• Résultat des test automatiques du test_bench

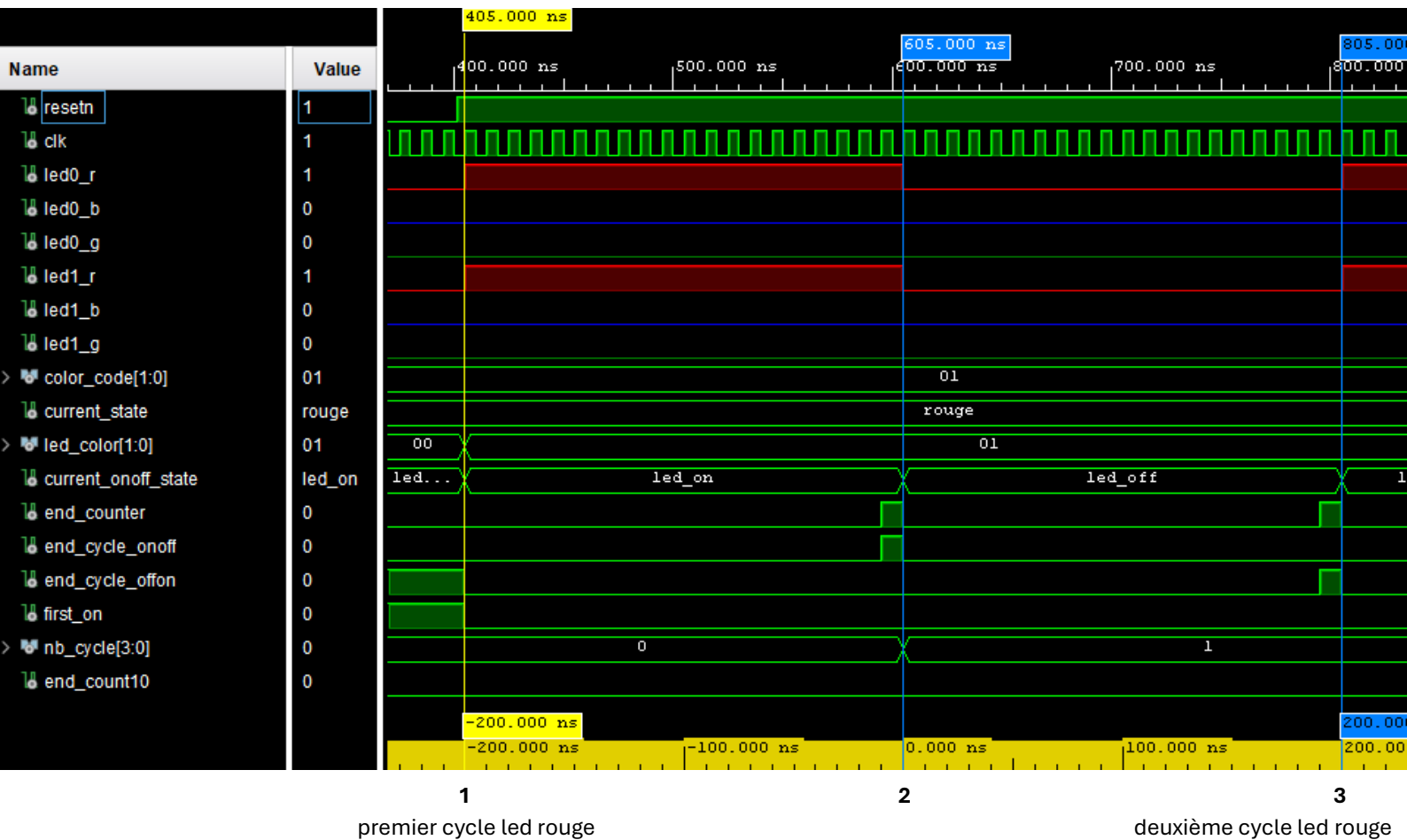
```
# run 37000ns
Note: PASS : TEST reset : les leds led0 et led1 sont eteinte OK
Time: 401 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS : TEST demarrage : les leds led0 et led1 sont eteinte OK
Time: 402 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS : TEST demarrage : les leds led0 et led1 sont rouge apres 1 periode OK
Time: 412 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 4406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence rouge
Time: 4406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 8406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence bleu
Time: 8406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 12406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence vert
Time: 12406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 16406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence rouge
Time: 16406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 20406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence bleu
Time: 20406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 24406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence vert
Time: 24406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 28406 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence rouge
Time: 28406 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS : TEST reset : les leds led0 et led1 sont eteinte OK
Time: 29613 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS : TEST demarrage : les leds led0 et led1 sont eteinte OK
Time: 30215 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS : TEST demarrage : les leds led0 et led1 sont rouge apres 1 periode OK
Time: 30225 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 34216 ns Iteration: 0 Process: /tb_top_1clk/line__137 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: PASS LED1 allumee 10 fois pour la sequence rouge
Time: 34216 ns Iteration: 0 Process: /tb_top_1clk/line__175 File: D:/Vivado/TP5/tb_top_1clk.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 34226 ns Iteration: 0 Process: /tb_top_1clk/line__75 File: D:/Vivado/TP5/tb_top_1clk.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_1clk_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 37000ns
) launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:08 . Memory (MB): peak = 2409.223 ; gain = 0.000
```

- Validation fonctionnelle



- 1- Test du reset: les leds led0 et led1 sont éteintes (led0_r = 0 , led0_b= 0 , led0_g= 0 , led1_r= 0 , led1_b= 0 , led1_g= 0)
 La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')
 Le compteur_unit est resetté (end_counter = 0)
 Le signal first_on est à 1 pour initier un LED allumée dès le démarrage après reset
 La FSM (current_onoff_state)est dans l'état led_off
 end_cycle_onoff= 0 resetté
 end_cycle_offon = 1 suit first_on = 1 quand la led est off
- 2- Démarrage après reset: les leds led0 et led1 commencent à clignoter en rouge
 (led0_r = 1 , led0_b= 0 , led0_g= 0 , led1_r= 1 , led1_b= 0 , led1_g= 0)
 La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')
 Le signal first_on passe à 0 au front de clk suivant la levée du reset
 Le counter_unit commence à compter (end_counter = 0/1 tous les cible* periode(clk))
 La FSM(current_onoff_state) bascule de led_on à led_off à chaque fin de cycle de comptage (end_counter = 1)
 end_cycle_onoff suit end_counter quand la led est on
 end_cycle_offon suit end_counter quand la led est off
- 3- Cycles de 10 clignotements sur les leds
 Les led led0 et led1 clignotent successivement en rouge, bleu puis vert avant de reboucler sur le rouge
 Les led led0 et led1 clignotent 10 fois sur chaque cycle.
- 4- Reset en cours de fonctionnement
 On se retrouve dans le même état que l'étape 1.
- 5- Redémarrage après reset
 Le système redémarre sur 10 cycles en rouge comme sur l'étape 2.

- Démarrage après un reset



1- A la levée du reset (resetn= 1) le signal end_cycle_offon = 1 = first_on

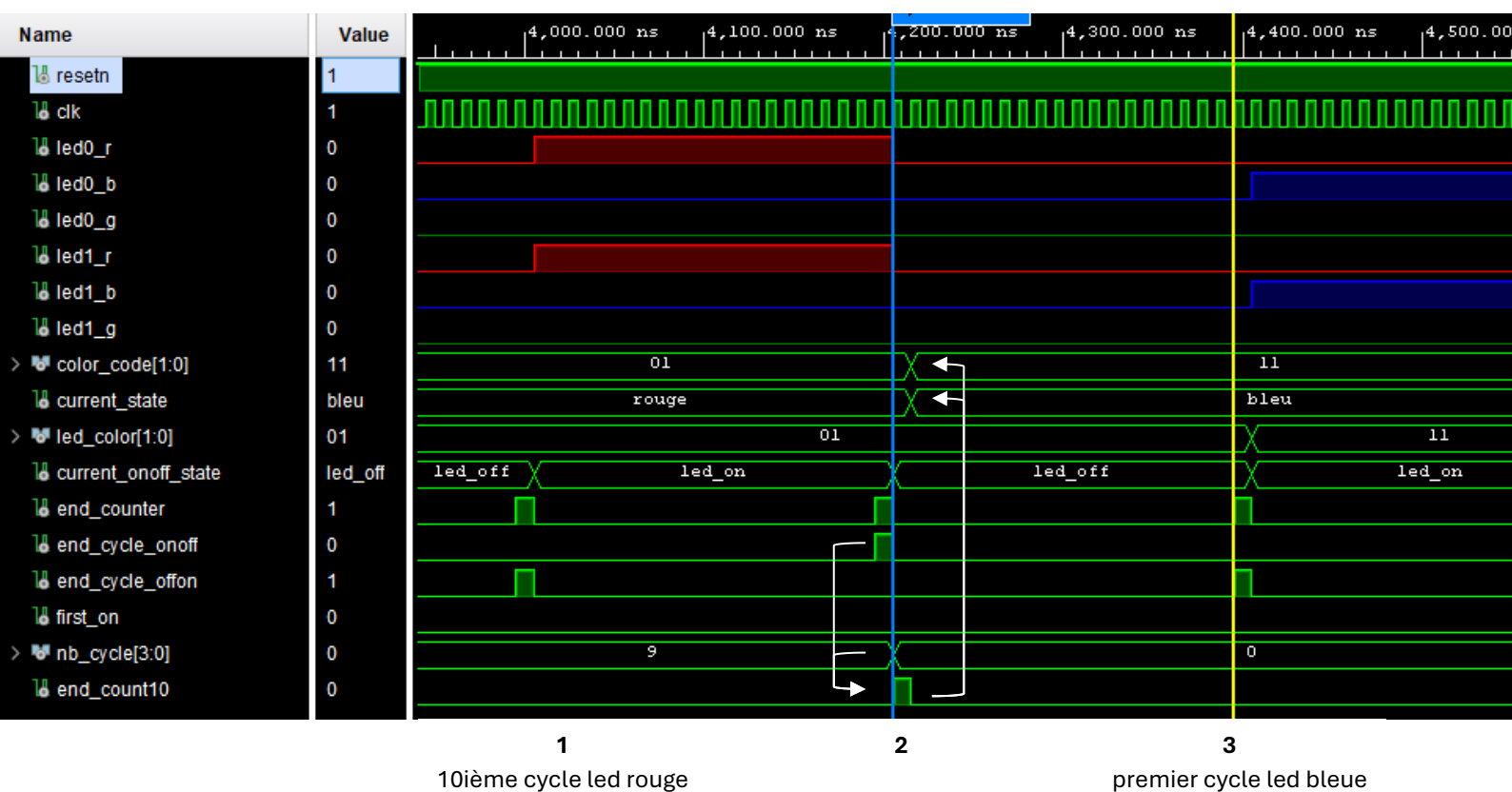
L'update du led_driver est autorisé: les leds led0 et led1 s'allument en rouge selon le color_code = "01"

La durée d'allumage est de 200ns = 20*periode(clk)

2- Le compteur de cycle s'incrémente quand end_cycle_onoff = 1 (quand les leds s'éteignent)

3- Le second cycle d'allumage des leds est autorisé par le signal end_cycle_offon = 1 = end_counter quand led_off

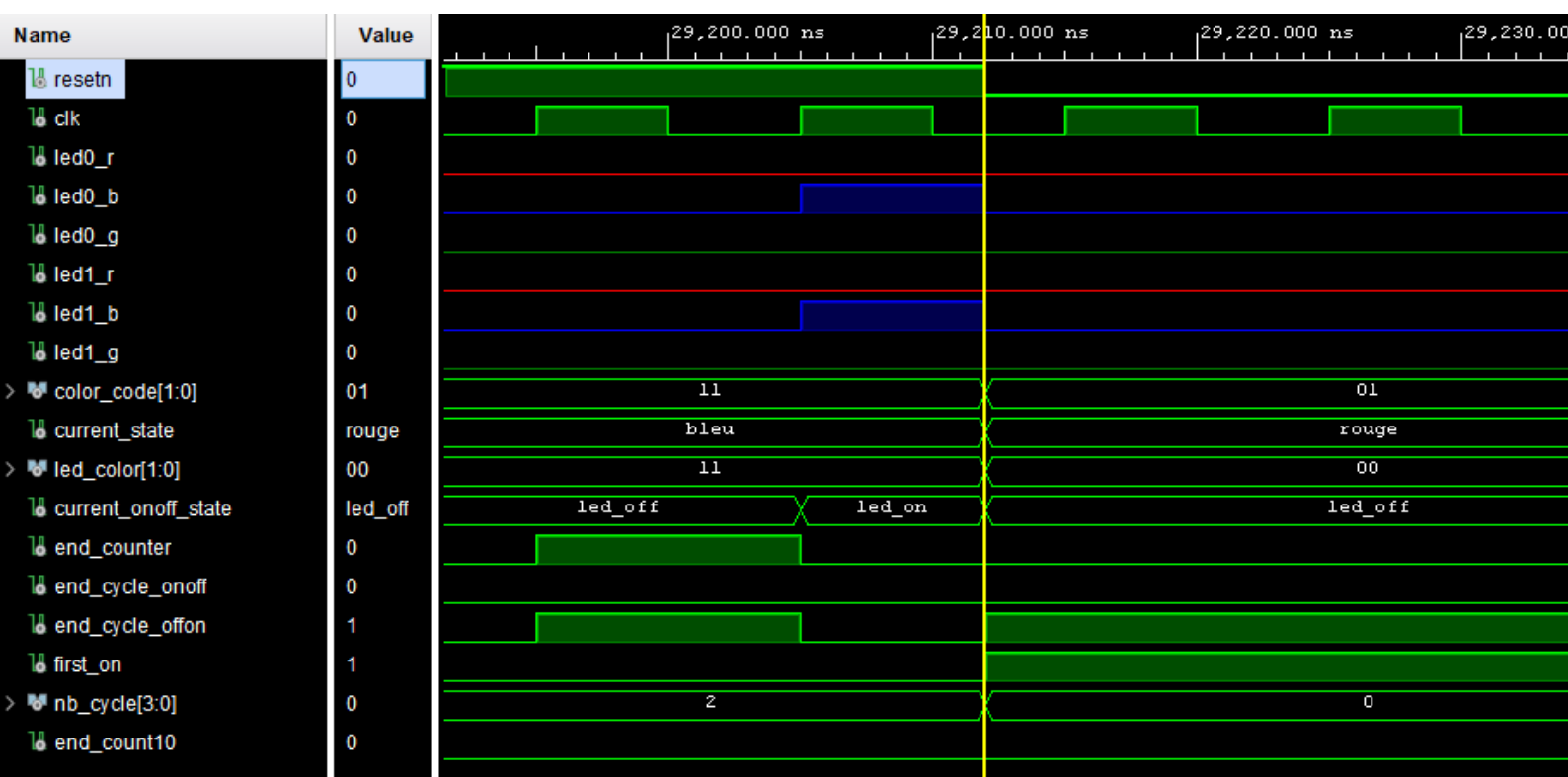
- Changement de couleur



- 1- Lors du 10^{ième} cycle en rouge, le compteur de cycle = 9
- 2- le signal end_cycle_onoff update autorise le compte du nombre de cycle nb_cycle
 end_count10 passe à 1 pendant periode(clk) lorsque nb_cycle = 9
 le color_code est mis à jour sur la valeur correspondant à bleu "11" et l'état de la color FSM passe de rouge à bleu sur le coup d'horloge suivant
- 3- l'update du led_driver intervient lors du prochain allumage des led (end_cycle_offon = 1)
le signal end_cycle_offon update le led_driver.
 Les leds led0 et led1 s'allument en bleu selon le color_code = "11" sur le coup d'horloge suivant
 La durée d'allumage est de 200ns = 20*periode(clk)

Note: le décalage des mise à jour de la color fsm et du led_driver permet d'avoir un changement de couleur propre en début de cycle. Le code couleur est très bien établi quand il est utilisé par le led_driver pour mettre à jour la couleur des leds

- reset en cours d'allumage



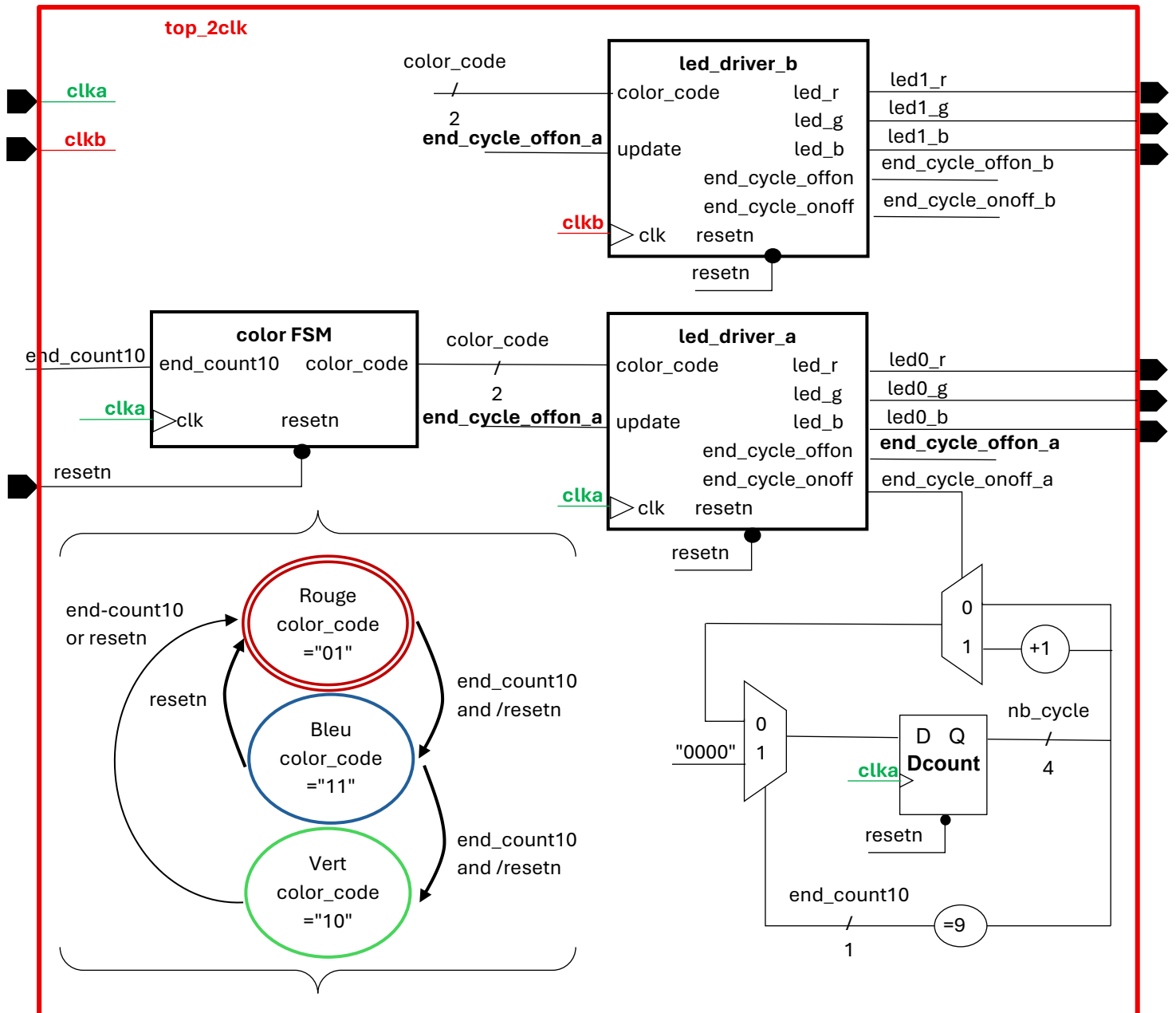
Lors du reset , toutes le signaux sont resetté de façon asynchrone , les leds sont off immédiatement.

4. Modifiez votre design pour gérer deux signaux d'horloge différents.

La première horloge, clkA, est associée à la logique en dehors des modules LED_driver et au module LED_driver de la LED0.

La deuxième horloge, clkB, est associée au module LED_driver de la LED1.

Le changement de couleur des deux LEDs à lieu lorsque la LED0 à clignoté 10 fois.



- le module `led_driver` est dupliqué: `led_driver_a` contrôle les signaux de la led0
`led_driver_b` contrôle les signaux de la led1
- le signal `clkA` contrôle : `led_driver_a`
la machine d'état `color fsm`
le compteur de cycle qui compte les 10 cycles (`Dcount`)
- le signal `clkB` contrôle : `led_driver_b`
- Le comptage des 10 cycles se fait à partir du signal `end_count10` généré par un compteur déclenché par `end_cycle_onoff_a` du `led_driver_a` (clock `clkA`)
- La mise à jour des 2 `led_driver` se fait à partir du signal généré `led_driver_a` : `end_cycle_offon_a`

- code VHDL de l'architecture

le fichier est top_2clk.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_2clk is
    generic (
        cible : integer := 100000000
    );
    port (
        clka          : in std_logic;
        clk_b         : in std_logic;
        resetn        : in std_logic;
        led0_r         : out std_logic;
        led0_b         : out std_logic;
        led0_g         : out std_logic;
        led1_r         : out std_logic;
        led1_b         : out std_logic;
        led1_g         : out std_logic
    );
end top_2clk;

architecture behavioral of top_2clk is
    type state is (rouge, bleu, vert);
    signal current_state : state;
    signal next_state : state;
    signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal end_cycle_onoff_a : std_logic:= '0';
    signal end_cycle_offon_a : std_logic:= '0';
    signal end_cycle_onoff_b : std_logic:= '0';
    signal end_cycle_offon_b : std_logic:= '0';
    signal nb_cycle : unsigned (3 downto 0) := (others => '0');
    signal end_count10 : std_logic:= '0';
    signal led_r : std_logic:= '0';
    signal led_b : std_logic:= '0';
    signal led_g : std_logic:= '0';

    begin
        led_driver_a: entity work.led_driver(behavioral)
            generic map (
                cible => cible
            )
            port map (
                clk => clka,
                resetn => resetn,
                color_code => color_code,
                update => end_cycle_offon_a,
                end_cycle_offon => end_cycle_offon_a,
                end_cycle_onoff => end_cycle_onoff_a,
                led_r => led0_r,
                led_g => led0_g,
                led_b => led0_b
            );
        led_driver_b: entity work.led_driver(behavioral)
            generic map (
                cible => cible
            )
            port map (
                clk => clk_b,
                resetn => resetn,
                color_code => color_code,
                update => end_cycle_offon_b,
                end_cycle_offon => end_cycle_offon_b,
                end_cycle_onoff => end_cycle_onoff_b,
                led_r => led1_r,
                led_g => led1_g,
                led_b => led1_b
            );
    end;

```

Appel de led_driver_a
et connexion des signaux
d'entrée sortie
utilisation de clka
update par end_cycle_offon_a
sortie sur led0

Appel de led_driver_b
et connexion des signaux
d'entrée sortie
utilisation de clk_b
update par end_cycle_offon_b
sortie sur led1

```

process(clka, resetn)
begin
    if resetn = '0' then
        nb_cycle <= (others => '0');
    elsif rising_edge(clka) then
        end_count10 <= '0';
        if end_cycle_onoff_a = '1' then
            if nb_cycle = 9 then
                end_count10 <= '1';
                nb_cycle <= "0000";
            else
                nb_cycle <= nb_cycle + 1;
            end if;
        end if;
    end if;
end process;

```

Compteur du nombre de cycle et
génération du signal end_count10
sur clka

```

-- color fsm
process ( resetn, clka)
begin
    if resetn = '0' then
        current_state <= rouge;
    elsif rising_edge(clka) then
        current_state <= next_state;
    end if;
end process;
process (current_state, end_count10)
begin
    next_state <= current_state;
    case current_state is
        when rouge =>
            if end_count10 = '1' then
                next_state <= bleu;
            end if;
        when bleu =>
            if end_count10 = '1' then
                next_state <= vert;
            end if;
        when vert =>
            if end_count10 = '1' then
                next_state <= rouge;
            end if;
    end case;
end process;
color_code <= "01" when current_state = rouge else
               "11" when current_state = bleu else
               "10" when current_state = vert else
               "00";
end behavioral;

```

machine d'état color FSM
génération du code couleur color_code
sur clka

- Simulation et test bench

- a. Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn

Liste des signaux de sortie à vérifier : led0_r, led0_b, led0_g, led1_r, led1_b, led1_g,

Liste des signaux internes à vérifier : color_code,

end_cycle_onoff_a, end_cycle_offon_a,

end_cycle_onoff_b, end_cycle_offon_b,

nb_cycle, end_count10,

led_driver_a: led_color_a; led_driver_a: end_counter_a,

led_driver_b: led_color_b; led_driver_b: end_counter_b

Listes des états à vérifier: current_state, led_driver_a: current_onoff_state, led_driver_b: current_onoff_state

L'horloge clka est définie à 250Mhz

L'horloge clkb est définie à 50Mhz

Note : les simulations ont été faite avec une cible de 20 soit un temps de comptage du compteur unitaire de 80ns

- b. Déroulé de la validation et du test bench

- 1. Test du reset au démarrage

- 1.1. Les entrées seront définies comme suit :

- **resetn = 0**

- 1.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g= 0

- led1_r = 0 , led1_b = 0 , led1_g= 0

- 1.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car au reset l'état de départ de color FSM est rouge

- end_cycle_onoff_a = 0 , led_driver: end_counter car led_driver resetté

- end_cycle_offon_a = 1 car il suit led_driver_a: fisrt_on = 1 quand current_onoff_state_a = led_off

- end_cycle_onoff_b = 0 , led_driver: end_counter car led_driver resetté

- end_cycle_offon_b = 1 car il suit led_driver_b: fisrt_on = 1 quand current_onoff_state_b = led_off

- led_driver_a: led_color = off car led_driver_a resetté

- led_driver_a: current_onoff_state = led_off car led_driver_a resetté

- led_driver_b: led_color = off car led_driver_b resetté

- led_driver_b: current_onoff_state = led_off car led_driver_b resetté

- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée

- 1.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0 , led0_g= 0 après 2*cible*periode(clka)+1ns

- on teste les 3 sorties de led1 : led1_r=0, led1_b=0 , led1_g= 0 après 2*cible*periode(clkb)+1ns

- on attend cible*period(clkb) +1ns pour lancer la suite

- 2. Test du démarrage après reset

- 2.1. Les entrées seront définies comme suit :

- **resetn = 1**

- 2.2. Les sorties seront vérifiées

- **led0_r = 1** , led0_b = 0 , led0_g= 0 au front de clka suivant

- **led1_r =1** , led1_b = 0 , led1_g= 0 au front de clkb suivant

- 2.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0

- **led_driver_a: current_onoff_state = led_on au front de clka suivant**

- **led_driver_b: current_onoff_state = led_on au front de clkb suivant**

- led_driver_a: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_a soit après cible *periode(clka) = 80ns

- led_driver_b: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_b soit après cible *periode(clkb) = 400ns

- pour les driver a et b : `end_cycle_onoff = 0` suit `led_driver: end_counter` quand `led_driver: current_onoff_state = led_on`
- pour les driver a et b : **`end_cycle_offon = 0`** au front de leur `clk` suivant car il est à 0 quand `current_onoff_state = led_on`
- pour les driver a et b : **`led_driver: led_color = rouge`** au front de leur `clk` suivant
- `nb_cycle = 0` et commence à s'incrémenter sur `end_cycle_onoff_a`
- `end_count10 = 0` tant que `nb_cycle < 9`

2.4. Sur le test bench

- on teste les 3 sorties de `led0` : `led0_r = 1`, `led0_b = 0`, `led0_g = 0` après `periode(clka) + 1ns`
- on teste les 3 sorties de `led1` : `led1_r = 1`, `led1_b = 0`, `led1_g = 0` après `periode(clkb) + 1ns`
- on attend `periode(clk)` pour lancer la suite

3. Test du fonctionnement après démarrage

3.1. Les entrées seront définies comme suit :

- `resetn = 1`

3.2. Les sorties seront vérifiées

- la `led 0` vont clignoter 10 fois sur une couleur avant de passer à la couleur suivante
- le `led 1` vont clignoter 2 fois sur une couleur avant de passer à la couleur suivante sur la fin de cycle de la `led 0`.
(`nb_clignotement_led1 = nb_clignotement_led0 * clkb/clka = 2`)

3.3. Les signaux et états internes à vérifier

- `current_state = rouge / bleu/ vert` tous les `10*2*cible*periode(clka)` avec une mise à jour quand `end_cycle_onoff = 1`
- `color_code = 01/11/10` tous les `10*2*cible*periode(clka)` avec une mise à jour quand `end_cycle_onoff_a = 1`
- pour les driver a et b : `led_driver: current_onoff_state = led_on/led_off` tous les `cible*periode(clka et b)`
- pour les driver a et b : `led_driver: end_counter = 1` pendant `periode(clk)` tous les `cible*periode(clka et b)`
- pour les driver a et b : `end_cycle_onoff = 1` pendant `periode(clk)` tous les `2*cible*periode(clka et b)` quand `current_onoff_state = led_on`
- pour les driver a et b : `end_cycle_offon = 1` pendant `periode(clk)` tous les `2*cible*periode(clk)` quand `current_onoff_state = led_off`
- pour les driver a et b `led_driver: led_color = 01/11/10` tous les `10*2*cible*periode(clka et b)` avec une mise à jour quand `end_cycle_offon = 1`
- `nb_cycle` compte de 0 à 9 il s'incrémente tous les `2*cible*periode(clka)` quand `end_cycle_onoff_a = 1`
- `end_count10 = 1` quand `end_cycle = 9` pendant `periode(clka)`

3.4. Sur le test bench

- on attend le premier allumage de la `led` en rouge
- on teste les 3 sorties de `led0` : `led0_r = 0/1`, `led0_b = 0/1`, `led0_g = 0/1` après chaque période de `(2*cible)*periode(clka)`
- on teste les 3 sorties de `led1` : `led1_r = 0/1`, `led1_b = 0/1`, `led1_g = 0/1` après chaque période de `(2*cible)*periode(clkb)`
- sur la `led0` on vérifie successivement les sortie `led0` en rebouclant le test 10 fois pour chaque couleur
- sur la `led1` on sort de la boucle de comptage de cycle de la `led1` après 2 cycles sur `clkb`. on aura compté que 2 cycles
- Le test bench testera 7 changements de couleur avant de continuer.

4. Test du reset en fonctionnement

4.1. Les entrées seront définies comme suit :

- **`resetn = 0`**

4.2. Les sorties seront vérifiées

- `led0_r = 0`, `led0_b = 0`, `led0_g = 0`
- `led1_r = 0`, `led1_b = 0`, `led1_g = 0`

4.3. Les signaux et états internes à vérifier

Tous les signaux sont resetté de façon asynchrone dès que `resetn = 0`

- `current_state = rouge` et `color_code = "01"` car au reset l'état de départ de color FSM est rouge
- `end_cycle_onoff_a = 0`, `led_driver: end_counter` car `led_driver` resetté
- `end_cycle_offon_a = 1` car il suit `led_driver_a: fisrt_on = 1` quand `current_onoff_state_a = led_off`
- `end_cycle_onoff_b = 0`, `led_driver: end_counter` car `led_driver` resetté
- `end_cycle_offon_b = 1` car il suit `led_driver_b: fisrt_on = 1` quand `current_onoff_state_b = led_off`
- `led_driver_a: led_color = off` car `led_driver_a` resetté
- `led_driver_a: current_onoff_state = led_off` car `led_driver_a` resetté

- led_driver_b: led_color = off car led_driver_b resetté
- led_driver_b: current_onoff_state = led_off car led_driver_b resetté
- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée

4.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0 , led0_g= 0 après 1ns
- on teste les 3 sorties de led1 : led1_r=0, led1_b=0 , led1_g= 0 après 1ns
- on attend $2 \times \text{cible} \times \text{period}(\text{clkb}) + 1\text{ns}$ pour lancer la suite

5. Test du démarrage après reset

5.1. Les entrées seront définies comme suit :

- **resetn = 1**

5.2. Les sorties seront vérifiées

- **led0_r = 1** , led0_b = 0 , led0_g= 0 au front de clka suivant
- **led1_r = 1** , led1_b = 0 , led1_g= 0 au front de clkb suivant

5.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0
- **led_driver_a: current_onoff_state = led_on au front de clka suivant**
- **led_driver_b: current_onoff_state = led_on au front de clkb suivant**
- led_driver_a: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_a soit après $\text{cible} \times \text{periode}(\text{clka}) = 80\text{ns}$
- led_driver_b: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_b soit après $\text{cible} \times \text{periode}(\text{clkb}) = 400\text{ns}$
- pour les driver a et b : end_cycle_onoff = 0 suit led_driver: end_counter quand led_driver: current_onoff_state = led_on
- pour les driver a et b : **end_cycle_offon = 0** au front de leur clk suivant car il est à 0 quand current_onoff_state = led_on
- pour les driver a et b : **led_driver: led_color = rouge** au front de leur clk suivant
- nb_cycle = 0 et commence à s'incrémenter sur end_cycle_onoff_a
- end_count10 = 0 tant que nb_cycle < 9

5.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=1, led0_b=0 , led0_g= 0 après $\text{periode}(\text{clka}) + 1\text{ns}$
- on teste les 3 sorties de led1 : led1_r=1, led1_b=0 , led1_g= 0 après $\text{periode}(\text{clkb}) + 1\text{ns}$
- le test est terminé

• Test bench

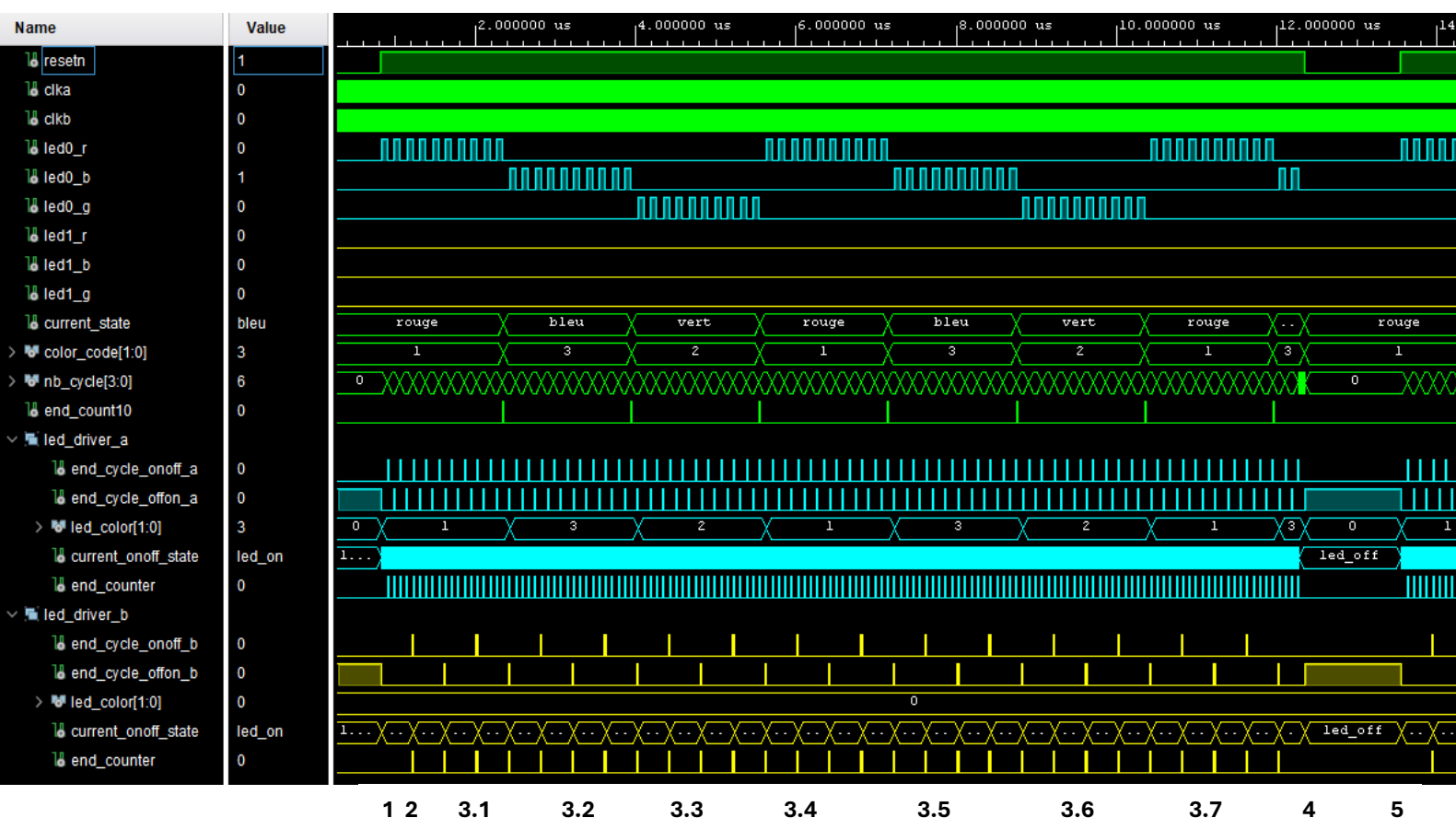
Le fichier de test bench est tb_top_2clk.vhd

- Résultat des test automatiques du test_bench

```
# run 17000ns
Note: PASS : TEST reset : led0 est eteinte OK
Time: 161 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST reset : led1 est eteinte OK
Time: 801 ns Iteration: 0 Process: /tb_top_2clk/line__206 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST demarrage : led1 est eteinte OK
Time: 823 ns Iteration: 0 Process: /tb_top_2clk/line__206 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST demarrage : led0 est eteinte OK
Time: 823 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: FAIL LED1 ne s'allume pas au demarrage
Time: 843 ns Iteration: 0 Process: /tb_top_2clk/line__206 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 2427 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 4027 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 5627 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 7227 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 8827 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 10427 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 12027 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST reset : la led0 est eteinte OK
Time: 12343 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST reset : la led1 est eteinte OK
Time: 12343 ns Iteration: 0 Process: /tb_top_2clk/line__206 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST demarrage : la led1 est eteinte OK
Time: 13544 ns Iteration: 0 Process: /tb_top_2clk/line__206 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST demarrage : la led0 est eteinte OK
Time: 13544 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: PASS : TEST demarrage : la led0 est rouge apres 1 periode clka OK
Time: 13548 ns Iteration: 0 Process: /tb_top_2clk/line__110 File: D:/Vivado/TP5/tb_top_2clk.vhd
Error: FAIL : TEST demarrage : la led1 est rouge apres 1 periode clkb KO
Time: 13564 ns Iteration: 0 Process: /tb_top_2clk/line__206 File: D:/Vivado/TP5/tb_top_2clk.vhd
Note: Nombre de test FAIL : 2
Time: 14024 ns Iteration: 0 Process: /tb_top_2clk/line__84 File: D:/Vivado/TP5/tb_top_2clk.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_2clk_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 17000ns
launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:10 . Memory (MB): peak = 1989.605 ; gain = 0.000
```

Le résultat du test automatique montre des fails sur l'affichage de la LED1: elle ne s'allume pas .

- Validation fonctionnelle



1- Test du reset: les leds led0 et led1 sont éteintes (led0_r = 0 , led0_b= 0 , led0_g= 0 , led1_r = 0 , led1_b= 0 , led1_g= 0)

La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')

La FSM (current_onoff_state) est dans l'état led_off

end_cycle_onoff_a= 0 est resetté et end_cycle_offon_a = 1

end_cycle_onoff_b = 0 est resetté et end_cycle_offon_b = 1

nb_cycle = 0 , end_count10 = 0

2- Démarrage après reset:

La led0 commence à clignoter en rouge

La led1 ne clignote pas alors qu'elle devrait commencer à clignoter

(led0_r = 1 , led0_b= 0 , led0_g= 0 , led1_r = 1 , led1_b= 0 , led1_g= 0)

La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')

Le led_driver_a compteur_unit commence à compter (end_counter = 0/1 tous les cible* periode(clka))

Le led_driver_b compteur_unit commence à compter (end_counter = 0/1 tous les cible* periode(clkb))

La led_driver_a FSM(current_onoff_state) = led_on/led_off à chaque de cycle de comptage (end_counter_a = 1)

La led_driver_b FSM(current_onoff_state) = led_on/led_off à chaque de cycle de comptage (end_counter_b = 1)

end_cycle_onoff_a suit end_counter_a quand la led est on

end_cycle_offon_a suit end_counter_a quand la led est off

end_cycle_onoff_a suit end_counter_b quand la led est on

end_cycle_offon_a suit end_counter_b quand la led est off

led_driver_a: led_color = "01"

led_driver_b: led_color = "00" , il devrait être à "01" rouge

3- Cycles de 10 clignotements sur les leds

La led0 clignote successivement en rouge, bleu puis vert avant de reboucler sur le rouge

La led0 clignote 10 fois sur chaque cycle.

La led1 ne clignote pas alors qu'elle devrait clignoter

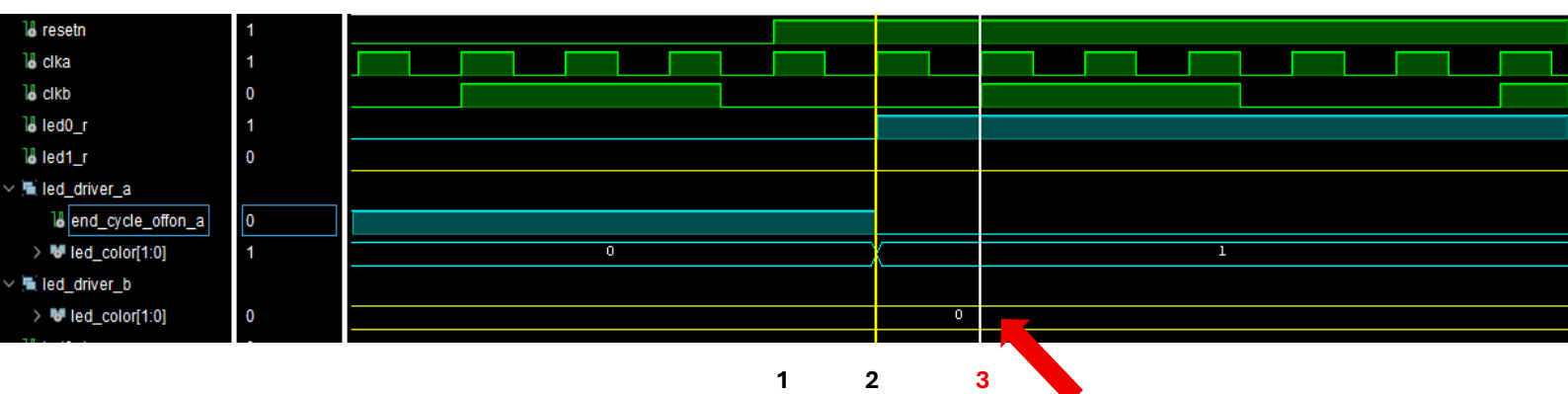
4- Reset en cours de fonctionnement

On se retrouve dans le même état que l'étape 1.

5- Redémarrage après reset

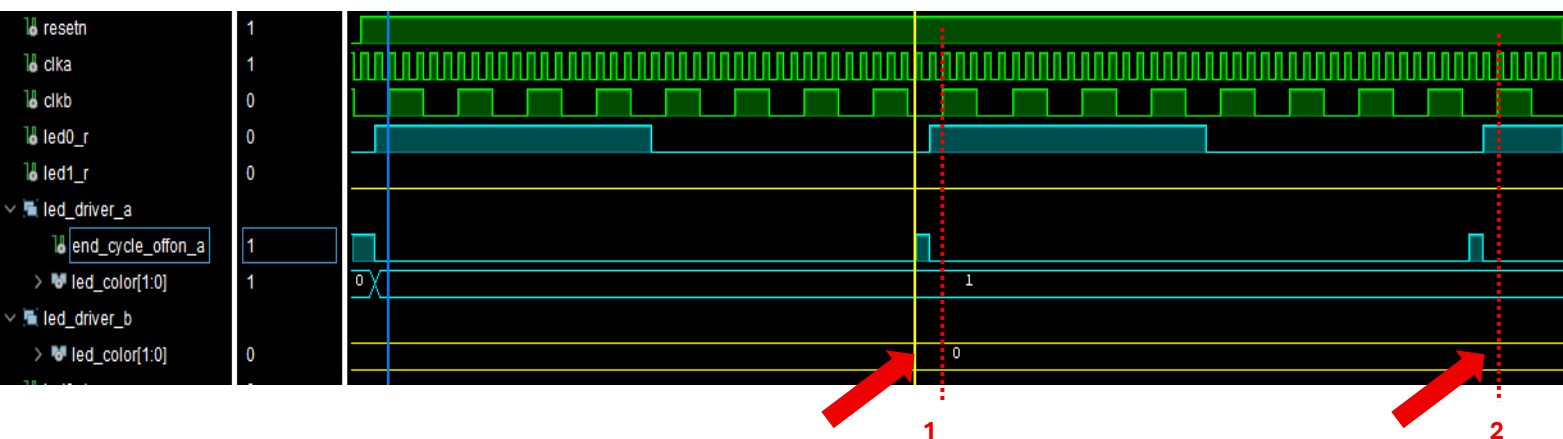
Le système redémarre sur 10 cycles en rouge comme sur l'étape 2.

- Zoom sur le démarrage



- 1- Au démarrage (resetn = 1), le signal end_cycle_offon_a reste à 1 jusqu'à ce qu'au prochain front montant de clka
- 2- Led_driver_a fait son update_a = end_cycle_offon_a sur le front montant de clka (led_color passe à rouge) et met à jour la couleur de la led0 (led0_r = 1)
- 3- Led_driver_b devrait faire son update quand update_b = end_cycle_offon_a sur le front montant de clkb
Or sur le front montant de clkb, le signal update_b = end_cycle_offon_a est à zéro.
le led_driver_b ne peut pas faire son update, la led1 reste à son état au reset c'est-à-dire off.

- Zoom sur les cycles suivants

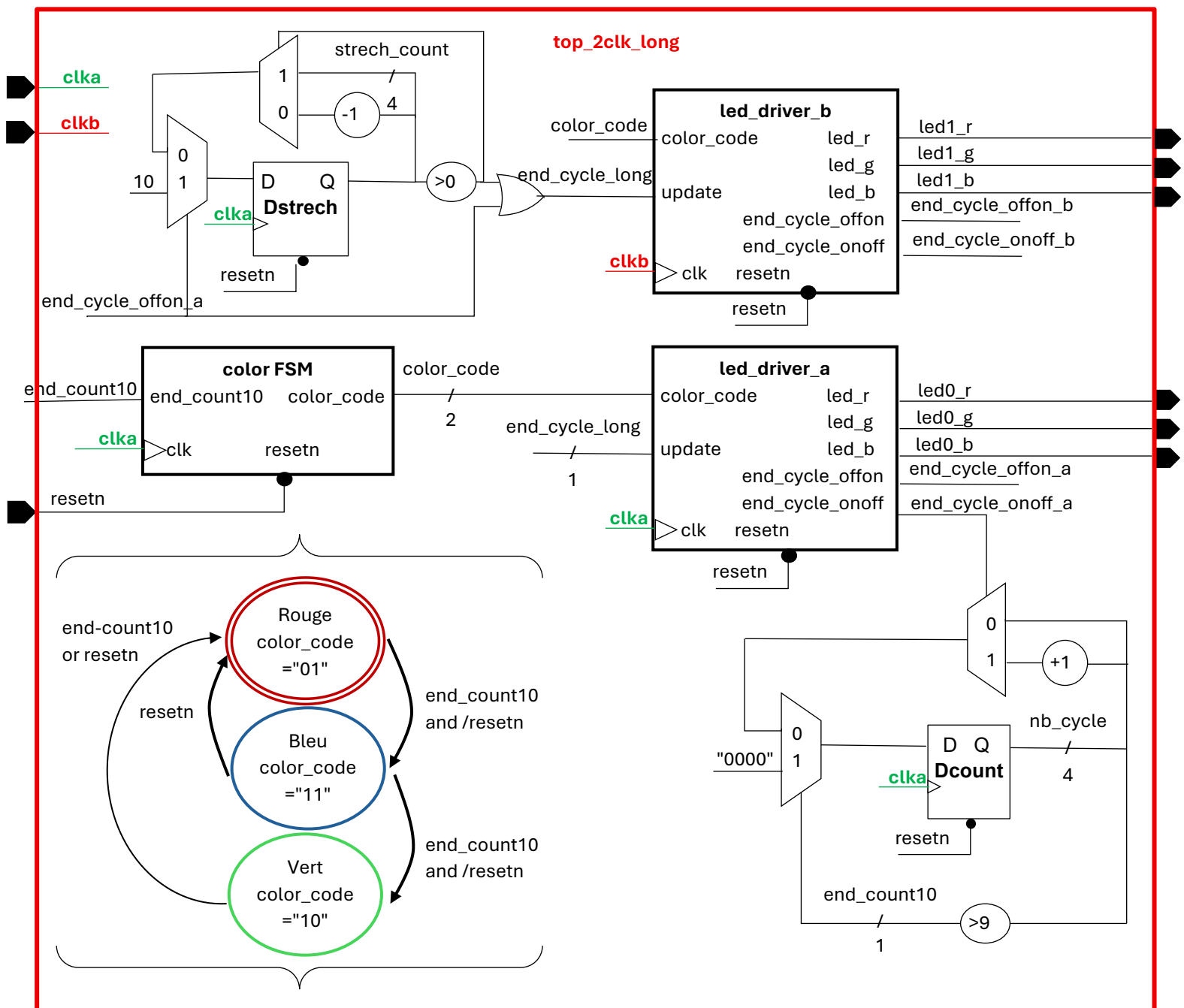


- 1-2- Led_driver_b devrait faire son update quand update_b = end_cycle_offon_a sur le front montant de clkb
Le signal end_cycle_offon_a reste à 1 sur une période de clka soit 4ns
Le led_driver_b fonctionne sur une période de 20ns.
Il est impossible de capturer le signal end_cycle_offon_a sur un front montant de l'horloge clkb pour mettre à jour le led_driver_b

7. Proposez une solution pour corriger le problème lié au changement de domaine d'horloge, vous pourrez vous aider du lien <https://nandland.com/lesson-14-crossing-clock-domains/>.

Après avoir consulté la page nandland/crossing-clock-domain: la solution est d'allonger la durée où le signal end_cycle_offon_a est à 1
Les recommandations sont d'allonger le signal end_cycle_offon_a de $2 \cdot f(\text{clka}) / f(\text{clkb})$ soit 10 cycles

8. Mettez en place votre solution et testez-la en simulation. Si votre résultat de simulation n'est toujours pas valide, proposez une autre solution.



Afin d'allonger le signal *end_cycle_offon_a* sur 10 périodes de *clka*, on a rajouté un compteur de 10 cycles de *clka* composé de 2 multiplexeurs, 1 soustracteur, un comparateur et un registre. Le nouveau signal *end_cycle_long* contrôle l'update des 2 *led_driver*.

- code VHDL de l'architecture

Le fichier est top_2clk_long.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_2clk_long is
    generic (
        cible : integer := 100000000
    );
    port (
        clka      : in std_logic;
        clk_b     : in std_logic;
        resetn    : in std_logic;
        led0_r    : out std_logic;
        led0_b    : out std_logic;
        led0_g    : out std_logic;
        led1_r    : out std_logic;
        led1_b    : out std_logic;
        led1_g    : out std_logic
    );
end top_2clk_long;

architecture behavioral of top_2clk_long is
    type state is (rouge, bleu, vert);
    signal current_state : state;
    signal next_state : state;
    signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal end_cycle_onoff_a : std_logic:= '0';
    signal end_cycle_offon_a : std_logic:= '0';
    signal end_cycle_onoff_b : std_logic:= '0';
    signal end_cycle_offon_b : std_logic:= '0';
    signal nb_cycle : unsigned (3 downto 0) := (others => '0');
    signal end_count10 : std_logic:= '0';
    signal led_r : std_logic:= '0';
    signal led_b : std_logic:= '0';
    signal led_g : std_logic:= '0';
    signal stretch_count : integer range 0 to 10 := 0;
    signal end_cycle_long : std_logic ;
begin
    led_driver_a: entity work.led_driver(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clka,
            resetn => resetn,
            color_code => color_code,
            update => end_cycle_long,
            end_cycle_offon => end_cycle_offon_a,
            end_cycle_onoff => end_cycle_onoff_a,
            led_r => led0_r,
            led_g => led0_g,
            led_b => led0_b
        );
    led_driver_b: entity work.led_driver(behavioral)
        generic map (
            cible => cible
        )
        port map (
            clk => clk_b,
            resetn => resetn,
            color_code => color_code,
            update => end_cycle_long,
            end_cycle_offon => end_cycle_offon_b,
            end_cycle_onoff => end_cycle_onoff_b,
            led_r => led1_r,
            led_g => led1_g,
            led_b => led1_b
        );
end architecture;

```

Appel de led_driver_a
et connexion des signaux
d'entrée sortie
utilisation de clka
update par end_cycle_long
sortie sur led0

Appel de led_driver_b
et connexion des signaux
d'entrée sortie
utilisation de clk_b
update par end_cycle_long
sortie sur led1

```

process(clka,resetn)
begin
    if resetn = '0' then
        nb_cycle <= (others => '0');
    elsif rising_edge(clka) then
        end_count10<= '0';
        if end_cycle_onoff_a = '1' then
            if nb_cycle = 9 then
                end_count10 <= '1';
                nb_cycle <= "0000";
            else
                nb_cycle <= nb_cycle +1;
            end if;
        end if;
    end if;
end process;

```

Compteur du nombre de cycle et
génération du signal end_count10
sur clka

```

-- color fsm
process ( resetn, clka)
begin
    if resetn = '0' then
        current_state <= rouge;
    elsif rising_edge(clka) then
        current_state <= next_state;
    end if;
end process;
process (current_state, end_count10)
begin
    next_state <= current_state;
    case current_state is
        when rouge =>
            if end_count10 = '1' then
                next_state <= bleu;
            end if;
        when bleu =>
            if end_count10 = '1' then
                next_state <= vert;
            end if;
        when vert =>
            if end_count10 = '1' then
                next_state <= rouge;
            end if;
    end case;
end process;
color_code <=  "01" when current_state = rouge else
               "11" when current_state = bleu else
               "10" when current_state = vert else
               "00";

```

machine d'état color FSM
génération du code couleur color_code
sur clka

```

-- stretch update
process (resetn, clka)
begin
    if resetn = '0' then
        strech_count <= 0;
    elsif rising_edge(clka) then
        if end_cycle_offon_a = '1' then
            strech_count <= 10;
        elsif strech_count >0 then
            strech_count <= strech_count - 1;
        end if;
    end if;
end process ;
end_cycle_long <= '1' when end_cycle_offon_a = '1' or strech_count >0 else '0';
end behavioral;

```

Compteur qui permet de
rallonger le signal
end_cycle_offon_a
Création du signal
end_cycle_long

- Simulation et test bench

- a. Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn

Liste des signaux de sortie à vérifier : led0_r, led0_b, led0_g, led1_r, led1_b, led1_g,

Liste des signaux internes à vérifier : color_code,

end_cycle_onoff_a, end_cycle_offon_a,

end_cycle_onoff_b, end_cycle_offon_b,

end_cycle_long, strech_count

nb_cycle, end_count10,

led_driver_a: led_color_a; led_driver_a: end_counter_a,

led_driver_b: led_color_b; led_driver_b: end_counter_b

Listes des états à vérifier: current_state, led_driver_a: current_onoff_state, led_driver_b: current_onoff_state

L'horloge clka est définie à 250Mhz

L'horloge clkb est définie à 50Mhz

Note : Les simulations ont été faite avec une cible de **60** soit un temps de comptage du compteur unitaire de 240ns

La cible a été augmentée de 20 à 60 afin de pouvoir bien identifier le comportement du signal end_cycle_long qui va durer 10 cycles et les cycles de éclairage des leds qui sont donc passés de 20 cycles à 60 cycles.

- b. Déroulé de la validation et du test bench

La validation à la même déroulé que l'architecture précédente. on rajoute le vérifications sur le signaux end_cycle_long et strech_count.

- 1. Test du reset au démarrage

- 1.1. Les entrées seront définies comme suit :

- **resetn = 0**

- 1.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g= 0

- led1_r = 0 , led1_b = 0 , led1_g= 0

- 1.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car au reset l'état de départ de color FSM est rouge

- end_cycle_onoff_a = 0 , led_driver: end_counter car led_driver resetté

- end_cycle_offon_a = 1 car il suit led_driver_a:fisrt_on = 1 quand current_onoff_state_a = led_off

- end_cycle_onoff_b = 0 , led_driver: end_counter car led_driver resetté

- end_cycle_offon_b = 1 car il suit led_driver_b:fisrt_on = 1 quand current_onoff_state_b = led_off

- **end_cycle_long = 1 car il suit end_cycle_offon_a**

- led_driver_a: led_color = off car led_driver_a resetté

- led_driver_a: current_onoff_state = led_off car led_driver_a resetté

- led_driver_b: led_color = off car led_driver_b resetté

- led_driver_b: current_onoff_state = led_off car led_driver_b resetté

- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée

- 1.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r =0, led0_b=0 , led0_g = 0 après 2*cible*periode(clka)+1ns

- on teste les 3 sorties de led1 : led1_r =0, led1_b=0 , led1_g = 0 après 2*cible*periode(clkb)+1ns

- on attend cible*period(clkb) +1ns pour lancer la suite

- 2. Test du démarrage après reset

- 2.1. Les entrées seront définies comme suit :

- **resetn = 1**

- 2.2. Les sorties seront vérifiées

- **led0_r = 1** , led0_b = 0 , led0_g= 0 au front de clka suivant

- **led1_r =1** , led1_b = 0 , led1_g= 0 au front de clkb suivant

- 2.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0

- led_driver_a: current_onoff_state = led_on au front de clka suivant

- led_driver_b: current_onoff_state = led_on au front de clkb suivant
- **end_cycle_long = 1 pendant 10* periode(clka)**
- led_driver_a: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_a soit après cible *periode(clka) = 80ns
- led_driver_b: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_b soit après cible *periode(clkb) = 400ns
- pour les driver a et b : end_cycle_onoff = 0 suit led_driver: end_counter quand led_driver: current_onoff_state = led_on
- pour les driver a et b : **end_cycle_offon = 0** au front de leur clk suivant car il est à 0 quand current_onoff_state = led_on
- pour les driver a et b : **led_driver: led_color = rouge** au front de leur clk suivant
- nb_cycle = 0 et commence à s'incrémenter sur end_cycle_onoff_a
- end_count10 = 0 tant que nb_cycle < 9

2.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=1, led0_b=0, led0_g= 0 après periode(clka)+1ns
- on teste les 3 sorties de led1 : led1_r=1, led1_b=0, led1_g= 0 après periode(clkb)+1ns
- on lance la suite

3. Test du fonctionnement après démarrage

3.1. Les entrées seront définies comme suit :

- resetn = 1

3.2. Les sorties seront vérifiées

- la led 0 vont clignoter 10 fois sur une couleur avant de passer à la couleur suivante
- le led 1 vont clignoter 2 fois sur une couleur avant de passer à la couleur suivante sur la fin de cycle de la led 0 .
(nb_clignotement_led1 = nb_clignotement_led0 * clkb/clka = 2)

3.3. Les signaux et états internes à vérifier

- current_state = rouge / bleu/ vert tous les 10*2*cycle*periode(clka) avec une mise à jour quand end_cycle_onoff = 1
- color_code = 01/11/10 tous les 10*2*cycle*periode(clka) avec une mise à jour quand end_cycle_onoff_a = 1
- pour les driver a et b : led_driver: current_onoff_state = led_on/led_off tous les cycle*periode(clka et b)
- pour les driver a et b : led_driver: end_counter = 1 pendant periode(clk) tous les cycle*periode(clka et b)
- pour les driver a et b : end_cycle_onoff = 1 pendant periode(clk) tous les 2*cycle*periode(clka et b) quand current_onoff_state = led_on
- pour les driver a et b : end_cycle_offon = 1 pendant periode(clk) tous les 2*cycle*periode(clk) quand current_onoff_state = led_off
- **end_cycle_long = 1 pendant 10* periode(clka) dès que end_cycle_offon_a passe à 1**
- pour les driver a et b led_driver: led_color = 01/11/10 tous les 10*2*cycle*periode(clka et b) avec une mise à jour quand end_cycle_offon = 1
- nb_cycle compte de 0 à 9 il s'incrémente tous les 2*cycle*periode(clka) quand end_cycle_onoff_a = 1
- end_count10 = 1 quand end_cycle = 9 pendant periode(clka)

3.4. Sur le test bench

- on attend le premier allumage de la led en rouge
- on teste les 3 sorties de led0 : led0_r=0/1, led0_b=0/1, led0_g= 0/1 après chaque période de (2*cible)*periode(clka)
- on teste les 3 sorties de led1 : led1_r=0/1, led1_b=0/1, led1_g= 0/1 après chaque période de (2*cible)*periode(clkb)
- sur la led0 on vérifie successivement les sortie led0 en rebouclant le test 10 fois pour chaque couleur
- sur la led1 on sort de la boucle de comptage de cycle de la led1 après 2 cycles sur clkb . on aura compté que 2 cycles
- Le test bench testera 7 changements de couleur avant de continuer.

4. Test du reset en fonctionnement

4.1. Les entrées seront définies comme suit :

- **resetn = 0**

4.2. Les sorties seront vérifiées

- led0_r = 0, led0_b = 0, led0_g= 0
- led1_r = 0, led1_b = 0, led1_g= 0

4.3. Les signaux et états internes à vérifier

Tous les signaux sont resettée de façon asynchrone dès que resetn = 0

- `current_state = rouge` et `color_code = "01"` car au reset l'état de départ de color FSM est rouge
- `end_cycle_onoff_a = 0` , led_driver: end_counter car led_driver resetté
- `end_cycle_offon_a = 1` car il suit led_driver_a: `fisrt_on = 1` quand `current_onoff_state_a = led_off`
- **end_cycle_long = 1 car il suit end_cycle_offon_a**
- `end_cycle_onoff_b = 0` , led_driver: end_counter car led_driver resetté
- `end_cycle_offon_b = 1` car il suit led_driver_b: `fisrt_on = 1` quand `current_onoff_state_b = led_off`
- led_driver_a: `led_color = off` car led_driver_a resetté
- led_driver_a: `current_onoff_state = led_off` car led_driver_a resetté
- led_driver_b: `led_color = off` car led_driver_b resetté
- led_driver_b: `current_onoff_state = led_off` car led_driver_b resetté
- `nb_cycle = 0` , `end_count10 = 0` car la bascule Dcount est resettée

4.4. Sur le test bench

- on teste les 3 sorties de led0 : `led0_r=0` , `led0_b=0` , `led0_g=0` après 1ns
- on teste les 3 sorties de led1 : `led1_r=0` , `led1_b=0` , `led1_g=0` après 1ns
- on attend $2 \times \text{cible} \times \text{period}(\text{clkb}) + 1\text{ns}$ pour lancer la suite

5. Test du démarrage après reset

5.1. Les entrées seront définies comme suit :

- **`resetn = 1`**

5.2. Les sorties seront vérifiées

- **`led0_r = 1`** , `led0_b = 0` , `led0_g = 0` au front de `clka` suivant
- **`led1_r = 1`** , `led1_b = 0` , `led1_g = 0` au front de `clkb` suivant

5.3. Les signaux et états internes à vérifier

- `current_state = rouge` et `color_code = "01"` car l'état de départ de color FSM est rouge tant que `end_count10 = 0`
- led_driver_a: `current_onoff_state = led_on` au front de `clka` suivant
- led_driver_b: `current_onoff_state = led_on` au front de `clkb` suivant
- led_driver_a: `end_counter = 0` jusqu' à la fin d' cycle de comptage de `counter_unit` du led_driver_a soit après $\text{cible} \times \text{periode}(\text{clka}) = 80\text{ns}$
- led_driver_b: `end_counter = 0` jusqu' à la fin d' cycle de comptage de `counter_unit` du led_driver_b soit après $\text{cible} \times \text{periode}(\text{clkb}) = 400\text{ns}$
- pour les driver a et b : `end_cycle_onoff = 0` suit led_driver: end_counter quand led_driver: `current_onoff_state = led_on`
- pour les driver a et b : `end_cycle_offon = 0` au front de leur clk suivant car il est à 0 quand `current_onoff_state = led_on`
- **end_cycle_long = 1 pendant $10 \times \text{periode}(\text{clka})$**
- pour les driver a et b : led_driver: `led_color = rouge` au front de leur clk suivant
- `nb_cycle = 0` et commence à s'incrémenter sur `end_cycle_onoff_a`
- `end_count10 = 0` tant que `nb_cycle < 9`

5.4. Sur le test bench

- on teste les 3 sorties de led0 : `led0_r=1` , `led0_b=0` , `led0_g=0` après $\text{periode}(\text{clka})+1\text{ns}$
- on teste les 3 sorties de led1 : `led1_r=1` , `led1_b=0` , `led1_g=0` après $\text{periode}(\text{clkb})+1\text{ns}$
- le test est terminé

• Test bench

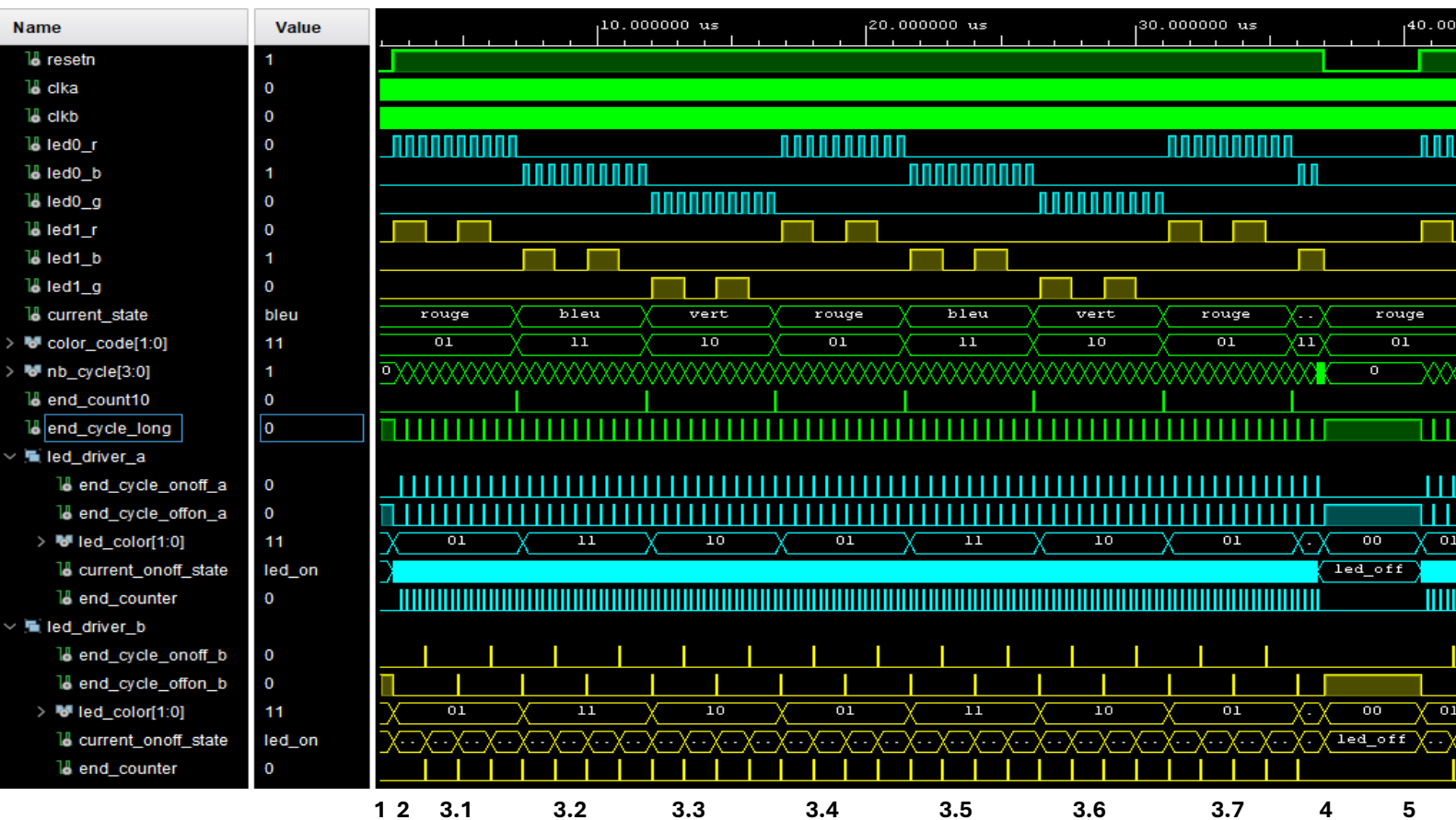
Le fichier de test bench est `tb_top_2clk_long.vhd`

Il est identique à `tb_top_2clk_.vhd`

- Résultat des test automatiques du test_bench

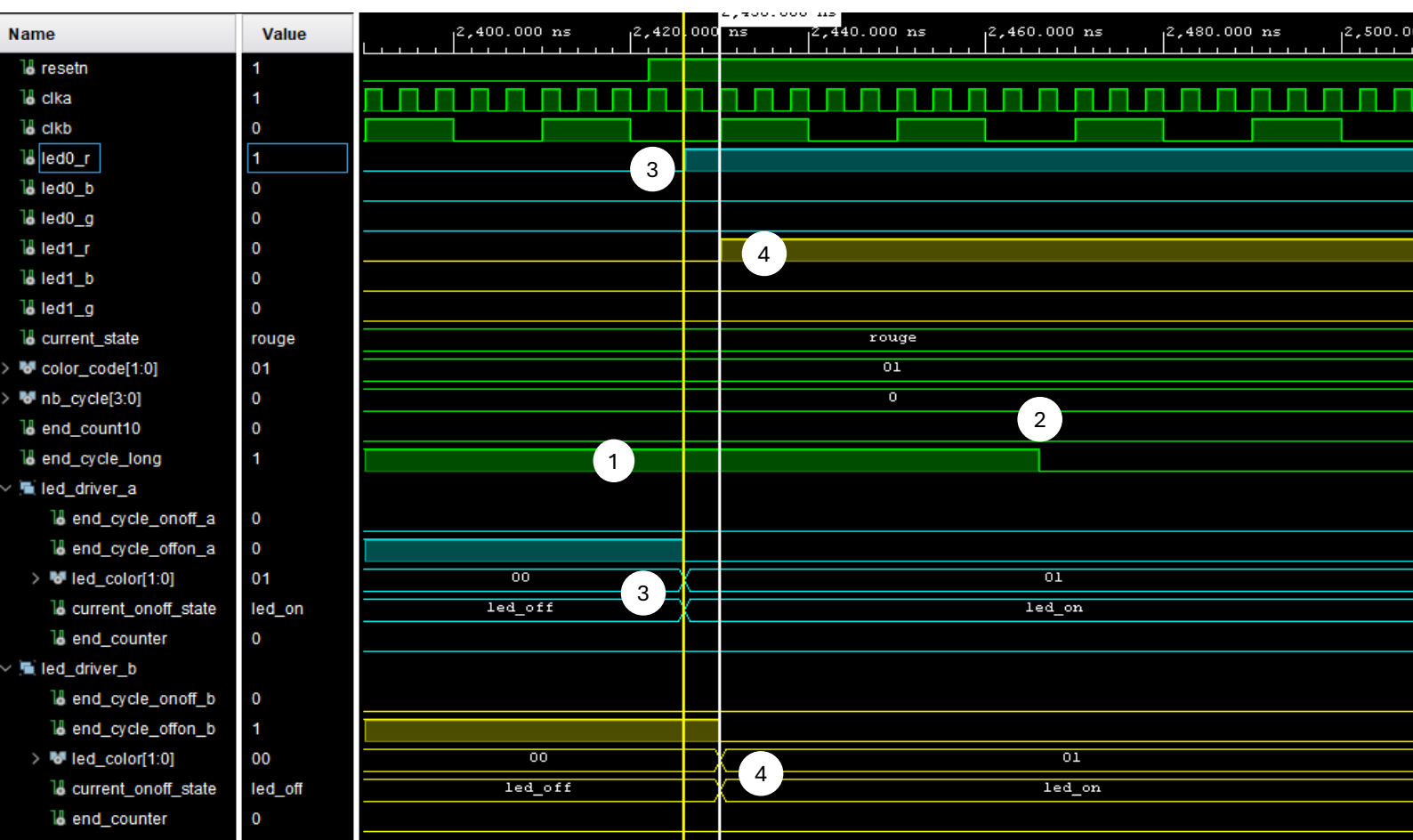
```
# run 46000ns
Note: PASS : TEST reset : led0 est eteinte OK
Time: 481 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST reset : led1 est eteinte OK
Time: 2401 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST demarrage : led1 est eteinte OK
Time: 2423 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST demarrage : led0 est eteinte OK
Time: 2423 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 7227 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence rouge
Time: 7231 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 12027 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence bleu
Time: 12031 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 16827 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence vert
Time: 16831 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 21627 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence rouge
Time: 21631 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 26427 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence bleu
Time: 26431 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 31227 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence vert
Time: 31231 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 36027 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS LED1 allumee 2 fois pour la sequence rouge
Time: 36031 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST reset : la led0 est eteinte OK
Time: 36983 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST reset : la led1 est eteinte OK
Time: 36983 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST demarrage : la led1 est eteinte OK
Time: 40584 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST demarrage : la led0 est eteinte OK
Time: 40584 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST demarrage : la led0 est rouge apres 1 periode clka OK
Time: 40588 ns Iteration: 0 Process: /tb_top_2clk_long/line__112 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: PASS : TEST demarrage : la led1 est rouge apres 1 periode clkb OK
Time: 40604 ns Iteration: 0 Process: /tb_top_2clk_long/line__208 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
Note: Nombre de test FAIL : 0
Time: 42024 ns Iteration: 0 Process: /tb_top_2clk_long/line__84 File: D:/Vivado/TP5/tb_top_2clk_long.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_2clk_long_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 46000ns
) launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:11 . Memory (MB): peak = 1989.605 ; gain = 0.000
```

- Validation fonctionnelle



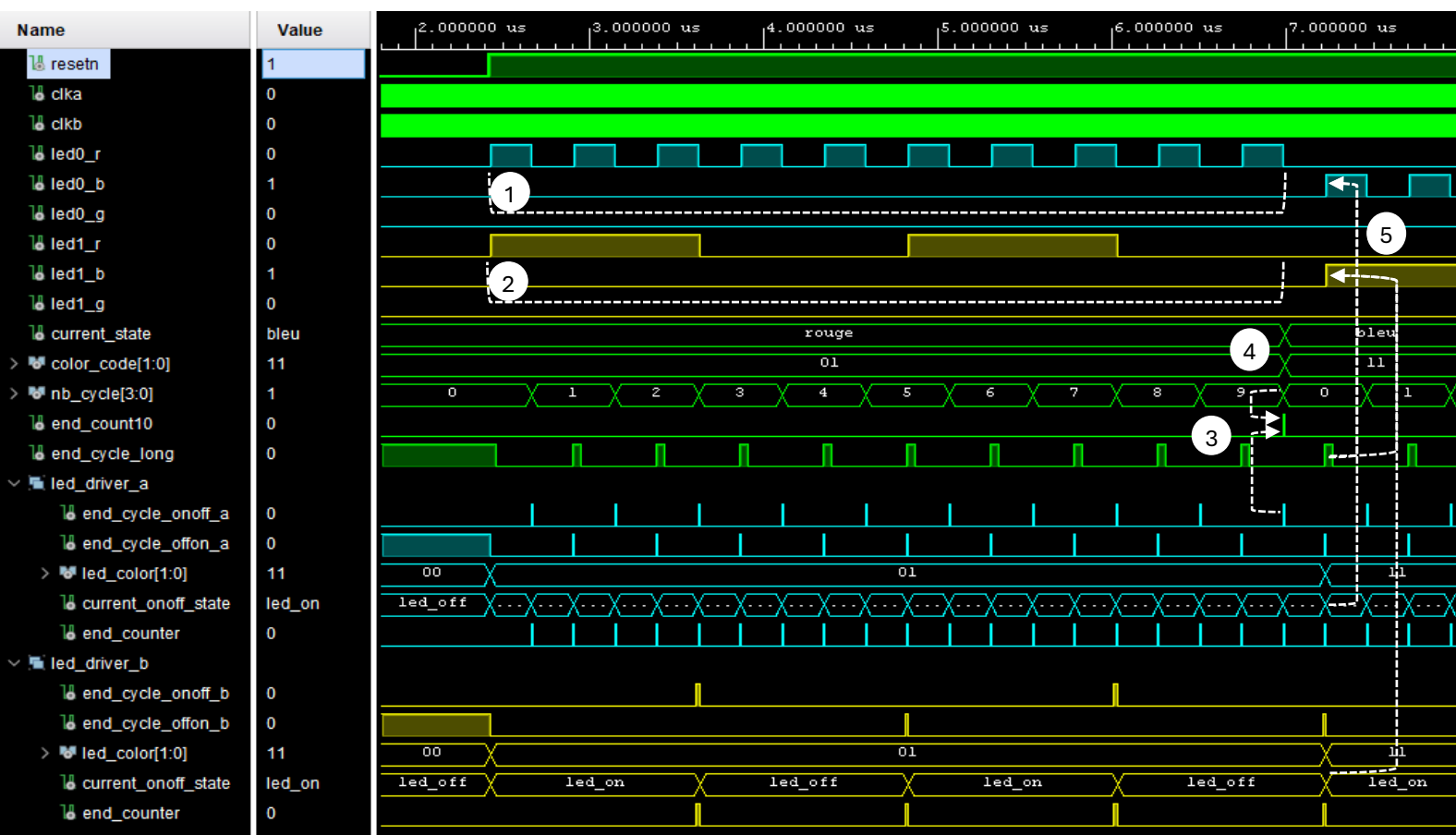
- 1- Test du reset: les leds led0 et led1 sont éteintes (led0_r = 0 , led0_b=0 , led0_g= 0 , led1_r= 0 , led1_b= 0 , led1_g= 0)
 La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')
 La FSM (current_onoff_state)est dans l'état led_off
 end_cycle_onoff= 0 resetté
end_cycle_long = 1
- 2- Démarrage après reset:
 La led0 commence à clignoter en rouge (led0_r = 1 , led0_b= 0 , led0_g= 0)
La led1 commence à clignoter en rouge (led1_r = 1 , led1_b= 0 , led1_g= 0)
 La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')
 Le led_driver_a compteur_unit commence à compter (end_counter = 0/1 tous les cible* periode(clka))
 Le led_driver_b compteur_unit commence à compter (end_counter = 0/1 tous les cible* periode(clkb))
 La led_driver_a FSM(current_onoff_state) = led_on/led_off à chaque de cycle de comptage (end_counter_a = 1)
 La led_driver_b FSM(current_onoff_state) = led_on/led_off à chaque de cycle de comptage (end_counter_b = 1)
 end_cycle_onoff_a suit end_counter_a quand la led est on
 end_cycle_offon_a suit end_counter_a quand la led est off
 end_cycle_onoff_a suit end_counter_b quand la led est on
 end_cycle_offon_a suit end_counter_b quand la led est off
end_cycle_long = 1
 led_driver_a: led_color rouge
led_driver_b: led_color rouge
- 3- Cycles de 10 clignotements sur les leds
 La led0 clignote successivement en rouge, bleu puis vert avant de reboucler sur le rouge
 La led0 clignote 10 fois sur chaque cycle.
 La led1 clignote 2 fois sur chaque cycle. (10 cycles *250/50 = 2)
- 4- Reset en cours de fonctionnement
 On se retrouve dans le même état que l'étape 1.
- 5- Redémarrage après reset
 Le système redémarre sur 10 cycles en rouge comme sur l'étape 2.

- Zoom sur le démarrage



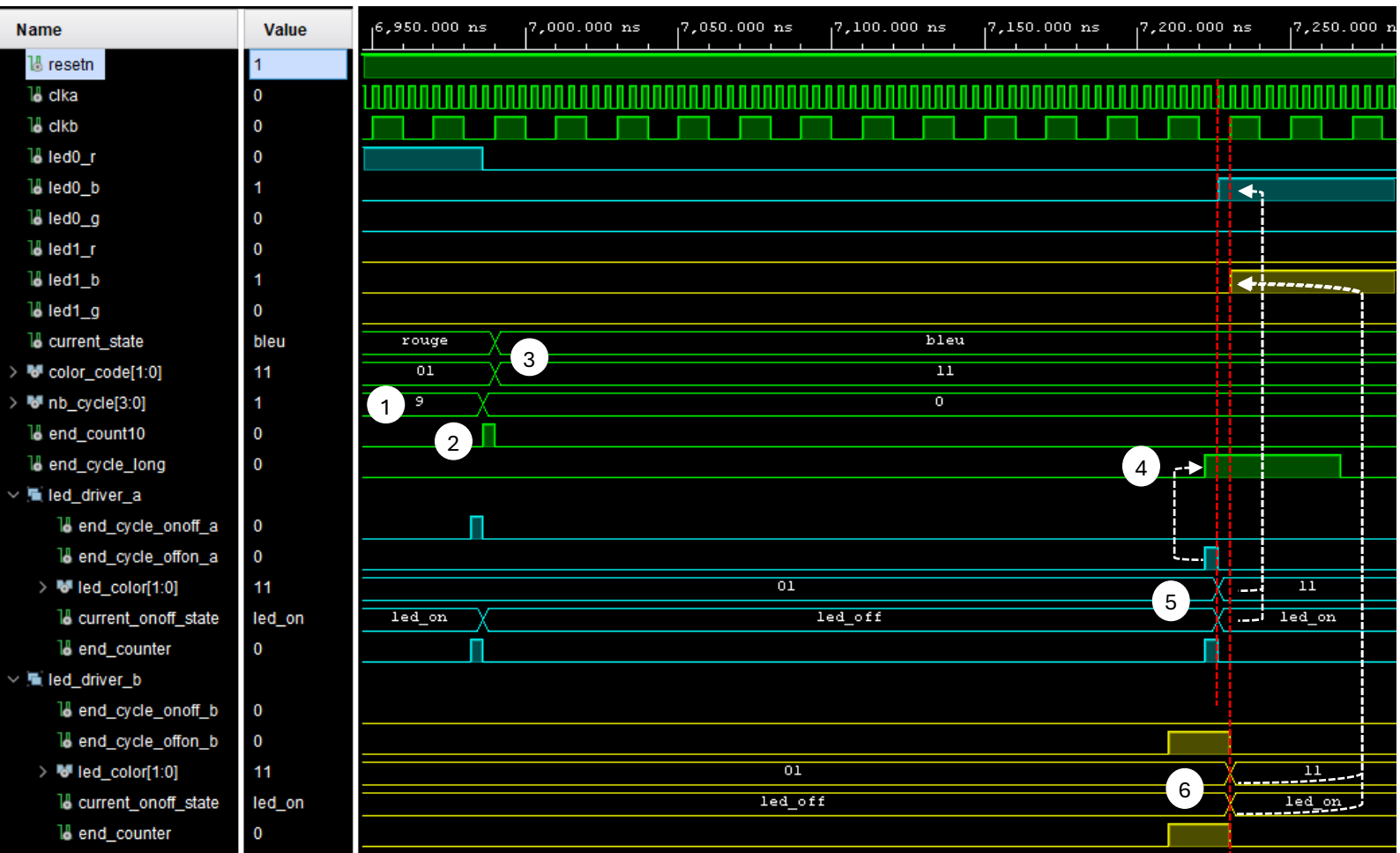
- 1- Au démarrage (resetn = 1), le signal end_cycle_off_on_a reste à 1 jusqu'à ce qu'au prochain front montant de clka
- 2- end_cycle_long suit end_cycle_off_on_a et garde sa valeur sur 10 périodes de clka
- 3- Led_driver_a fait son update_a = end_cycle_long sur le front montant de clka (led_color_a passe à rouge) et met à jour la couleur de la led0 (led0_r = 1)
- 4- Led_driver_b fait son update_b = end_cycle_long sur le front montant de clkb (led_color_b passe à rouge) et met à jour la couleur de la led1 (led1_r = 1)

- Zoom sur les cycles suivants



- 1- La led0 effectue 10 cycles on/off avant de changer de couleur quand end_count10 passe à 1
- 2- La led0 effectue 2 cycles on/off avant de changer de couleur quand end_count10 passe à 1
- 3- end_counth10 passe à 1 pendant une periode de clka quand nb_cycles = 9 et end_cycle_onoff_a = 1
- 4- le code couleur est mis à jour dans la color fsm
- 5- Le changement de couleur sur les leds d'effectue sur le cycle off to on suivant quand end_cycle_long = 1

- Zoom sur un changement de couleur



1- Le compteur de cycle a compté 10 cycles : nb_cycle = 9

2- Sur le cycle suivant on to off de la led 0 (end_cycle_onoff_a = 1 pendant une période(clka) le signal end_count10 passe à 1 pendant une période(clka)

3- Le code couleur est mis à jour quand end_count10 passe à 1 current_state change d'état, color_code est mis à jour.

4- Au changement d'état off to on de la led0 (end_cycle_offon_a = 1), le signal end_cycle_long passe à 1 pendant 1+10 période (clka)

5- La couleur de led0 est mise à jour sur le front de clka suivant

6- La couleur de led1 est mise à jour sur le front de clkb suivant

9. Pour générer plusieurs horloges vous aurez besoin d'une PLL. Trouvez quel est le nom de l'IP PLL de l'IP Catalog de Vivado puis ajoutez là à votre architecture.

La fréquence de l'horloge en entrée de la PLL est de **125MHz**. Les deux horloges en sortie doivent être à 50MHz et 250MHz.

L'IP Clocking Wizard est utilisé pour configurer la PLL

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

IP Symbol Resource

Show disabled ports

Component Name: clk_pll

Clocking Options Output Clocks Port Renaming PLLE2 Settings Summary

Clock Monitor

☐ Enable Clock Monitoring

Primitive

☐ MMCM ☒ PLL

Clocking Features **Jitter Optimization**

☒ Frequency Synthesis ☐ Minimize Power ☒ Balanced

☒ Phase Alignment ☐ Minimize Output Jitter

☐ Dynamic Reconfig ☐ Maximize Input Jitter filtering

☐ Safe Clock Startup

Dynamic Reconfig Interface Options

☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

Input Clock Information

	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☐	Primary	clk_in1	125.000	19.000 - 800.000	UI	0.010	Single ended clock capable pin
☐	Secondary	clk_in2	100.001	100.000 - 200.000		0.010	Single ended clock capable pin

Une clk d'entrée est choisie à 125Mhz.

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

IP Symbol Resource

Show disabled ports

Component Name: clk_pll

Clocking Options Output Clocks Port Renaming PLLE2 Settings Summary

The phase is calculated relative to the active input clock.

Output Clock	Port Name	Output Freq (MHz) Requested	Actual	Phase (degrees) Requested	Actual	Duty Cycle (%) Requested	Actual	Drives
☒ clk_out1	clk_out1	50.000	50.00000	0.000	0.000	50.000	50.0	BUFG
☒ clk_out2	clk_out2	250.000	250.00000	0.000	0.000	50.000	50.0	BUFG
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG

☐ USE CLOCK SEQUENCING

Clocking Feedback

Source **Signaling**

☒ Automatic Control On-Chip ☐ Automatic Control Off-Chip ☐ User-Controlled On-Chip ☐ User-Controlled Off-Chip

☒ Single-ended ☐ Differential

Enable Optional Inputs / Outputs for MMCM/PLL **Reset Type**

☒ reset ☐ power_down ☐ Active High ☒ Active Low

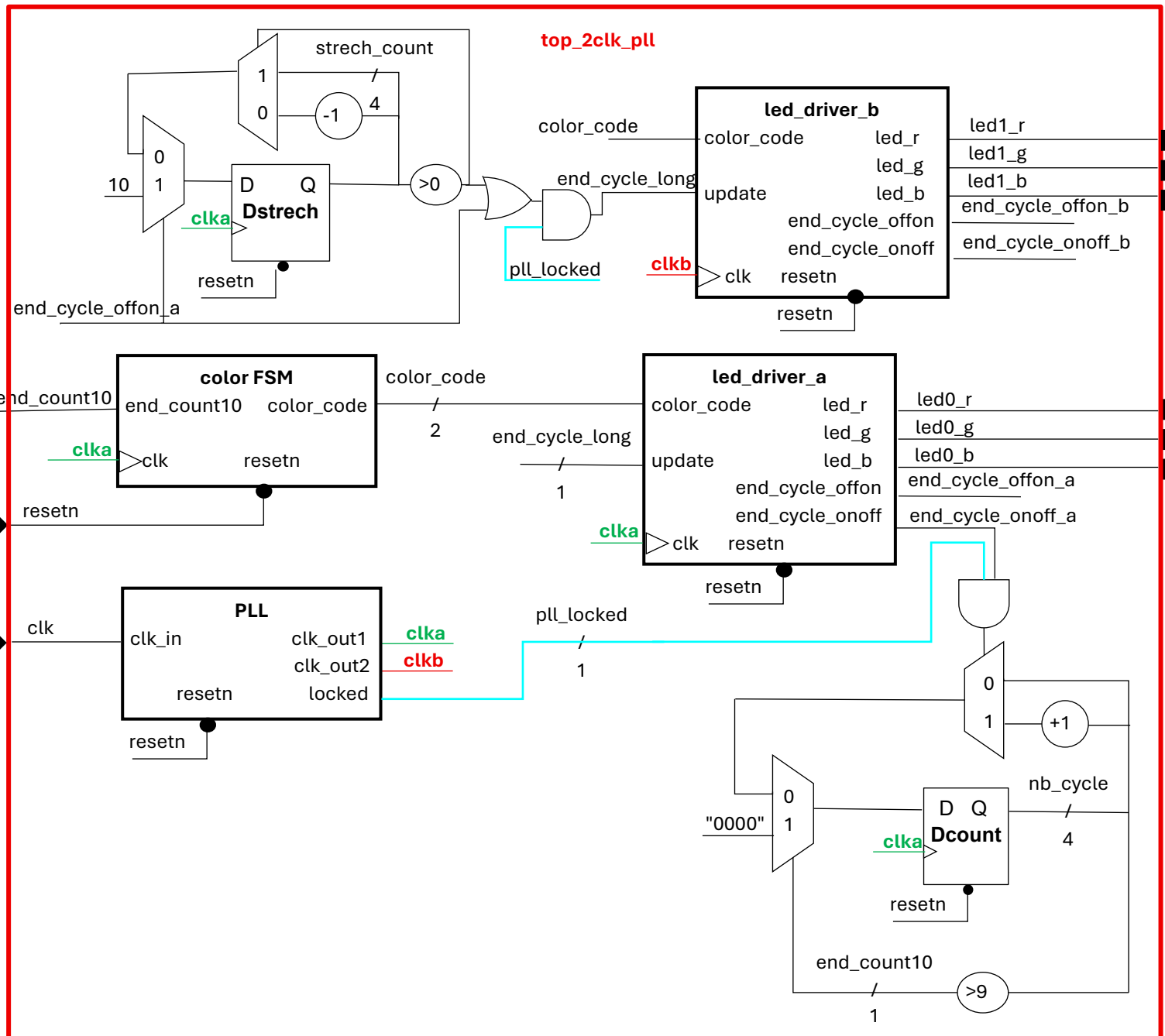
☒ locked

Les fréquences de sortie seront de 50Mhz et 250Mhz

On choisit le reset actif low

On sort l'information locked qui nous indiquera quand la PLL sera lockée

- Architecture RTL avec la PLL



La PLL a été rajoutée à l'architecture. Un seul signal de clock en entrée (clk) génère les deux signaux de clock clka et clkb. Le signal pll_locked en sortie de la PLL indique que les signaux de clock sont bien établis, il passe à 1 quand le PLL est lockée.

Afin de garantir un fonctionnement du circuit quand les horloges sont valides: l'update des led_driver (end_cycle_long) et le signal qui actionne le color FSM (end_count10) sont mis à zéro tant que le PLL n'est pas correctement démarrée.

Note: Cette architecture a été modifiée uniquement au niveau top_level afin de rajouter la PLL et contrôler le démarrage du circuit après le reset et après le **démarrage de la PLL**.

Si on veut un démarrage propre du circuit, c'est-à-dire attendre le démarrage de la PLL avant de lancer toute la logique, il faudrait modifier les blocks dans la hiérarchie:

- Dans le led_driver: le signal first_on qui permet de commencer le cycle avec une LED allumée passe actuellement à 0 après le reset. Il faudrait le conditionner aussi sur le signal pll_locked
- Dans le compteur unitaire : actuellement le compteur unitaire démarre dès le relâchement du reset. Il faudrait pouvoir autoriser le démarrage du compteur sur le signal pll_locked

Ces deux modifications n'ont pas été faites sur le code et la validation qui va suivre, elle seront faite dans un deuxième temps.

- code VHDL de l'architecture

Le fichier est top_2clk_pll.vhd

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_2clk_pll is
    generic (
        cible : integer := 100000000
    );
    port (
        clk          : in std_logic;
        resetn       : in std_logic;
        led0_r       : out std_logic;
        led0_b       : out std_logic;
        led0_g       : out std_logic;
        led1_r       : out std_logic;
        led1_b       : out std_logic;
        led1_g       : out std_logic
    );
end top_2clk_pll;

architecture behavioral of top_2clk_pll is
    type state is (rouge, bleu, vert);
    signal current_state : state;
    signal next_state : state;
    signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal end_cycle_onoff_a : std_logic := '0';
    signal end_cycle_offon_a : std_logic := '0';
    signal end_cycle_onoff_b : std_logic := '0';
    signal end_cycle_offon_b : std_logic := '0';
    signal nb_cycle : unsigned (3 downto 0) := (others => '0');
    signal end_count10 : std_logic := '0';
    signal led_r : std_logic := '0';
    signal led_b : std_logic := '0';
    signal led_g : std_logic := '0';
    signal stretch_count : integer range 0 to 10 := 0;
    signal end_cycle_long : std_logic;
    signal clka : std_logic := '0';
    signal clk_b : std_logic := '0';
    signal pll_locked : std_logic := '0';

    component clk_pll
    port (
        clk_out1 : out std_logic;
        clk_out2 : out std_logic;
        resetn   : in  std_logic;
        locked   : out std_logic;
        clk_in1  : in  std_logic
    );
    end component;

begin
    pll: clk_pll
        port map(
            clk_out1 => clk_b,
            clk_out2 => clka,
            resetn   => resetn,
            locked   => pll_locked,
            clk_in1  => clk
        );

```

Plus qu'une seule horloge en entrée : clk

instanciation de la PLL et connexion des signaux d'entrée sortie connexion des sorties sur les signaux d'horloge existants
 clk_b : clock à 50Mhz
 clka : clock à 250Mhz
 Le signal pll_locked sera utilisé plus bas pour contrôler les compteurs de cycle et l'update des led_driver

```

}
led_driver_a: entity work.led_driver(behavioral)
    generic map (
        cible => cible
    )
    port map (
        clk => clka,
        resetn => resetn,
        color_code => color_code,
        update => end_cycle_long,
        end_cycle_offon => end_cycle_offon_a,
        end_cycle_onoff => end_cycle_onoff_a,
        led_r => led0_r,
        led_g => led0_g,
        led_b => led0_b
    );
}
led_driver_b: entity work.led_driver(behavioral)
    generic map (
        cible => cible
    )
    port map (
        clk => clk_b,
        resetn => resetn,
        color_code => color_code,
        update => end_cycle_long,
        end_cycle_offon => end_cycle_offon_b,
        end_cycle_onoff => end_cycle_onoff_b,
        led_r => led1_r,
        led_g => led1_g,
        led_b => led1_b
    );
}
process(clka, resetn)
begin
    if resetn = '0' then
        nb_cycle <= (others => '0');
    elsif rising_edge(clka) then
        end_count10 <= '0';
        if (end_cycle_onoff_a = '1' and pll_locked = '1') then
            if nb_cycle = 9 then
                end_count10 <= '1';
                nb_cycle <= "0000";
            else
                nb_cycle <= nb_cycle + 1;
            end if;
        end if;
    end if;
end process;
}

```

Appel de led_driver_a
et connexion des signaux
d'entrée sortie
utilisation de clka
update par end_cycle_long
sortie sur led0

Appel de led_driver_b
et connexion des signaux
d'entrée sortie
utilisation de clk_b
update par end_cycle_long
sortie sur led1

Ajout de la condition
pll_locked sur le compteur
de 10 cycles afin que le
comptage ne commence
pas avant l'établissement
des horloges

```

-- color fsm
process ( resetn, clka)
begin
    if resetn = '0' then
        current_state <= rouge;
    elsif rising_edge(clka) then
        current_state <= next_state;
    end if;
end process;
process (current_state, end_count10)
begin
    next_state <= current_state;
    case current_state is
        when rouge =>
            if end_count10 = '1' then
                next_state <= bleu;
            end if;
        when bleu =>
            if end_count10 = '1' then
                next_state <= vert;
            end if;
        when vert =>
            if end_count10 = '1' then
                next_state <= rouge;
            end if;
        end case;
    end process;
    color_code <= "01" when current_state = rouge else
                  "11" when current_state = bleu else
                  "10" when current_state = vert else
                  "00";

-- stretch update
process (resetn, clka)
begin
    if resetn = '0' then
        strech_count <= 0;
    elsif rising_edge(clka) then
        if end_cycle_offon_a = '1' then
            strech_count <= 10;
        elsif strech_count > 0 then
            strech_count <= strech_count - 1;
        end if;
    end if;
end process ;
end_cycle_long <= '1' when (end_cycle_offon_a = '1' or strech_count > 0) and pll_locked = '1' else '0';

end behavioral;

```

Ajout de la condition
 pll_locked sur le signal
 d'update des led_drivers
 pour ne pas mettre à jour
 tant que la pll n'est pas
 lockée.

- Simulation et test bench

- a. Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn

Liste des signaux de sortie à vérifier : led0_r, led0_b, led0_g, led1_r, led1_b, led1_g,

Liste des signaux internes à vérifier : color_code,

pll_locked, clka, clk

end_cycle_onoff_a, end_cycle_offon_a,

end_cycle_onoff_b, end_cycle_offon_b,

end_cycle_long, stretch_count

nb_cycle, end_count10,

led_driver_a: led_color_a; led_driver_a: end_counter_a,

led_driver_b: led_color_b; led_driver_b: end_counter_b

Listes des états à vérifier: current_state, led_driver_a: current_onoff_state, led_driver_b: current_onoff_state

L'horloge clk est définie à 125Mhz

Note : Les simulations ont été faites avec une cible de **60** soit un temps de comptage du compteur unitaire de 240ns

La cible a été augmentée de 20 à 60 afin de pouvoir bien identifier le comportement du signal end_cycle_long qui va durer 10 cycles et les cycles de éclairage des leds qui sont donc passés de 20 cycles à 60 cycles.

- b. Déroulé de la validation et du test bench

- 1. Test du reset au démarrage

- 1.1. Les entrées seront définies comme suit :

- **resetn = 0**

- 1.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g = 0

- led1_r = 0 , led1_b = 0 , led1_g = 0

- 1.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car au reset l'état de départ de color FSM est rouge

- end_cycle_onoff_a = 0 , led_driver: end_counter car led_driver resetté

- end_cycle_offon_a = 1 car il suit led_driver_a: fisrt_on = 1 quand current_onoff_state_a = led_off

- end_cycle_onoff_b = 0 , led_driver: end_counter car led_driver resetté

- end_cycle_offon_b = 1 car il suit led_driver_b: fisrt_on = 1 quand current_onoff_state_b = led_off

- **end_cycle_long = 0**

- led_driver_a: led_color = off car led_driver_a resetté

- led_driver_a: current_onoff_state = led_off car led_driver_a resetté

- led_driver_b: led_color = off car led_driver_b resetté

- led_driver_b: current_onoff_state = led_off car led_driver_b resetté

- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée

- **pll_locked = 0 , clka = 0 , clk = 0** : la pll est resettée

- 1.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0 , led0_g = 0 après 2*cible*periode(clka)+1ns

- on teste les 3 sorties de led1 : led1_r=0, led1_b=0 , led1_g = 0 après 2*cible*periode(clkb)+1ns

- on attend cible*period(clkb) +1ns pour lancer la suite

- 2. Test du démarrage après reset

- 2.1. Les entrées seront définies comme suit :

- **resetn = 1**

- 2.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g = 0 **tant que pll_locked = 0;**

- led1_r = 0 , led1_b = 0 , led1_g = 0 **tant que pll_locked = 0;**

- 2.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0

- led_driver_a: current_onoff_state = led_on au front de clka suivant

- led_driver_b: current_onoff_state = led_on au front de clk suivant

- **end_cycle_long = 0 tant que pll_locked = 0;**
- **end_count10 = 0 tant que pll_locked = 0;**
- led_driver_a: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_a soit après cible *periode(clka)
Note le premier comptage ne sera pas exactement de 240ns car clka ne sera pas complètement établie au reset
- led_driver_b: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_b soit après cible *periode(clkb)
Note le premier comptage ne sera pas exactement de 1200ns car clka ne sera pas complètement établie au reset
- pour les driver a et b : end_cycle_onoff = 0 suit led_driver: end_counter quand led_driver: current_onoff_state = led_on
- pour les driver a et b : **end_cycle_offon = 0** au front de leur clk suivant car il est à 0 quand current_onoff_state = led_on
- pour les driver a et b : **led_driver: led_color = '00' tant que end_cycle_long = 0**
- **nb_cycle = 0 tant que pll_locked = 0;**
- **pll_locked passe à 1 après un certain temps (je n'ai pas trouvé d'informations sur le lock time dans les documentation de la PLL)**
- **clka et clkb démarrent, elle seront établies quand pll_locked = 1**

2.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0 , led0_g= 0 après periode(clka)+1ns
- on teste les 3 sorties de led1 : led1_r=0, led1_b=0 , led1_g= 0 après periode(clkb)+1ns
- on attend que la led1 s'allume pour lancer la suite car on n'a pas d'informations sur le temps mis par la PLL pour démarrer

3. Test du fonctionnement après démarrage de la PLL

3.1. Les entrées seront définies comme suit :

- **resetsn = 1**

3.2. Les sorties seront vérifiées

- **led0_r = 1** , led0_b = 0 , led0_g= 0 au front de clka suivant
- **led1_r=1** , led1_b = 0 , led1_g= 0 au front de clkb suivant

3.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0
- led_driver_a: current_onoff_state = led_on au front de clka suivant
- led_driver_b: current_onoff_state = led_on au front de clkb suivant
- **end_cycle_long = 1 pendant 10* periode(clka) dès que end_cycle_offon_a passe à 1**
- **end_count10 = 1 quand end_cycle = 9 pendant periode(clka)**
- led_driver_a: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_a soit après cible *periode(clka)
Note le premier comptage ne sera pas exactement de 240ns car la compteur a commencé à s'incrémenter sur un horloge pas parfaitement établie
- led_driver_b: end_counter = 0 jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_b soit après cible *periode(clkb)
Note le premier comptage ne sera pas exactement de 1200ns car la compteur a commencé à s'incrémenter sur un horloge pas parfaitement établie
- pour les driver a et b : end_cycle_onoff = 0 suit led_driver: end_counter quand led_driver: current_onoff_state = led_on
- pour les driver a et b : **end_cycle_offon = 0** suit led_driver: end_counter quand led_driver: current_onoff_state = led_off
- pour les driver a et b : **led_driver: led_color = '01' quand end_cycle_long = 1**
- **nb_cycle compte de 0 à 9 il s'incrémente tous les 2*cible*periode(clka) quand end_cycle_onoff_a = 1**
- **pll_locked =1**
- **clka et clkb sont établies , periode (clka) = 4ns, periode(clkb)= 20ns**

3.4. Sur le test bench

- on ne peut pas faire de test automatique au moment du démarrage de la PLL car il n'y a aucun signaux sur le top design qui indique cet état. Les test automatiques se feront à l'étape suivante au guettant le premier allumage des leds.
- on lance la suite

4. Test du fonctionnement

4.1. Les entrées seront définies comme suit :

- **resetrn = 1**

4.2. Les sorties seront vérifiées

- la led 0 vont clignoter 10 fois sur une couleur avant de passer à la couleur suivante
- le led 1 vont clignoter 2 fois sur une couleur avant de passer à la couleur suivante sur la fin de cycle de la led 0 .
(nb_clignotement_led1 = nb_clignotement_led0 * clkb/clka = 2)

4.3. Les signaux et états internes à vérifier

- **current_state** = rouge / bleu/ vert tous les $10 \times 2 \times \text{cible} \times \text{periode}(\text{clka})$ avec une mise à jour quand **end_cycle_onoff** = 1
- **color_code** = 01/11/10 tous les $10 \times 2 \times \text{cible} \times \text{periode}(\text{clka})$ avec une mise à jour quand **end_cycle_onoff_a** = 1
- pour les driver a et b : **led_driver: current_onoff_state** = led_on/led_off tous les $\text{cible} \times \text{periode}(\text{clka et b})$
- pour les driver a et b : **led_driver: end_counter** = 1 pendant $\text{periode}(\text{clk})$ tous les $\text{cible} \times \text{periode}(\text{clka et b})$
- pour les driver a et b : **end_cycle_onoff** = 1 pendant $\text{periode}(\text{clka et b})$ tous les $2 \times \text{cible} \times \text{periode}(\text{clka et b})$ quand **current_onoff_state** = led_on
- pour les driver a et b : **end_cycle_offon** = 1 pendant $\text{periode}(\text{clk})$ tous les $2 \times \text{cible} \times \text{periode}(\text{clka et b})$ quand **current_onoff_state** = led_off
- **end_cycle_long** = 1 pendant $10 \times \text{periode}(\text{clka})$ dès que **end_cycle_offon_a** passe à 1
- pour les driver a et b **led_driver: led_color** = rouge / bleu/ vert tous les $10 \times 2 \times \text{cible} \times \text{periode}(\text{clka et b})$ avec une mise à jour quand **end_cycle_offon** = 1
- **nb_cycle** compte de 0 à 9 il s'incrémente tous les $2 \times \text{cible} \times \text{periode}(\text{clka})$ quand **end_cycle_onoff** = 1
- **end_count10** = 1 tant que **end_cycle** = 9 pendant $\text{periode}(\text{clka})$
- **pll_locked** = 1 $\text{periode}(\text{clka}) = 4\text{ns}$, $\text{periode}(\text{clkb}) = 20\text{ns}$

4.4. Sur le test bench

- on attend le premier allumage de la led0 en rouge puis on teste les 3 sorties de led0 : led0_r=0/1, led0_b=0/1 , led0_g= 0/1 après chaque période de $(2 \times \text{cible}) \times \text{periode}(\text{clka})$
- on attend le premier allumage de la led1 en rouge puis on teste les 3 sorties de led1 : led1_r=0/1, led1_b=0/1 , led1_g= 0/1 après chaque période de $(2 \times \text{cible}) \times \text{periode}(\text{clkb})$
- Le test bench testera 7 changements de couleur avant de continuer.

5. Test du reset en fonctionnement

5.1. Les entrées seront définies comme suit :

- **resetrn = 0**

5.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g= 0
- led1_r = 0 , led1_b = 0 , led1_g= 0

5.3. Les signaux et états internes à vérifier

Tous les signaux sont resettée de façon asynchrone dès que **resetrn** = 0

- **current_state** = rouge et **color_code** = "01" car au reset l'état de départ de color FSM est rouge
- **end_cycle_onoff_a** = 0 , **led_driver: end_counter** car led_driver resetté
- **end_cycle_offon_a** = 1 car il suit **led_driver_a: fisrt_on** = 1 quand **current_onoff_state_a** = led_off
- **end_cycle_onoff_b** = 0 , **led_driver: end_counter** car led_driver resetté
- **end_cycle_offon_b** = 1 car il suit **led_driver_b: fisrt_on** = 1 quand **current_onoff_state_b** = led_off
- **end_cycle_long** = 0
- **led_driver_a: led_color** = off car led_driver_a resetté
- **led_driver_a: current_onoff_state** = led_off car led_driver_a resetté
- **led_driver_b: led_color** = off car led_driver_b resetté
- **led_driver_b: current_onoff_state** = led_off car led_driver_b resetté
- **nb_cycle** = 0 , **end_count10** = 0 car la bascule Dcount est resettée
- **pll_locked** = 0 , **clka** = 0 , **clkb** = 0 : la pll est resettée

5.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0 , led0_g= 0 après 1ns
- on teste les 3 sorties de led1 : led1_r=0, led1_b=0 , led1_g= 0 après 1ns
- on attend $2 \times \text{cible} \times \text{periode}(\text{clkb}) + 1\text{ns}$ pour lancer la suite

6. Test du redémarrage après reset

La procédure et les check au redémarrage sont identiques aux partie 1 et 2 ci dessus

Le test sera terminé après ces checks

- Test bench

Le fichier de test bench est tb_top_2clk_pll.vhd

Le principe est identique à tb_top_2clk_long.vhd. On a juste changé les temps d'attente d'allumage des led après reset car celui-ci dépend maintenant du temps de démarrage de la pll. On a aussi remis tous les checks de tous les allumages de led afin de bien compter les cycles

- Résultat des tests automatiques du test_bench

Les test automatiques ont été modifiés car le temps de démarrage de la pll n'est pas connu et le compteur unitaire a déjà démarré quand les clignotements commencent, il est plus difficile de tester automatiquement les 10 et les 2 premiers cycles des leds?

Voici une capture d'une partie du test bench

```
# run 44000ns
Note: PASS : TEST reset : led0 est eteinte OK
Time: 481 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: PASS : TEST reset : led1 est eteinte OK
Time: 2401 ns Iteration: 0 Process: /tb_top_2clk_pll/line__179 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : led1 est eteinte OK
Time: 2423 ns Iteration: 0 Process: /tb_top_2clk_pll/line__179 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : led0 est eteinte OK
Time: 2423 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 3105 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED1 rouge
Time: 3109 ns Iteration: 0 Process: /tb_top_2clk_pll/line__179 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 3585 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 4065 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 4545 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 5025 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 5505 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED1 rouge
Time: 5509 ns Iteration: 0 Process: /tb_top_2clk_pll/line__179 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 5985 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 6465 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 rouge
Time: 6945 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 7425 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 7905 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED1 bleue
Time: 7909 ns Iteration: 0 Process: /tb_top_2clk_pll/line__179 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 8385 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 8865 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 9345 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 9825 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 10305 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED1 bleue
Time: 10309 ns Iteration: 0 Process: /tb_top_2clk_pll/line__179 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 10785 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 11265 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 bleue
Time: 11745 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
Note: LED0 verte
Time: 12225 ns Iteration: 0 Process: /tb_top_2clk_pll/line__109 File: D:/Vivado/TP5/tb_top_2clk_pll.vhd
---
```

la LED1 clignote
2 fois en rouge
au cycle de
démarrage

la LED0
clignote
9 fois
au lieu de 10
au premier
démarrage

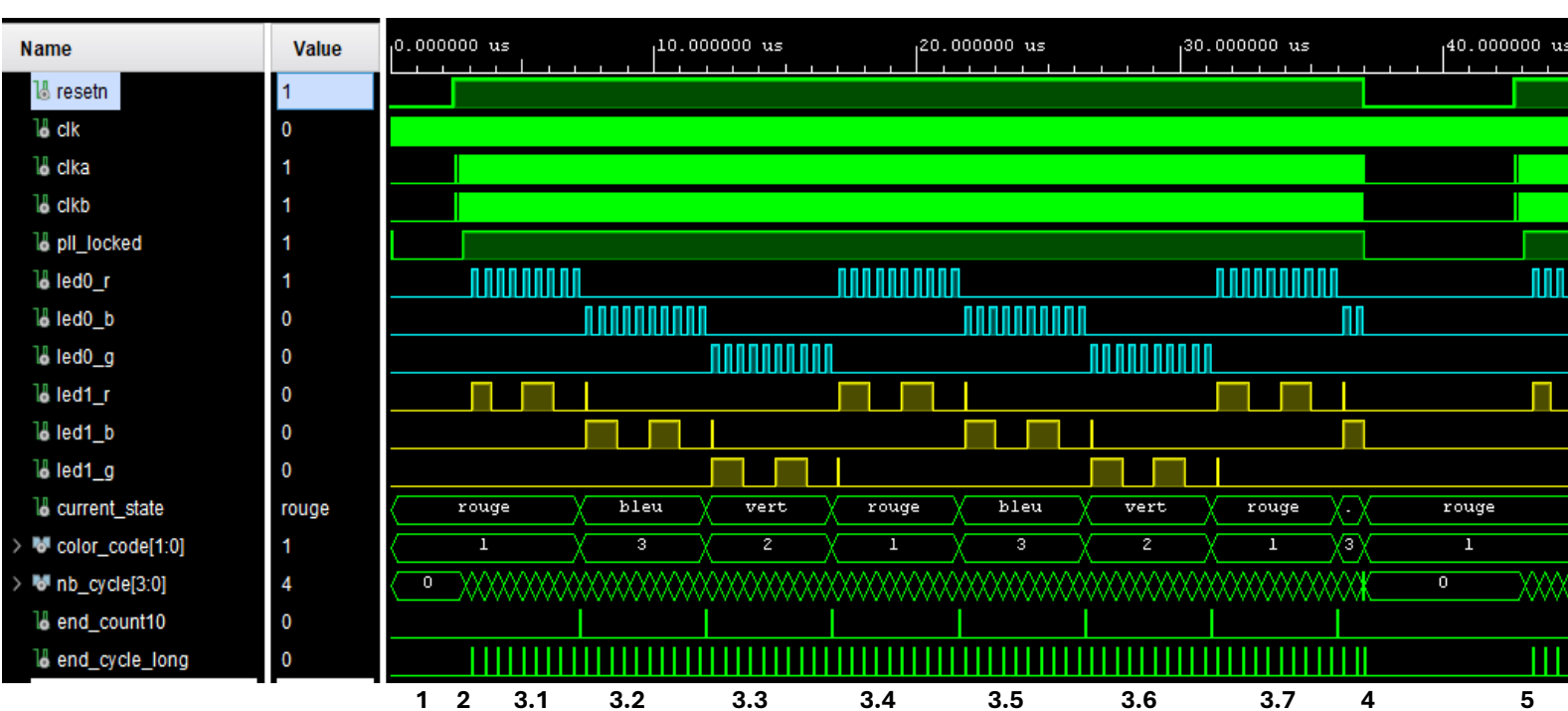
la LED1 clignote
2 fois en bleu
au cycle de
démarrage

la LED0
clignote
10 fois
au cycle
suivant

La LED0 passe en vert

Les résultats du test bench montrent que le cycle de démarrage de le led0 est incomplet, il manque un clignotement.

- Validation fonctionnelle



1- Test du reset: les leds led0 et led1 sont éteintes (led0_r = 0 , led0_b= 0 , led0_g= 0, led1_r = 0 , led1_b= 0 , led1_g= 0)

La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')

La FSM (current_onoff_state)est dans l'état led_off

end_cycle_onoff= 0 resetté

end_cycle_long = 0 et end_count10 = 0

La PLL est resettée clka = 0 et clkb= 0

2- Démarrage après reset:

La PLL démarre: clka et clkb démarrent à leurs fréquences respectives, elles sont établies quand pll_locked = 1

La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')

end_count10 reste à 0 jusqu'au prochain comptage de 10 cycle où il passera à 1 sur une période(clka)

Après le démarrage de la PLL :

La led0 commence à clignoter en rouge (led0_r = 1 , led0_b= 0 , led0_g= 0)

La led1 commence à clignoter en rouge (led1_r = 1 , led1_b= 0 , led1_g= 0)

voir le détail du démarrage plus bas

3- Cycles de 10 clignotements sur les leds

La led0 clignote successivement en rouge, bleu puis vert avant de reboucler sur le rouge

La led0 clignote 10 fois sur chaque cycle **sauf le premier cycle où elle clignote 9 fois**

La led1 clignote 2 fois sur chaque cycle. (10 cycles *250/50 = 2). **Il reste un "résidu" de la couleur précédente à chaque changement de couleur**

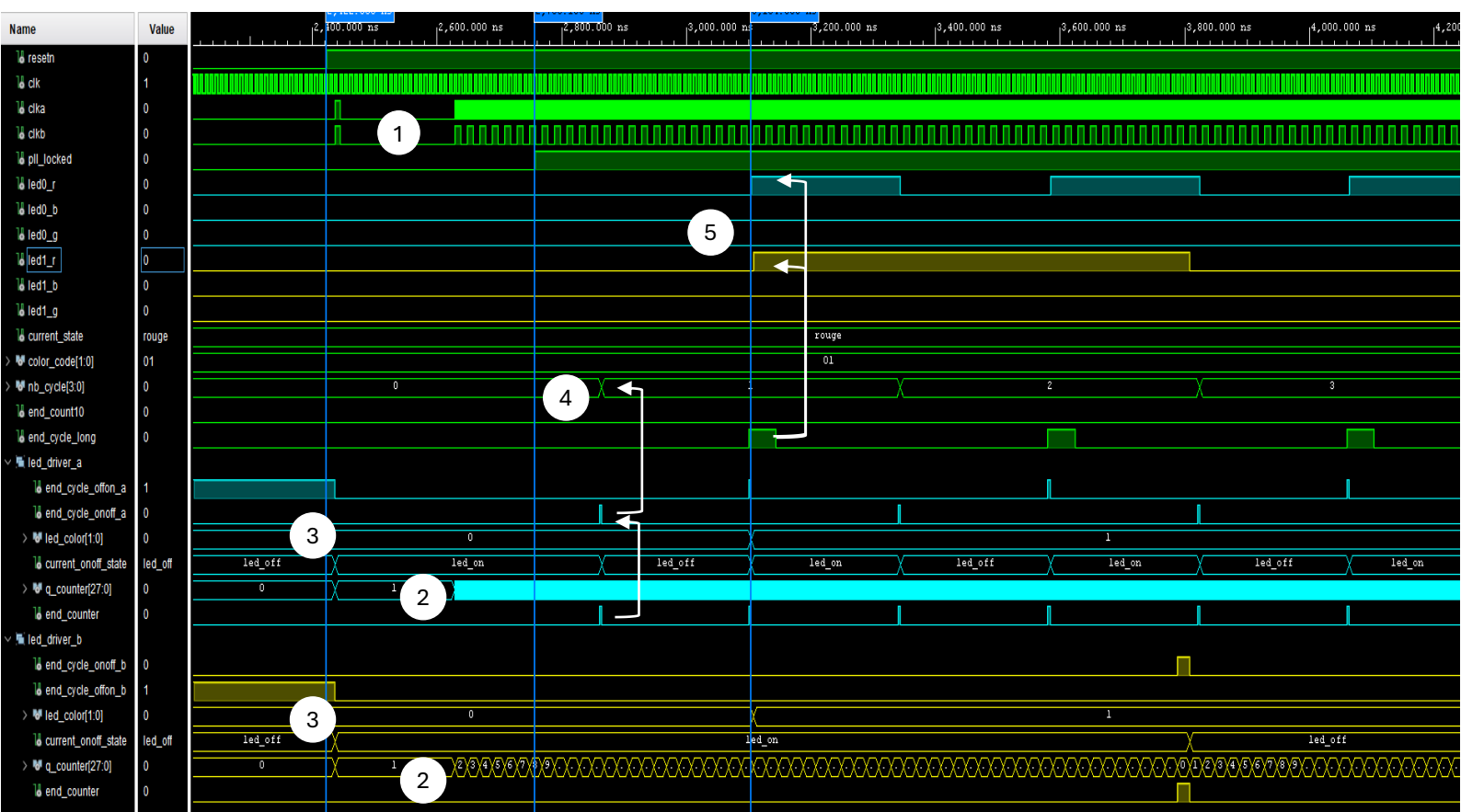
4- Reset en cours de fonctionnement

On se retrouve dans le même état que l'étape 1.

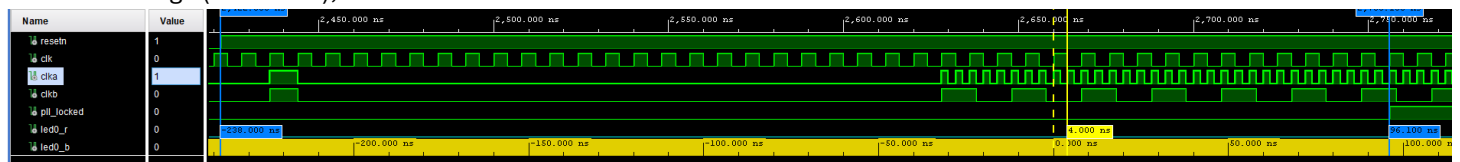
5- Redémarrage après reset

Le système redémarre comme sur l'étape 2.

- Zoom sur le démarrage



1- Au démarrage (resetn = 1), la PLL démarre



clka démarre à 250Mhz et clkb démarre à 50Mhz avec une fréquence d'entrée clk à 125Mhz

la PLL est démarrée 334ns après le reset quand pll_locked = 1

2- Au démarrage (resetn = 1), les compteurs unitaires démarrent et s'incrément sur les fronts d'horloge des horloges clka et clkb qui ne sont pas encore établies, induisant un premier comptage de cycle unitaire faux.

3- Au démarrage (resetn = 1), sur les 2 led_drivers, la machine d'état onoff passe sur led_on sur le signal end_cycle_offon. Par contre le signal led_color reste à '00' car il n'y a pas d'update du led_driver. Les 2 leds restent éteintes au démarrage

4- Au démarrage (pll_locked = 1), Le compteur nb_cycle s'incrémente sur le signal end_cycle_onoff_a il suit end_counter_a quand current_onoff_state_a = led_on

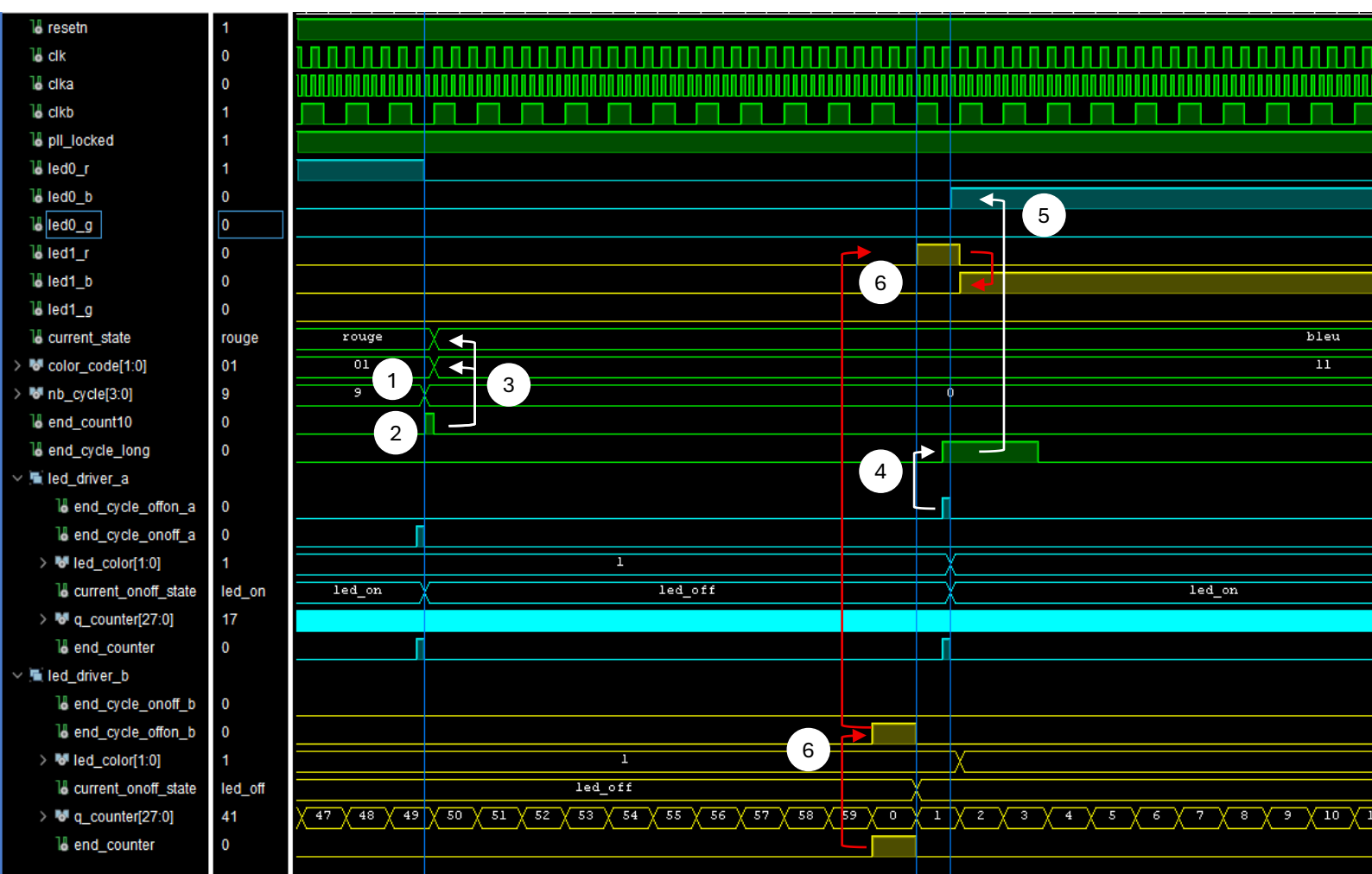
5- Au démarrage (pll_locked = 1), led leds s'allument en rouge sur le premier passage à 1 de end_cycle_long.

led0 et led1 s'allument respectivement sur un front de clka et de clkb

Note 1 : les led ne sont pas allumées sur le premier cycle (nb_cycle = 0) : il y a donc que 9 clignotements au lieu de 10 sur le premier cycle de démarrage en rouge

Note 2 : le premier cycle on de la LED1 est incomplet = 700ns au lieu de 1200ns = cible * periode(clkb) car le compteur unitaire a déjà commencé de compter

- Zoom sur le changement de couleur



1- Le compteur de cycle a compté 10cycles : nb_cycle = 9

2- Sur le cycle suivant on to off de la led 0 (end_cycle_onoff_a = 1 pendant une periode(clka) le signal end_count10 passe à 1 pendant une période(clka)

3 Le code couleur est mis à jour quand end_count10 passe à 1
current_state change d'état, color_code est mis à jour.

4-Au changement d'état off to on de la led0 (end_cycle_offon_a = 1) , le signal end_cycle_long passe à 1 pendant 1+10 période (clka)

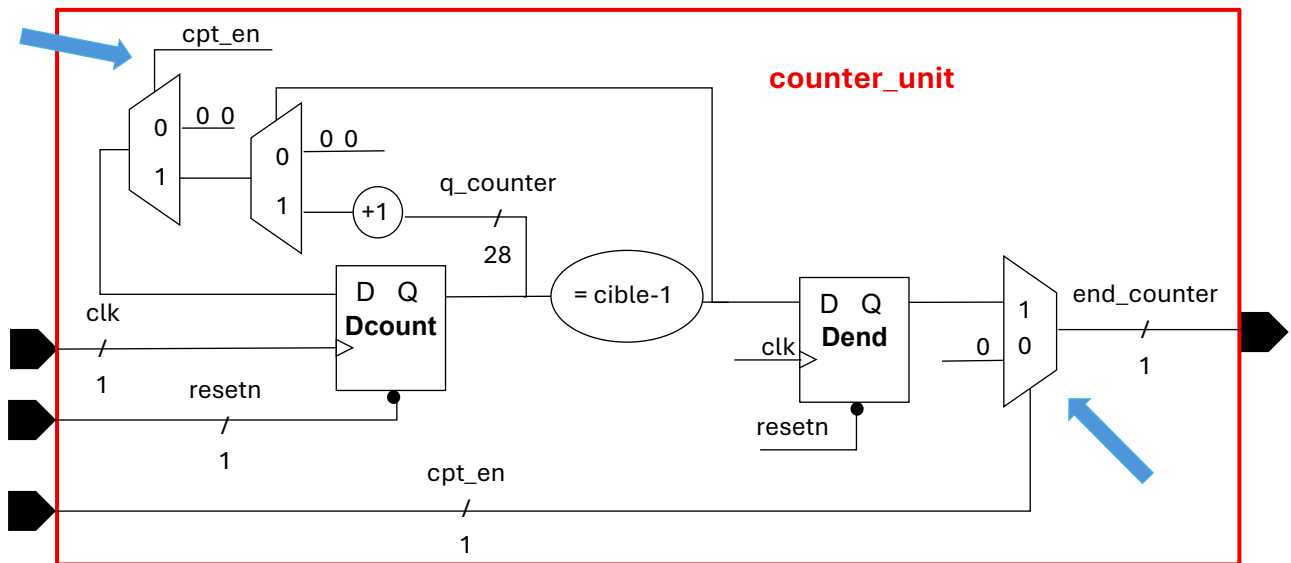
5-La couleur de led0 est mise à jour sur le front de clka suivant

6-La couleur de led1 est mise à jour sur le front de clkb suivant

Pendant le changement de couleur sur la led1 , celle-ci reste sur la couleur précédente pendant (ici pendant 1 periode(clkb))

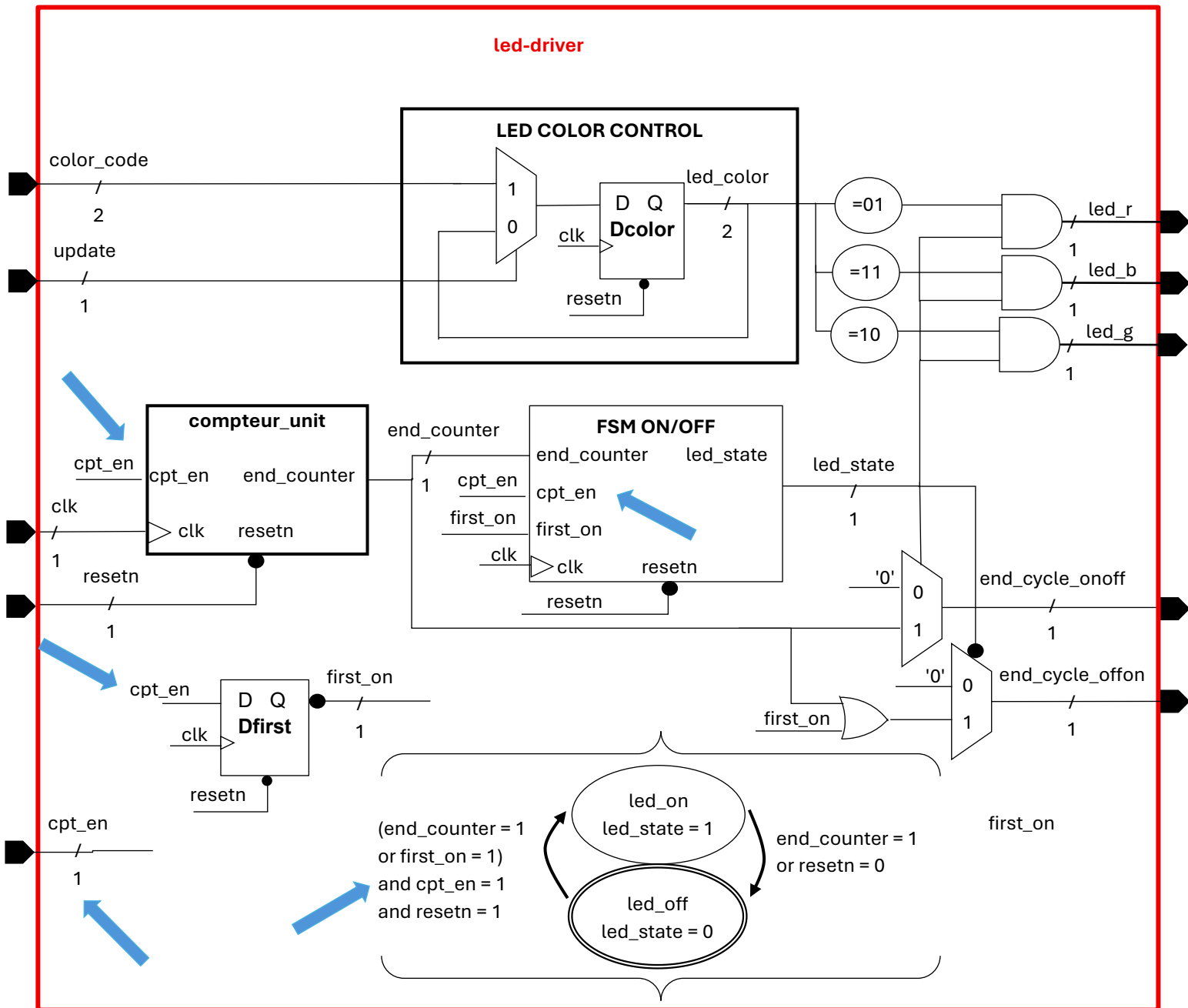
Les compteur unitaire comptent depuis la levée du resetn , leur comptage est donc décalé des autres machines d'état ou des autres logiques du circuit. Dès que ce compteur a terminé de compter , la led change d'état onoff ou offon. Dans notre cas la led1 se rallume avant que le cycle de changement de couleur soit terminé, il y a donc un éclairage parasite de la couleur précédente avant le changement de couleur

9.1 Proposition d'une architecture qui résous les imprécisions au démarrage du système.



En ajoutant un enable **cpt_en** sur le compteur unitaire, on pourra contrôler son démarrage quand les horloges seront établies

Le signal **cpt_en** contrôle la remise à zéro du compteur et la mise à zéro de la sortie **end_counter**

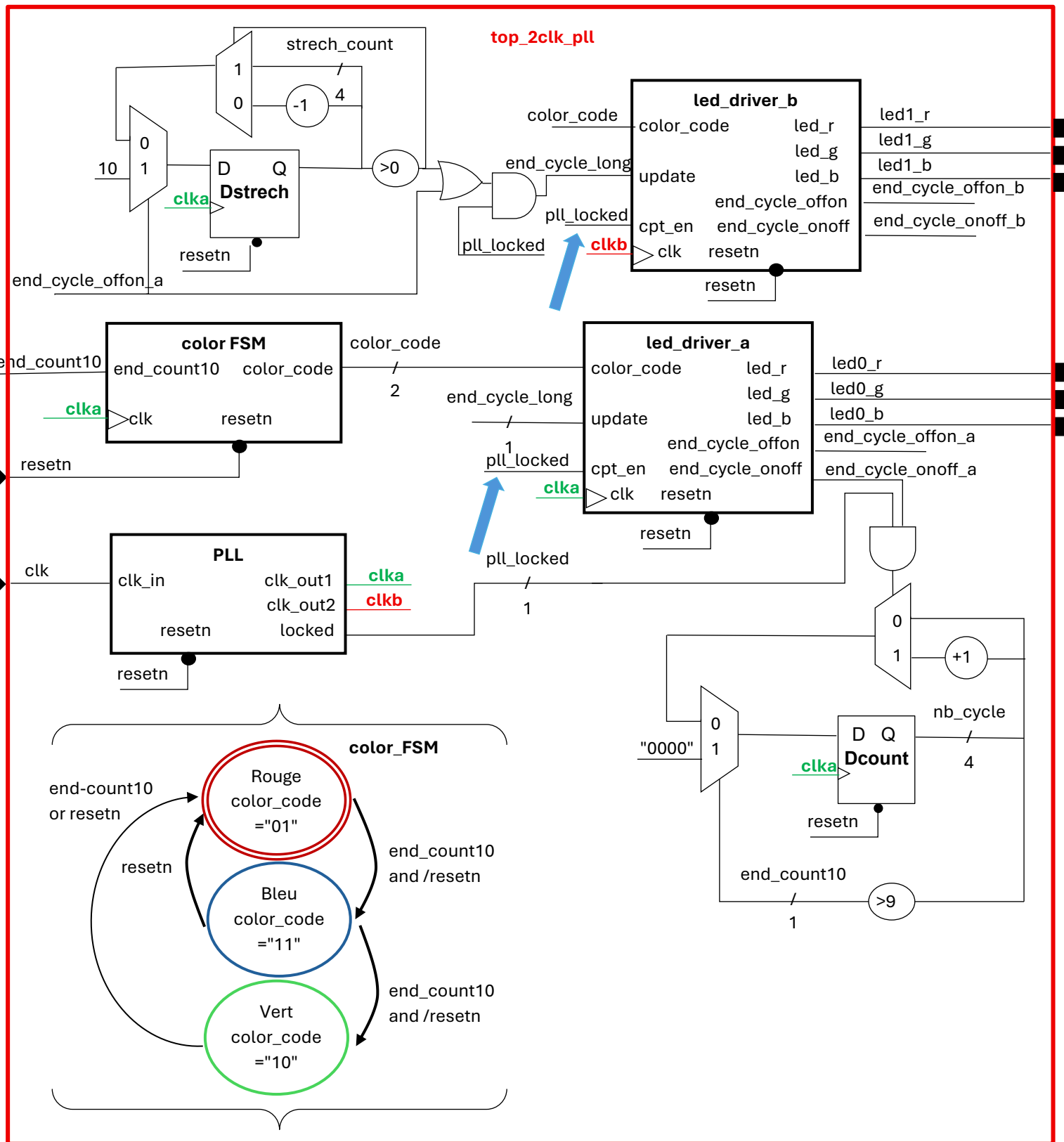


Le `led_driver` a été modifié afin de rajouter le nouveau signal `cpt_en`

1- `cpt_en` contrôle l'enable du `compteur_unit`

2- `cpt_en` contrôle le signal `first_on` qui permet de démarrer les LED sur un cycle on

3- `cpt_en` a été rajouté dans la machine d'état dans les conditions pour passer dans un état on.



Sur le top de l'architecture, le signal `cpt_en` des 2 `led_driver` a été connecté au signal `pll_locked`

- code VHDL de l'architecture

Note: les nouveaux fichiers ont été créés dans un nouveau projet vivado, ils sont fournis dans le sous dossier TP5_cpten
Le fichier counter_unit.vhd a été modifié

```

library IEEE;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity counter_unit is
    generic (
        cible : integer := 200000000
    );
    port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          cpt_en : in STD_LOGIC;
          end_counter : out STD_LOGIC
    );
end counter_unit;

architecture Behavioral of counter_unit is
    signal q_counter : unsigned ( 27 downto 0 ) := (others => '0');
    begin
        process(clk, resetn)
        begin
            if (resetn = '0') then
                q_counter <= (others => '0');
                end_counter <= '0';
            elsif(rising_edge(clk)) then
                if cpt_en = '0' then
                    q_counter <= (others => '0');
                    end_counter <= '0';
                elsif q_counter = to_unsigned(cible - 1, q_counter'length) then
                    q_counter <= (others => '0');
                    end_counter <= '1';
                else
                    q_counter <= q_counter+1;
                    end_counter <= '0';
                end if;
            end if;
        end process;
    end Behavioral;

```

Rajout du port cpt_en

Rajout de la condition cpt_en sur le contrôle des signaux end_counter et q_counter

le fichier led_diver.vhd a été modifié

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity led_driver is
    generic (
        cible : integer := 20
    );
    Port ( clk : in STD_LOGIC;
          resetn : in STD_LOGIC;
          color_code : in STD_LOGIC_VECTOR (1 downto 0);
          update : in STD_LOGIC;
          cpt_en: in  STD_LOGIC;
          end_cycle_offon : out STD_LOGIC;
          end_cycle_onoff : out std_logic;
          led_r : out STD_LOGIC;
          led_g : out STD_LOGIC;
          led_b : out STD_LOGIC);
end led_driver;

architecture Behavioral of led_driver is
    type onoff_state is (led_on,led_off);
    signal current_onoff_state : onoff_state;
    signal next_onoff_state : onoff_state;
    signal led_state: std_logic := '0';
    signal first_on : std_logic := '0';
    signal led_color : std_logic_vector (1 downto 0):= "00";
    signal end_counter: std_logic:='0';
    begin
        cpt: entity work.counter_unit(behavioral)
            generic map (
                cible => cible
            )
            port map (
                clk => clk,
                resetn => resetn,
                end_counter => end_counter,
                cpt_en => cpt_en
            );
        process(clk,resetn)
        begin
            if resetn = '0' then
                current_onoff_state <= led_off;
                first_on <= '1';
            elsif rising_edge(clk) then
                current_onoff_state <= next_onoff_state;
                first_on <= not cpt_en;
            end if ;
        end process;
    end
```

Rajout du port cpt_en

Rajout de la condition
cpt_en sur la génération
du signal first_on

```

-- FSM ON OFF
process(current_onoff_state, end_counter, first_on, cpt_en)
begin
    next_onoff_state <= current_onoff_state;
    case current_onoff_state is
        when led_off =>
            if (end_counter = '1' or first_on = '1' ) and cpt_en = '1' then
                next_onoff_state <= led_on;
            end if;
        when led_on =>
            if end_counter = '1' then
                next_onoff_state <= led_off;
            end if;
        end case;
    end process;
-- LED COLOR CONTROL
process( clk, resetn)
begin
    if resetn = '0' then
        led_color <= "00";
    elsif rising_edge(clk) then
        if update = '1' then
            led_color <= color_code;
        end if ;
    end if;
end process;
led_state <= '1' when current_onoff_state = led_on else '0';
end_cycle_offon <= end_counter or first_on when led_state = '0' else '0';
end_cycle_onoff <= end_counter when led_state = '1' else '0';
led_r <= '1' when led_color = "01" and led_state = '1' else '0';
led_b <= '1' when led_color = "11" and led_state = '1' else '0';
led_g <= '1' when led_color = "10" and led_state = '1' else '0';
end Behavioral;

```

Rajout de la condition cpt_en
 sur le contrôle de la machine
 d'état onoff

le fichier top_2clk_pll.vhd a été modifié

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;
use ieee.numeric_std.all;
entity top_2clk_pll is
    generic (
        cible : integer := 100000000
    );
    port (
        clk          : in std_logic;
        resetn       : in std_logic;
        led0_r       : out std_logic;
        led0_b       : out std_logic;
        led0_g       : out std_logic;
        led1_r       : out std_logic;
        led1_b       : out std_logic;
        led1_g       : out std_logic
    );
end top_2clk_pll;

architecture behavioral of top_2clk_pll is
    type state is (rouge, bleu, vert);
    signal current_state : state;
    signal next_state : state;
    signal color_code : STD_LOGIC_VECTOR (1 downto 0) := "00";
    signal end_cycle_onoff_a : std_logic := '0';
    signal end_cycle_offon_a : std_logic := '0';
    signal end_cycle_onoff_b : std_logic := '0';
    signal end_cycle_offon_b : std_logic := '0';
    signal nb_cycle : unsigned (3 downto 0) := (others => '0');
    signal end_count10 : std_logic := '0';
    signal led_r : std_logic := '0';
    signal led_b : std_logic := '0';
    signal led_g : std_logic := '0';
    signal stretch_count : integer range 0 to 10 := 0;
    signal end_cycle_long : std_logic;
    signal clka : std_logic := '0';
    signal clkb : std_logic := '0';
    signal pll_locked : std_logic := '0';

    component clk_pll
    port (
        clk_out1 : out std_logic;
        clk_out2 : out std_logic;
        resetn   : in  std_logic;
        locked   : out std_logic;
        clk_in1  : in  std_logic
    );
    end component;

begin
    pll: clk_pll
        port map(
            clk_out1 => clkb,
            clk_out2 => clka,
            resetn   => resetn,
            locked   => pll_locked,
            clk_in1  => clk
        );
end;
```

```

led_driver_a: entity work.led_driver(behavioral)
  generic map (
    cible => cible
  )
  port map (
    clk => clka,
    resetn => resetn,
    color_code => color_code,
    update => end_cycle_long,
    cpt_en => pll_locked,
    end_cycle_offon => end_cycle_offon_a,
    end_cycle_onoff => end_cycle_onoff_a,
    led_r => led0_r,
    led_g => led0_g,
    led_b => led0_b
  );
led_driver_b: entity work.led_driver(behavioral)
  generic map (
    cible => cible
  )
  port map (
    clk => clk_b,
    resetn => resetn,
    color_code => color_code,
    update => end_cycle_long,
    cpt_en => pll_locked,
    end_cycle_offon => end_cycle_offon_b,
    end_cycle_onoff => end_cycle_onoff_b,
    led_r => led1_r,
    led_g => led1_g,
    led_b => led1_b
  );
process(clka,resetn)
begin
  if resetn = '0' then
    nb_cycle <= (others => '0');
  elsif rising_edge(clka) then
    end_count10 <= '0';
    if (end_cycle_onoff_a = '1' and pll_locked = '1') then
      if nb_cycle = 9 then
        end_count10 <= '1';
        nb_cycle <= "0000";
      else
        nb_cycle <= nb_cycle + 1;
      end if;
    end if;
  end if;
end process;

```

connexion du signal cpt_en
 des led_drivers au signal
 pll_locked

```

-- color fsm
process ( resetn, clka)
begin
    if resetn = '0' then
        current_state <= rouge;
    elsif rising_edge(clka) then
        current_state <= next_state;
    end if;
end process;
process (current_state, end_count10)
begin
    next_state <= current_state;
    case current_state is
        when rouge =>
            if end_count10 = '1' then
                next_state <= bleu;
            end if;
        when bleu =>
            if end_count10 = '1' then
                next_state <= vert;
            end if;
        when vert =>
            if end_count10 = '1' then
                next_state <= rouge;
            end if;
    end case;
end process;
color_code <=  "01" when current_state = rouge else
                "11" when current_state = bleu else
                "10" when current_state = vert else
                "00";
-- stretch update
process (resetn, clka)
begin
    if resetn = '0' then
        strech_count <= 0;
    elsif rising_edge(clka) then
        if end_cycle_offon_a = '1' then
            strech_count <= 10;
        elsif strech_count >0 then
            strech_count <= strech_count - 1;
        end if;
    end if;
end process ;
end_cycle_long <= '1' when (end_cycle_offon_a = '1' or strech_count >0) and pll_locked = '1' else '0';
end behavioral;

```

- Simulation et test bench

- a. Liste des signaux à vérifier

Liste des signaux d'entrée à tester : resetn

Liste des signaux de sortie à vérifier : led0_r, led0_b, led0_g, led1_r, led1_b, led1_g,

Liste des signaux internes à vérifier : color_code,

pll_locked, clka, clk_b

end_cycle_onoff_a, end_cycle_offon_a,

end_cycle_onoff_b, end_cycle_offon_b,

end_cycle_long, strech_count

nb_cycle, end_count10,

led_driver_a: led_color_a; led_driver_a: end_counter_a,

led_driver_b: led_color_b; led_driver_b: end_counter_b

Listes des états à vérifier: current_state, led_driver_a: current_onoff_state, led_driver_b: current_onoff_state

L'horloge clk est définie à 125Mhz

Note : Les simulations ont été faite avec une cible de **60** soit un temps de comptage du compteur unitaire de 240ns

b. Déroulé de la validation et du test bench

La validation à la même déroulé que l'architecture précédente.

c. Déroulé de la validation et du test bench

1. Test du reset au démarrage

1.1. Les entrées seront définies comme suit :

- **resetn = 0**

1.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g = 0
- led1_r = 0 , led1_b = 0 , led1_g = 0

1.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car au reset l'état de départ de color FSM est rouge
- end_cycle_onoff_a = 0 , led_driver: end_counter car led_driver resetté
- end_cycle_offon_a = 1 car il suit led_driver_a: first_on = 1 quand current_onoff_state_a = led_off
- end_cycle_onoff_b = 0 , led_driver: end_counter car led_driver resetté
- end_cycle_offon_b = 1 car il suit led_driver_b: first_on = 1 quand current_onoff_state_b = led_off
- **end_cycle_long = 0 resetté**
- led_driver_a: led_color = off car led_driver_a resetté
- led_driver_a: current_onoff_state = led_off car led_driver_a resetté
- led_driver_b: led_color = off car led_driver_b resetté
- led_driver_b: current_onoff_state = led_off car led_driver_b resetté
- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée
- **pll_locked = 0 , clka = 0 , clkb = 0** : la pll est resettée

1.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r = 0, led0_b = 0, led0_g = 0 après $2 \times \text{cible} \times \text{periode}(\text{clka}) + 1\text{ns}$
- on teste les 3 sorties de led1 : led1_r = 0, led1_b = 0, led1_g = 0 après $2 \times \text{cible} \times \text{periode}(\text{clkb}) + 1\text{ns}$
- on attend $\text{cible} \times \text{periode}(\text{clkb}) + 1\text{ns}$ pour lancer la suite

2. Test du démarrage après reset

2.1. Les entrées seront définies comme suit :

- **resetn = 1**

2.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g = 0 **tant que pll_locked = 0**
- led1_r = 0 , led1_b = 0 , led1_g = 0 **tant que pll_locked = 0**

2.3. Les signaux et états internes à vérifier

- current_state = rouge et color_code = "01" car l'état de départ de color FSM est rouge tant que end_count10 = 0
- led_driver_a: current_onoff_state = led_off **tant que pll_locked = 0**
- led_driver_b: current_onoff_state = led_off **tant que pll_locked = 0**
- **end_cycle_long = 0 tant que pll_locked = 0;**
- **end_count10 = 0 tant que pll_locked = 0;**
- **led_driver_a: end_counter = 0, le counter_unit ,n'a pas démarré**
- **led_driver_b: end_counter = 0, le counter_unit ,n'a pas démarré**
- pour les driver a et b : end_cycle_onoff = 0 quand current_onoff_state = led_off
- pour les driver a et b : **end_cycle_offon = 1** suit first_on **tant que pll_locked = 0**
- pour les driver a et b : **led_driver: led_color = '00' tant que end_cycle_long = 0**
- **nb_cycle = 0 tant que pll_locked = 0;**
- **pll_locked passe à 1 après un certain temps (je n'ai pas trouvé d'informations sur le lock time dans les documentation de la PLL)**
- **clka et clkb démarrent, elle seront établies quand pll_locked = 1**

2.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r = 0, led0_b = 0, led0_g = 0 après $\text{periode}(\text{clka}) + 1\text{ns}$
- on teste les 3 sorties de led1 : led1_r = 0, led1_b = 0, led1_g = 0 après $\text{periode}(\text{clkb}) + 1\text{ns}$
- on attend que la led0 s'allume pour lancer la suite car on n'a pas d'informations sur le temps mis par la PLL pour démarrer

3. Test du fonctionnement après démarrage de la PLL (pll_locked = 1)

3.1. Les entrées seront définies comme suit :

- **resetn = 1**

3.2. Les sorties seront vérifiées

- **led0_r = 1** , led0_b = 0 , led0_g = 0 au front de clka suivant
- **led1_r = 1** , led1_b = 0 , led1_g = 0 au front de clkb suivant

3.3. Les signaux et états internes à vérifier

- **current_state = rouge** et **color_code = "01"** car l'état de départ de color FSM est rouge tant que **end_count10 = 0**
- **led_driver_a: current_onoff_state = led_on** au front de clka suivant
- **led_driver_b: current_onoff_state = led_on** au front de clkb suivant
- **led_driver_a: end_counter = 0** jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_a soit après $\text{cible} * \text{periode}(\text{clka}) = 240\text{ns}$
- **led_driver_b: end_counter = 0** jusqu' à la fin d' cycle de comptage de counter_unit du led_driver_b soit après $\text{cible} * \text{periode}(\text{clkb}) = 1200\text{ns}$
- pour les driver a et b : **end_cycle_onoff = 0** suit led_driver: end_counter quand led_driver: current_onoff_state = led_on
- pour les driver a et b : **end_cycle_offon = 0** au front suivant de leur clk respectifs
- **end_cycle_long = 1** pendant $10 * \text{periode}(\text{clka})$ car **end_cycle_offon_a** passe à 1
- **nb_cycle** compte de 0 à 9 il s'incrémente tous les $2 * \text{cible} * \text{periode}(\text{clka})$ quand **end_cycle_offon_a = 1**
- **end_count10 = 1** quand **nb_cycle = 9**)
- pour les driver a et b : **led_driver: led_color = '01'** quand **end_cycle_long = 1**
- **pll_locked = 1**
- **clka et clkb sont établies , periode (clka) = 4ns, periode(clkb)= 20ns**

3.4. Sur le test bench

- on ne peut pas faire de test automatique au moment du démarrage de la PLL car il n'y a aucun signaux sur le top design qui indique cet état. Les test automatiques se feront à l'étape suivante au guettant le premier allumage des leds.
- on lance la suite

4. Test du fonctionnement

4.1. Les entrées seront définies comme suit :

- **resetsn = 1**

4.2. Les sorties seront vérifiées

- la led 0 vont clignoter 10 fois sur une couleur avant de passer à la couleur suivante
- le led 1 vont clignoter 2 fois sur une couleur avant de passer à la couleur suivante sur la fin de cycle de la led 0 .
($\text{nb_clignotement_led1} = \text{nb_clignotement_led0} * \text{clkb}/\text{clka} = 2$)

4.3. Les signaux et états internes à vérifier

- **current_state = rouge / bleu/ vert** tous les $10 * 2 * \text{cible} * \text{periode}(\text{clka})$ avec une mise à jour quand **end_cycle_onoff = 1**
- **color_code = 01/11/10** tous les $10 * 2 * \text{cible} * \text{periode}(\text{clka})$ avec une mise à jour quand **end_cycle_onoff_a = 1**
- pour les driver a et b : **led_driver: current_onoff_state = led_on/led_off** tous les $\text{cible} * \text{periode}(\text{clka et b})$
- pour les driver a et b : **led_driver: end_counter = 1** pendant $\text{periode}(\text{clk})$ tous les $\text{cible} * \text{periode}(\text{clka et b})$
- pour les driver a et b : **end_cycle_onoff = 1** pendant $\text{periode}(\text{clka et b})$ tous les $2 * \text{cible} * \text{periode}(\text{clka et b})$ quand **current_onoff_state = led_on**
- pour les driver a et b : **end_cycle_offon = 1** pendant $\text{periode}(\text{clk})$ tous les $2 * \text{cible} * \text{periode}(\text{clka et b})$ quand **current_onoff_state = led_off**
- **end_cycle_long = 1** pendant $10 * \text{periode}(\text{clka})$ dès que **end_cycle_offon_a** passe à 1
- pour les driver a et b **led_driver: led_color = rouge / bleu/ vert** tous les $10 * 2 * \text{cible} * \text{periode}(\text{clka et b})$ avec une mise à jour quand **end_cycle_offon = 1**
- **nb_cycle** compte de 0 à 9 il s'incrémente tous les $2 * \text{cible} * \text{periode}(\text{clka})$ quand **end_cycle_onoff = 1**
- **end_count10 = 1** tant que **end_cycle = 9** pendant $\text{periode}(\text{clka})$
- **pll_locked = 1 periode (clka) = 4ns, periode(clkb)= 20ns**

4.4. Sur le test bench

- on attend le premier allumage de la led0 en rouge puis on teste les 3 sorties de led0 : **led0_r = 0/1** , **led0_b = 0/1** , **led0_g = 0/1** après chaque période de $(2 * \text{cible}) * \text{periode}(\text{clka})$
- on attend le premier allumage de la led1 en rouge puis on teste les 3 sorties de led1 : **led1_r = 0/1** , **led1_b = 0/1** , **led1_g = 0/1** après chaque période de $(2 * \text{cible}) * \text{periode}(\text{clkb})$
- Le test bench testera 7 changements de couleur avant de continuer.

5. Test du reset en fonctionnement

5.1. Les entrées seront définies comme suit :

- **resetsn = 0**

5.2. Les sorties seront vérifiées

- led0_r = 0 , led0_b = 0 , led0_g = 0
- led1_r = 0 , led1_b = 0 , led1_g = 0

5.3. Les signaux et états internes à vérifier

Tous les signaux sont resettés de façon asynchrone dès que resetn = 0

- current_state = rouge et color_code = "01" car au reset l'état de départ de color FSM est rouge
- end_cycle_onoff_a = 0 , led_driver: end_counter car led_driver resetté
- end_cycle_offon_a = 1 car il suit led_driver_a: first_on = 1 quand current_onoff_state_a = led_off
- end_cycle_onoff_b = 0 , led_driver: end_counter car led_driver resetté
- end_cycle_offon_b = 1 car il suit led_driver_b: first_on = 1 quand current_onoff_state_b = led_off
- **end_cycle_long = 0**
- led_driver_a: led_color = off car led_driver_a resetté
- led_driver_a: current_onoff_state = led_off car led_driver_a resetté
- led_driver_b: led_color = off car led_driver_b resetté
- led_driver_b: current_onoff_state = led_off car led_driver_b resetté
- nb_cycle = 0 , end_count10 = 0 car la bascule Dcount est resettée
- **pll_locked = 0 , clka = 0 , clkb = 0** : la pll est resettée

5.4. Sur le test bench

- on teste les 3 sorties de led0 : led0_r=0, led0_b=0, led0_g= 0 après 1ns
- on teste les 3 sorties de led1 : led1_r=0, led1_b=0, led1_g= 0 après 1ns
- on attend 2*cible*period(clkb) +1ns pour lancer la suite

6. Test du redémarrage après reset

La procédure et les checks au redémarrage sont identiques aux parties 1 et 2 ci dessus

Le test sera terminé après ces checks

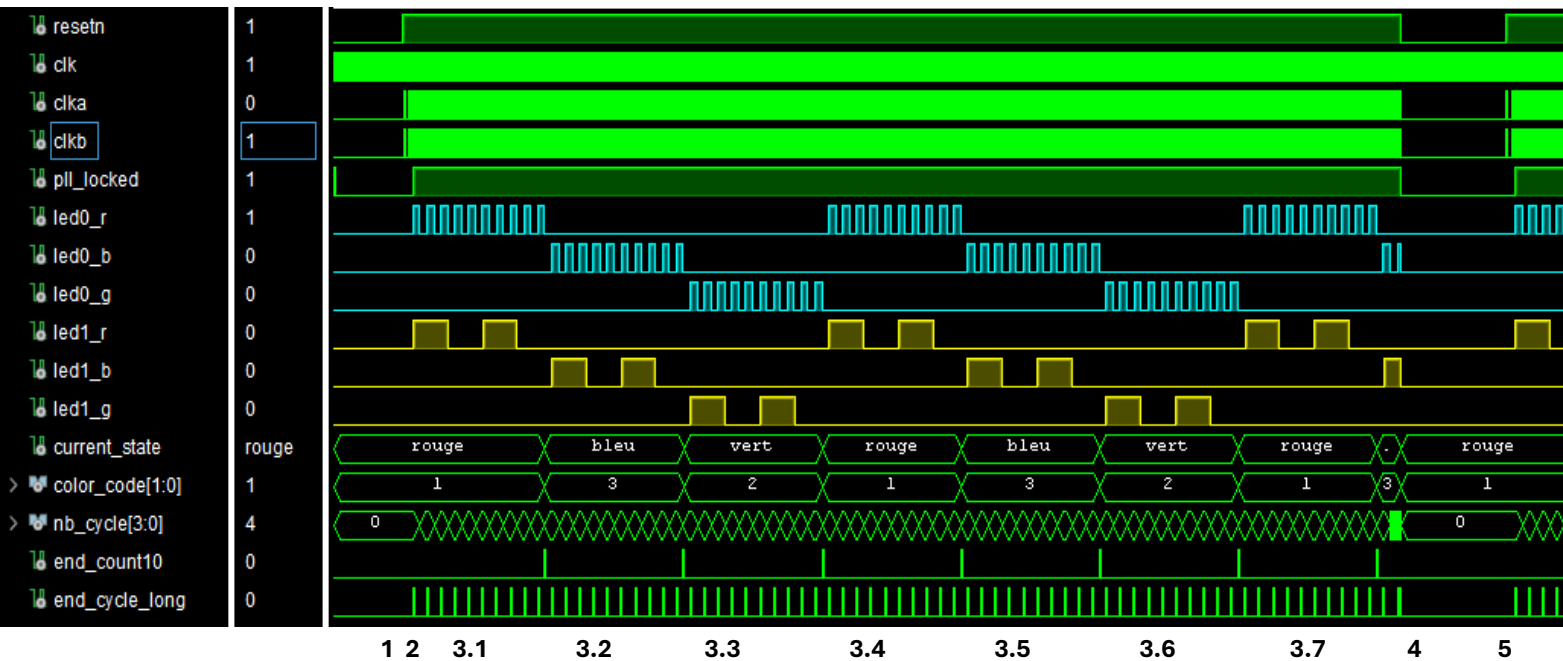
- Test bench

Le fichier de test bench est tb_top_2clk_pll.vhd

- Résultat des tests automatiques du test_bench

```
# run 43000ns
Note: PASS : TEST reset : led0 est eteinte OK
Time: 481 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST reset : led1 est eteinte OK
Time: 2401 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : led1 est eteinte OK
Time: 2423 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : led0 est eteinte OK
Time: 2423 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 7561 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence rouge
Time: 7569 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 12361 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence bleu
Time: 12369 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 17161 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence vert
Time: 17169 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 21961 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence rouge
Time: 21969 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence bleu
Time: 26761 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence bleu
Time: 26769 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence vert
Time: 31561 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence vert
Time: 31569 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED0 allumee 10 fois pour la sequence rouge
Time: 36361 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS LED1 allumee 2 fois pour la sequence rouge
Time: 36369 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST reset : la led0 est eteinte OK
Time: 36983 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST reset : la led1 est eteinte OK
Time: 36983 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : la led1 est eteinte OK
Time: 40584 ns Iteration: 0 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : la led0 est eteinte OK
Time: 40584 ns Iteration: 0 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : la led0 s'allume en rouge OK
Time: 40920 ns Iteration: 7 Process: /tb_top_2clk_pll/line__107 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: PASS : TEST demarrage : la led1 s'allume en rouge OK
Time: 40920 ns Iteration: 7 Process: /tb_top_2clk_pll/line__203 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
Note: Nombre de test FAIL : 0 AUCUN TEST FAILED
Time: 42024 ns Iteration: 0 Process: /tb_top_2clk_pll/line__79 File: D:/Vivado/TP5 PLL/tb_top_2clk_pll.vhd
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_top_2clk_pll_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 43000ns
) launch_simulation: Time (s): cpu = 00:00:05 ; elapsed = 00:00:13 . Memory (MB): peak = 2351.738 ; gain = 6.527
```

- Validation fonctionnelle



- 1- Test du reset: les leds led0 et led1 sont éteintes (led0_r = 0 , led0_b= 0 , led0_g= 0, led1_r= 0 , led1_b= 0 , led1_g= 0)
 La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')
 nb_cycle reste à 0 , end_count10=0
 end_cycle_long = 0
La PLL est resettée clka = 0 et clkb= 0

- 2- Démarrage après reset:

La PLL démarre: clka et clkb démarrent à leurs fréquences respectives, elles sont établies quand pll_locked = 1

La color FSM est dans l'état rouge (current_state = rouge, color_code = '01')

end_count10 reste à 0 jusqu'au prochain comptage de 10 cycle où il passera à 1 sur une période(clka)

Après le démarrage de la PLL :

La led0 commence à clignoter en rouge (led0_r = 1 , led0_b= 0 , led0_g= 0)

La led1 commence à clignoter en rouge (led1_r = 1 , led1_b= 0 , led1_g= 0)

voir le détail du démarrage plus bas

- 3- Cycles de 10 clignotements sur les leds

La led0 clignote successivement en rouge, bleu puis vert avant de reboucler sur le rouge

La led0 clignote 10 fois sur chaque cycle.

La led1 clignote 2 fois sur chaque cycle. (10 cycles * 250/50 = 2)

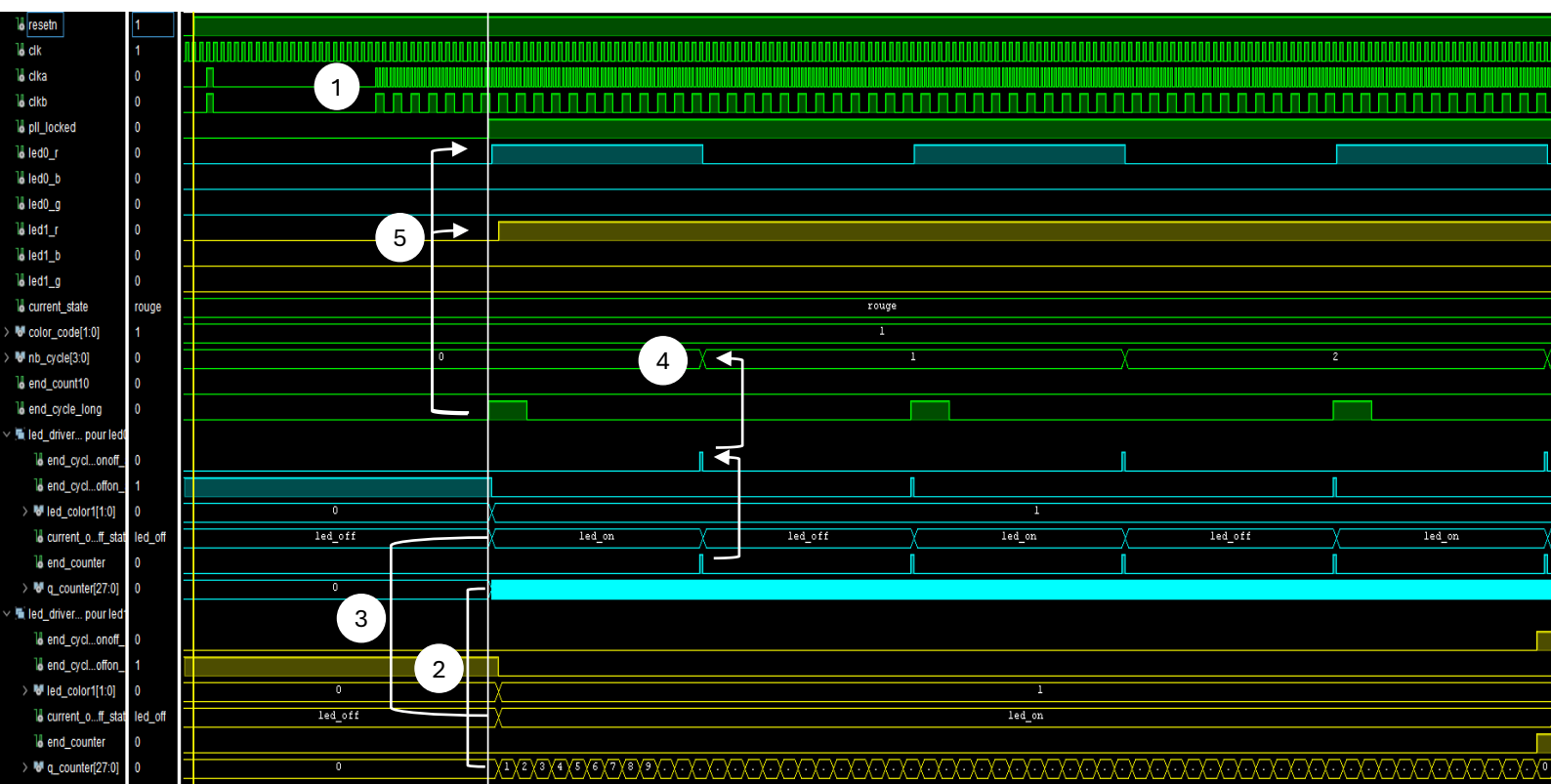
- 4- Reset en cours de fonctionnement

On se retrouve dans le même état que l'étape 1.

- 5- Redémarrage après reset

Le système redémarre sur 10 cycles en rouge comme sur l'étape 2.

- Zoom sur le démarrage



Au démarrage (resetn = 1):

1- La PLL démarre

clka démarre à 250Mhz et clkb démarre à 50Mhz avec une fréquence d'entrée clk à 125Mhz

la PLL est démarrée 334ns après le reset quand pll_locked = 1

Il ne se passe rien d'autre quand resetn est relâché.

Après le démarrage de la PLL (pll_locked = 1):

2- Les compteurs unitaires des led_driver a et b démarrent et s'incrémentent sur les fronts d'horloge des horloges clka et clkb

3 Sur les 2 led_drivers, la machine d'état onoff passe sur led_on sur le signal end_cycle_offon.

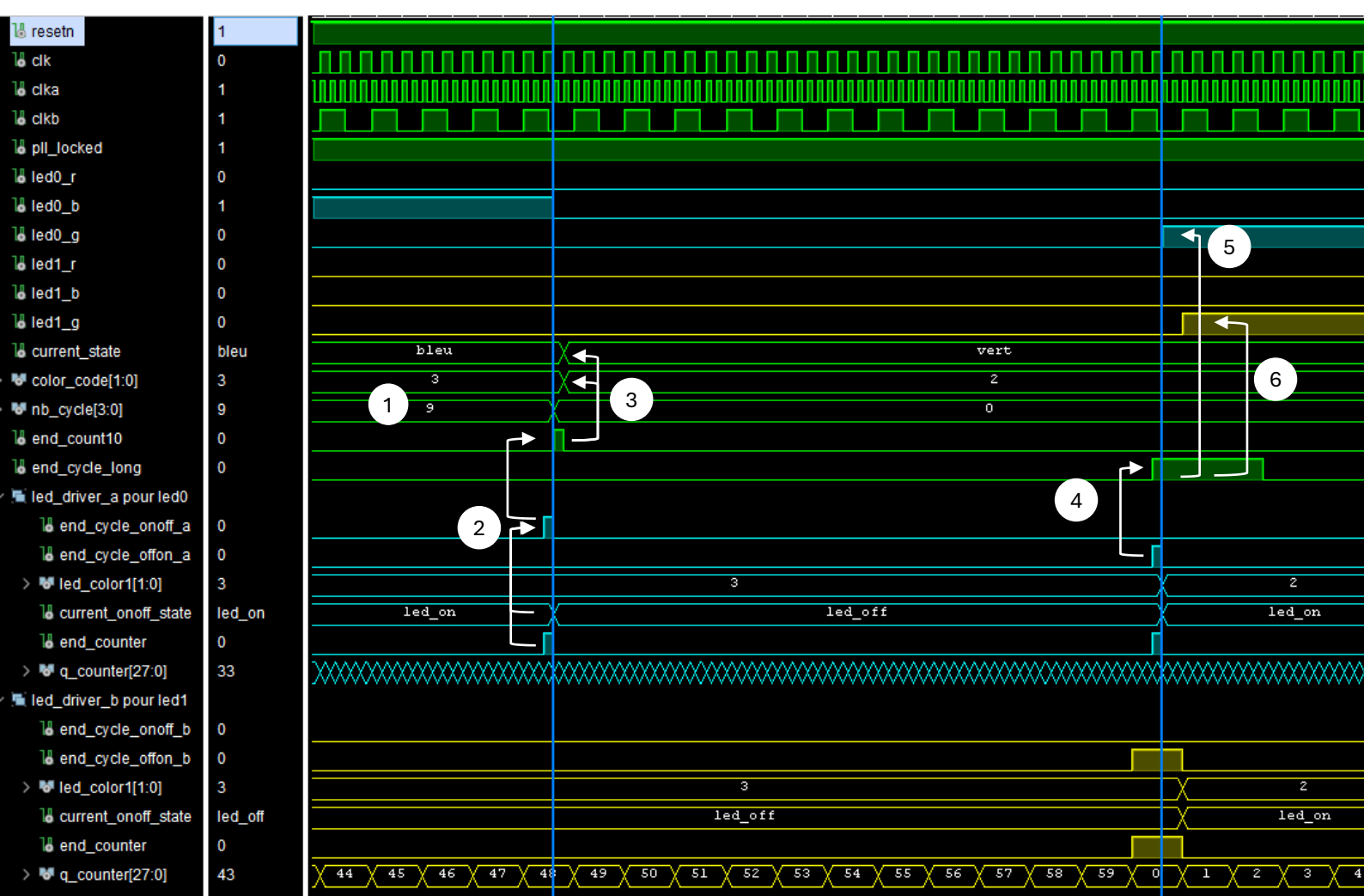
Leur signal led_color est mis à jour à '01' sur les fronts d'horloge des horloges clka et clkb

4- Le compteur nb_cycle s'incrémente sur le signal end_cycle_onoff_a

5- Led leds s'allument en rouge sur le premier passage à 1 de end_cycle_long.

led0 et led 1 s'allument respectivement sur un front de clka et de clkb

- Zoom sur le changement de couleur



1- Le compteur de cycle a compté 10cycles : nb_cycle = 9

2- Sur le cycle suivant on to off de la led 0 (end_cycle_onoff_a = 1 pendant une periode(clka) le signal end_count10 passe à 1 pendant une période(clka)

3 Le code couleur est mis à jour quand end_count10 passe à 1
current_state change d'état, color_code est mis à jour.

4-Au changement d'état off to on de la led0 (end_cycle_offon_a = 1) , le signal end_cycle_long passe à 1 pendant 1+10 période (clka)

5-La couleur de led0 est mise à jour sur le front de clka suivant

6-La couleur de led1 est mise à jour sur le front de clkb suivant

10. Effectuez la synthèse et placer des sondes (ILA) sur les signaux pertinents pour vérifier que le changement de domaine d'horloge s'est correctement passé.

Dans un premier temps, la synthèse et l'implémentation est effectuée sans ILA afin de repérer les timings et les chemins critiques du design.

- Définition du fichier de contraintes

```
# PL System Clock#
set_property -dict {PACKAGE_PIN H16 IOSTANDARD LVCMOS33} [get_ports clk]
#create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];#set

# RGB LEDs
set_property -dict {PACKAGE_PIN L16 IOSTANDARD LVCMOS33} [get_ports led0_b]
set_property -dict {PACKAGE_PIN G17 IOSTANDARD LVCMOS33} [get_ports led0_g]
set_property -dict {PACKAGE_PIN N15 IOSTANDARD LVCMOS33} [get_ports led0_r]
set_property -dict {PACKAGE_PIN G14 IOSTANDARD LVCMOS33} [get_ports led1_b]
set_property -dict {PACKAGE_PIN L14 IOSTANDARD LVCMOS33} [get_ports led1_g]
set_property -dict {PACKAGE_PIN M15 IOSTANDARD LVCMOS33} [get_ports led1_r]

## User Digital I/O Header J1
set_property -dict {PACKAGE_PIN L19 IOSTANDARD LVCMOS33} [get_ports resetn]
#set_property -dict { PACKAGE_PIN M19 IOSTANDARD LVCMOS33 } [get_ports { user_dio[2] }]; #IO_L7P_T1_AD2P_35 Sch=user_dio[2]
#set_property -dict { PACKAGE_PIN N20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[3] }]; #IO_L14P_T2_SRCC_34 Sch=user_dio[3]
#set_property -dict { PACKAGE_PIN P20 IOSTANDARD LVCMOS33 } [get_ports { user_dio[4] }]; #IO_L14N_T2_SRCC_34 Sch=user_dio[4]
```

Le port clk est routé Mais l'horloge est déjà déclarée dans l'ip PLL , je ne pense pas qu'il soit nécessaire de la recréer dans ce fichier de contraintes.

Dans le fichier clk_pll.xdc de l'ip PLL, on trouve ceci :

```
#-----
# Connect to input port when clock capable pin is selected for input
create_clock -period 8.000 [get_ports clk_in1]
set_input_jitter [get_clocks -of_objects [get_ports clk_in1]] 0.08

set_property PHASESHIFT_MODE WAVEFORM [get_cells -hierarchical *adv*]
```

le signal de resetn est routé sur un user_io[1] pour pouvoir être connecté à une alimentation sur la carte

- Résultat de la synthèse

INFO: [Synth 8-802] inferred FSM for state register 'current_state_reg' in module 'top_2clk_pll'

State	New Encoding	Previous Encoding
rouge	001	00
bleu	010	01
vert	100	10

INFO: [Synth 8-3354] encoded FSM with state register 'current_state_reg' using encoding 'one-hot' in module 'top_2clk_pll'

Il a reconnu la machine d'état color FSM et a changé son encodage. Le signal color_code est maintenant codé sur 3 bits.

Start RTL Component Statistics

Detailed RTL Component Info :

+---Adders :

2 Input 28 Bit Adders := 2
2 Input 4 Bit Adders := 2

+---Registers :

28 Bit Registers := 2
4 Bit Registers := 2
2 Bit Registers := 2
1 Bit Registers := 7

+---Muxes :

2 Input 28 Bit Muxes := 2
2 Input 4 Bit Muxes := 2
3 Input 2 Bit Muxes := 1
2 Input 1 Bit Muxes := 6

Finished RTL Component Statistics

Additionneurs :

- 2*1 additionneur pour les 2 counter_unit sur 28 bits
- 1 additionneur sur 4 bits pour le nb_cycle
- 1 additionneur sur 4 bits pour le strech_count

Registres

- 2*1 registre 28 bits pour les 2 counter_unit
- 1 registre 4 bits pour le nb_cycle
- 1 registre 4 bits pour le strech_count
- 2*1 registre 2 bits pour le décodage du code couleur dans les 2 led_driver
- 2*1 registre 1 bits sur le signal end_counter pour les 2 counter_unit
- 2*1 registre 1 bits sur la machine d'état on/off des 2 led_driver
- 2*1 registre 1 bits sur le signal first_on des 2 led_driver
- 1 registre 1 bit pour le signal end_count10

Multiplexeurs

- 2*1 mux 28 bits pour le counter_unit
- 1 mux 2 entrées 4 bits pour le nb_cycle
- 1 mux 2 entrées 4 bits pour le strech_count
- 1 mux 3 entrées 2 bits pour le color_code
- 2*1 mux 2 entrée 1 bit pour le signal end_cycle_onoff des 2 led_driver
- 2*1 mux 2 entrée 1 bit pour le signal end_cycle_offon des 2 led_driver
- 1 mux 2 entrées 1 bit pour le compteur de cycle
- 1 mux 2 entrées 1 bit pour le strech_count

Report BlackBoxes:

	BlackBox name	Instances
1	clk_pll	1

Report Cell Usage:

	Cell	Count
1	clk_pll	1
2	CARRY4	14
3	LUT1	3
4	LUT2	2
5	LUT3	12
6	LUT4	16
7	LUT5	65
8	LUT6	4
9	FDCE	72
10	FDPE	5
11	FDRE	1
12	IBUF	1
13	OBUF	6

1 clk_pll

14 CARRY4 : 7*2 additionneurs de 4 bits pour les 2 counter_unit 28 bits

3 LUT 1 1 inverseur pour le resetn

1 inverseur pour le signal cpt_en utilisé pour la génération de first_on il est ensuite utilisé dans les 2 led_driver

1 inverseur sur l'entrée du premier registre du compteur de cycle nb_cycle

2 LUT 2 2*1 LUT2 pour les 2 counter_unit 28 bits

12 LUT3 3 LUT3 pour la color FSM

1 LUT3 pour le compteur de cycle nb_cycle

1 LUT3 pour contrôler l'enable des registres du comteur de cycle nb_cycle

2*3 LUT3 pour la logique de contrôle des signaux des 2 leds

1 LUT3 qui gère la combinatoire d'entrée du compteur strech_count

16 LUT4: 2*4 LUT4 pour les 2 counter_unit 28 bits

2*2 LUT4 pour la génération du signal end_counter des 2 counter_unit

1 LUT4 pour la génération du signal end_count10

2 LUT4 pour le compteur de cycle nb_cycle

1 LUT4 pour la génération du signal strech_count

65 LUT5: 2*29 LUT5 pour les 2 counter_unit 28 bits

3 LUT5 pour la génération du signal strech_count

2*2 LUT5 pour la génération du signal led_color des 2 led_driver

4 LUT6 2*1 LUT6 pour les 2 counter_unit 28 bits

1 LUT6 pour la génération du signal strech_count

1 LUT6 pour la génération du signal end_count10

72 FDCE 2*28 FDCE pour les 2 counter_unit 28 bits

2 FDCE pour la colorFSM codée sur 3 bits

4 FDCE pour le compteur de cycle nb_cycle 4 bits

4 FDCE pour le signal strech_count4 bits

2*2 FDCE pour le signal led_color 2 bits des led_drivers

2*1 FDCE pour le signal end_counter 1 bit des 2 counter_unit

5 FDPE 1 FDPE pour la colorFSM codée sur 3 bits

2*1 FDPE pour la machine d'état onoff des 2 led_driver

2*1 FDPE pour le signal first_on des 2 led_driver

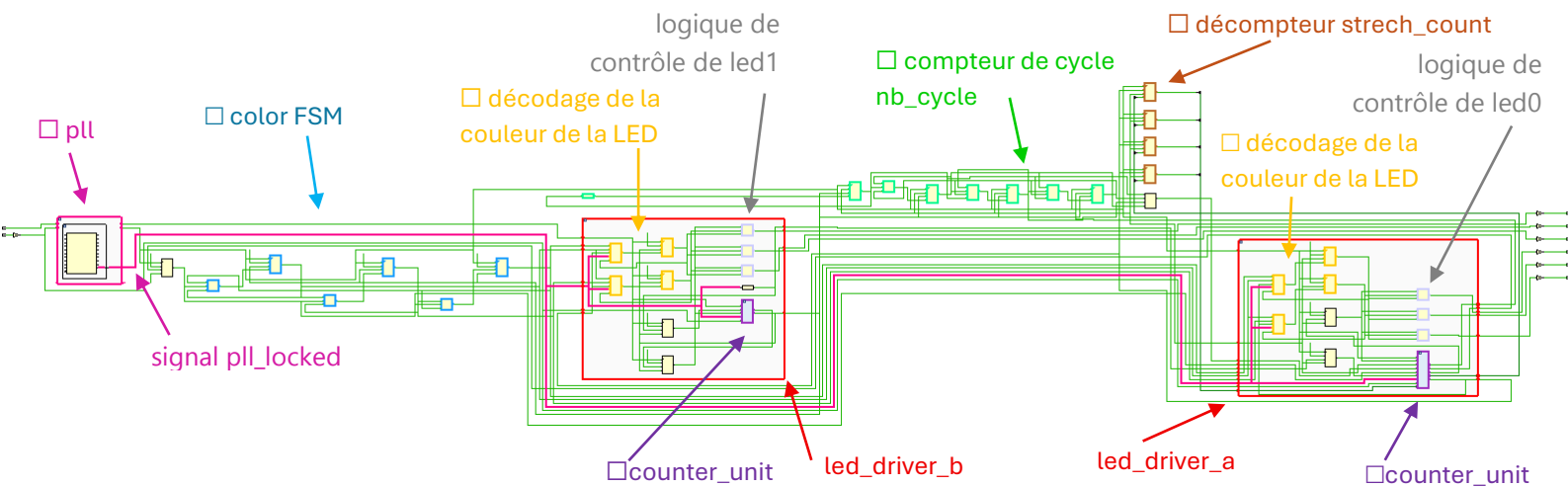
1 FDRE 1 FDRE pour le signal end_count10

1 IBUF 1 buffer d'entrée pour clk

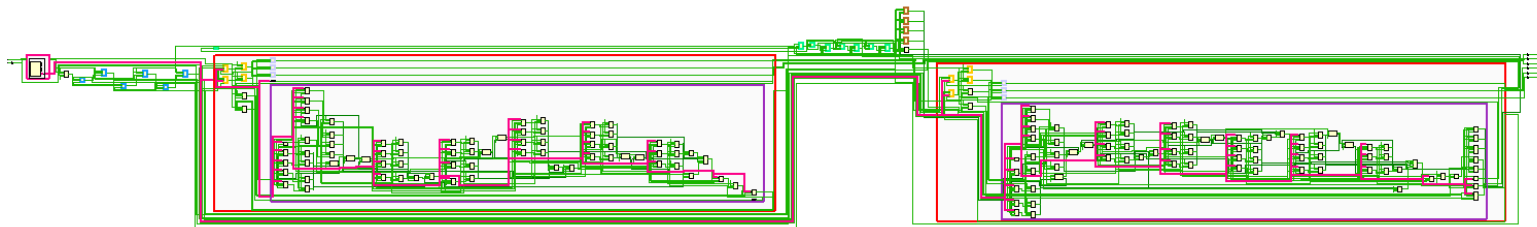
6 OBUF 2*3 buffers de sortie pour les 3 sorties de contrôle des 2 leds: led0_r, led0_g, led0_b, led1_r, led1_g, led1_b,

- Identification des blocs sur le schéma implémenté

Afin de faciliter la visibilité du schéma dans le rapport, je n'ai pas affiché l'intérieur des counter_unit



Vue du circuit avec la vue éclatée des counter_unit, ce n'est pas trop lisible dans le rapport



- Implémentation / rapport de timings

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,521 ns	Worst Hold Slack (WHS): 0,190 ns	Worst Pulse Width Slack (WPWS): 1,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 84	Total Number of Endpoints: 84	Total Number of Endpoints: 86

All user specified timing constraints are met.

Le rapport de timing a été réalisé sans ILA afin d'identifier correctement le chemin critique.

Le design respecte les contraintes de timings pour une horloge à 125Mhz.

Il n'y a pas de violation de setup et de hold TNS = 0 et THS = 0.

Dans le rapport de timing, on peut voir plusieurs rapports concernant des timings intra_clock et des timings inter_clock

Tcl ConsoleMessagesLogReportsDesign RunsDRCPowerTiming x?

Intra-Clock Paths

Intra-Clock Paths

clk

Pulse Width 2,000 ns (6)

clk_out1_clk_pll

Setup 15,058 ns (10)

Hold 0,190 ns (10)

Pulse Width 9,500 ns (31)

clk_out2_clk_pll

Setup 0,532 ns (10)

Hold 0,233 ns (10)

Pulse Width 1,500 ns (31)

clkfbout_clk_pll

Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)	WPWS (ns)	TPWS (ns)	Failing Endpoints (TPWS)	Total Endpoints (TPWS)
clk											2,000	0,000	0	1
clk_out1_clk_pll	rise - rise	15,058	0,000	0	32	rise - rise	0,190	0,000	0	32	9,500	0,000	0	35
clk_out2_clk_pll	rise - rise	0,532	0,000	0	52	rise - rise	0,233	0,000	0	52	1,500	0,000	0	47
clkfbout_clk_pll											5,845	0,000	0	3

Le rapport de timings est bon pour tous les chemins intra clocks

Le chemin critique sur la clock b = 50Mhz reste dans le compteur du led_driver_b

Summary	
Name	Path 1
Slack	15.058ns
Source	led_driver_b/cpt/q_counter_reg[0]/C (rising edge-triggered cell FDCE clocked by clk_out1_clk_pll {rise@0.000ns fall@10.000ns period=20.000ns})
Destination	led_driver_b/cpt/q_counter_reg[4]/D (rising edge-triggered cell FDCE clocked by clk_out1_clk_pll {rise@0.000ns fall@10.000ns period=20.000ns})
Path Group	clk_out1_clk_pll
Path Type	Setup (Max at Slow Process Corner)
Requirement	20.000ns (clk_out1_clk_pll rise@20.000ns - clk_out1_clk_pll rise@0.000ns)
Data Path Delay	4.914ns (logic 1.056ns (21.489%) route 3.858ns (78.511%))
Logic Levels	3 (LUT4=1 LUT5=1 LUT6=1)
Clock Path Skew	-0.025ns
Clock Un...rtainty	0.080ns

Data Path				
Delay Type	Incr (ns)	Path (...)	Location	Netlist Resource(s)
FDCE (Prop fdce C Q)	(f) 0.456	-1.710	Site: SLICE_X36Y61	led_driver_b/cpt/q_counter_reg[0]/Q
net (fo=3, routed)	1.408	-0.302		led_driver_b/cpt/q_counter_reg_n_0_[0]
			Site: SLICE_X36Y61	led_driver_b/cpt/q_counter[27]_i_8__0/1
LUT4 (Prop lut4 I1 O)	(r) 0.150	-0.152	Site: SLICE_X36Y61	led_driver_b/cpt/q_counter[27]_i_8__0/O
net (fo=1, routed)	0.723	0.571		led_driver_b/cpt/q_counter[27]_i_8__0_n_0
			Site: SLICE_X36Y65	led_driver_b/cpt/q_counter[27]_i_4__0/15
LUT6 (Prop lut6 I5 O)	(r) 0.326	0.897	Site: SLICE_X36Y65	led_driver_b/cpt/q_counter[27]_i_4__0/O
net (fo=28, routed)	1.727	2.624		led_driver_b/cpt/q_counter[27]_i_4__0_n_0
			Site: SLICE_X38Y61	led_driver_b/cpt/q_counter[4]_i_1__0/13
LUT5 (Prop lut5 I3 O)	(r) 0.124	2.748	Site: SLICE_X38Y61	led_driver_b/cpt/q_counter[4]_i_1__0/O
net (fo=1, routed)	0.000	2.748		led_driver_b/cpt/q_counter[4]_i_1__0_n_0
FDCE			Site: SLICE_X38Y61	led_driver_b/cpt/q_counter_reg[4]/D
Arrival Time		2.748		

Le chemin critique reste dans le counter_unit du led_driver_b qui compte sur l'horloge de 50Mhz

Le chemin critique sur la clock a = 250Mhz reste dans le compteur du led_driver_a

Summary	
Name	Path 21
Slack	0.532ns
Source	led_driver_a/cpt/q_counter_reg[0]/C (rising edge-triggered cell FDCE clocked by clk_out2_clk_pll {rise@0.000ns fall@2.000ns period=4.000ns})
Destination	led_driver_a/cpt/q_counter_reg[25]/D (rising edge-triggered cell FDCE clocked by clk_out2_clk_pll {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group	clk_out2_clk_pll
Path Type	Setup (Max at Slow Process Corner)
Requirement	4.000ns (clk_out2_clk_pll rise@4.000ns - clk_out2_clk_pll rise@0.000ns)
Data Path Delay	3.404ns (logic 2.265ns (66.535%) route 1.139ns (33.465%))
Logic Levels	8 (CARRY4=7 LUT5=1)
Clock Path Skew	-0.030ns
Clock Uncertainty	0.063ns

Data Path				
Delay Type	Incr (ns)	Path (...)	Location	Netlist Resource(s)
FDCE (Prop fdce C_Q)	(r) 0.419	-1.745	Site: SLICE_X40Y61	led_driver_a/cpt/q_counter_reg[0]/Q
net (fo=3, routed)	0.545	-1.199		led_driver_a/cpt/q_counter[0]
			Site: SLICE_X39Y61	led_driver_a/cpt/plusOp_carry/CYINIT
CARRY4 (Prop carry4 CYINIT CO[3])	(r) 0.755	-0.444	Site: SLICE_X39Y61	led_driver_a/cpt/plusOp_carry/CO[3]
net (fo=1, routed)	0.000	-0.444		led_driver_a/cpt/plusOp_carry_n_0
			Site: SLICE_X39Y62	led_driver_a/cpt/plusOp_carry__0/CI
CARRY4 (Prop carry4 CI_CO[3])	(r) 0.114	-0.330	Site: SLICE_X39Y62	led_driver_a/cpt/plusOp_carry__0/CO[3]
net (fo=1, routed)	0.000	-0.330		led_driver_a/cpt/plusOp_carry__0_n_0
			Site: SLICE_X39Y63	led_driver_a/cpt/plusOp_carry__1/CI
CARRY4 (Prop carry4 CI_CO[3])	(r) 0.114	-0.216	Site: SLICE_X39Y63	led_driver_a/cpt/plusOp_carry__1/CO[3]
net (fo=1, routed)	0.000	-0.216		led_driver_a/cpt/plusOp_carry__1_n_0
			Site: SLICE_X39Y64	led_driver_a/cpt/plusOp_carry__2/CI
CARRY4 (Prop carry4 CI_CO[3])	(r) 0.114	-0.102	Site: SLICE_X39Y64	led_driver_a/cpt/plusOp_carry__2/CO[3]
net (fo=1, routed)	0.000	-0.102		led_driver_a/cpt/plusOp_carry__2_n_0
			Site: SLICE_X39Y65	led_driver_a/cpt/plusOp_carry__3/CI
CARRY4 (Prop carry4 CI_CO[3])	(r) 0.114	0.012	Site: SLICE_X39Y65	led_driver_a/cpt/plusOp_carry__3/CO[3]
net (fo=1, routed)	0.000	0.012		led_driver_a/cpt/plusOp_carry__3_n_0
			Site: SLICE_X39Y66	led_driver_a/cpt/plusOp_carry__4/CI
CARRY4 (Prop carry4 CI_CO[3])	(r) 0.114	0.126	Site: SLICE_X39Y66	led_driver_a/cpt/plusOp_carry__4/CO[3]
net (fo=1, routed)	0.000	0.126		led_driver_a/cpt/plusOp_carry__4_n_0
			Site: SLICE_X39Y67	led_driver_a/cpt/plusOp_carry__5/CI
CARRY4 (Prop carry4 CI_O[0])	(r) 0.222	0.348	Site: SLICE_X39Y67	led_driver_a/cpt/plusOp_carry__5/O[0]
net (fo=1, routed)	0.594	0.941		led_driver_a/cpt/data0[25]
			Site: SLICE_X40Y67	led_driver_a/cpt/q_counter[25]/i_1/I4
LUT5 (Prop lut5 I4_O)	(r) 0.299	1.240	Site: SLICE_X40Y67	led_driver_a/cpt/q_counter[25]/i_1/O
net (fo=1, routed)	0.000	1.240		led_driver_a/cpt/q_counter[25]/i_1_n_0
FDCE			Site: SLICE_X40Y67	led_driver_a/cpt/q_counter_reg[25]/D
Arrival Time		1.240		

Le chemin critique reste dans le counter_unit du led_driver_a qui compte sur l'horloge de 250Mhz

Le chemin critique WNS indiqué sur le design timing summary est un chemin inter_clock

Timing													
Inter-Clock Paths													
From Clock	To Clock	Edges (WNS)	WNS (ns)	TNS (ns)	Failing Endpoints (TNS)	Total Endpoints (TNS)	Edges (WHS)	WHS (ns)	THS (ns)	Failing Endpoints (THS)	Total Endpoints (THS)		
clk_out2_clk_pll	clk_out1_clk_pll	rise - rise	0.521	0.000	0	2	rise - rise	0.229	0.000	0	2		

Le chemin part de logique sous clk_a à 250Mhz sur une logique sous clk_b à 50Mhz

Le chemin critique a un WNS = 0.521ns

Summary

Name	↳ Path 41
Slack	0.521ns
Source	led_driver_a/first_on_reg/C (rising edge-triggered cell FDPE clocked by clk_out2_clk_pll {rise@0.000ns fall@2.000ns period=4.000ns})
Destination	led_driver_b/led_color1_reg[1]/D (rising edge-triggered cell FDCE clocked by clk_out1_clk_pll {rise@0.000ns fall@10.000ns period=20.000ns})
Path Group	clk_out1_clk_pll
Path Type	Setup (Max at Slow Process Corner)
Requirement	4.000ns (clk_out1_clk_pll rise@20.000ns - clk_out2_clk_pll rise@16.000ns)
Data Path Delay	2.964ns (logic 1.122ns (37.848%) route 1.842ns (62.152%))
Logic Levels	3 (LUT3=1 LUT5=2)
Clock Path Skew	-0.345ns
Clock Un...rtainty	0.200ns

Le chemin part du led_driver_a avec une clock à 250Mhz et arrive dans led_driver_b avec un clock à 50Mhz

Data Path

Delay Type	Incr (ns)	Path (...)	Location	Netlist Resource(s)	
FDPE (Prop fdpe C Q)	(r) 0.518	14.352	Site: SLICE_X42Y63	led_driver_a/first_on_reg/Q	logique sous 250Mhz qui participe à la générations du signal d'update des led_drivers
net (fo=5, routed)	0.596	14.948		led_driver_a/cpt/first_on	
			Site: SLICE_X43Y63	led_driver_a/cpt/strech_count[3]_i_3/I0	
LUT3 (Prop lut3 I0 O)	(f) 0.153	15.101	Site: SLICE_X43Y63	led_driver_a/cpt/strech_count[3]_i_3/O	
net (fo=2, routed)	0.595	15.695		led_driver_a/cpt/strech_count[3]_i_3_n_0	
			Site: SLICE_X42Y62	led_driver_a/cpt/strech_count[3]_i_1/I0	logique sous 50Mhz qui update le led_driver_b
LUT5 (Prop lut5 I0 O)	(r) 0.327	16.022	Site: SLICE_X42Y62	led_driver_a/cpt/strech_count[3]_i_1/O	
net (fo=8, routed)	0.652	16.675		led_driver_b/sel	
			Site: SLICE_X43Y61	led_driver_b/led_color1[1]_i_1/2	
LUT5 (Prop lut5 I2 O)	(r) 0.124	16.799	Site: SLICE_X43Y61	led_driver_b/led_color1[1]_i_1/O	
net (fo=1, routed)	0.000	16.799		led_driver_b/led_color1[1]_i_1_n_0	
FDCE			Site: SLICE_X43Y61	led_driver_b/led_color1_reg[1]/D	
Arrival Time		16.799			

Destination Clock Path

Le chemin critique part du registre first_on du led_driver_a.

Ce signal est utilisé pour générer le signal d'update = end_cycle_long commun aux 2 leds_drivers.

end_cycle_long est généré sur une logique à 205Mhz (en partie dans le led_driver_a)puis envoyé dans le led_driver_b pour l'update.

- Validation sur la carte: mesures sur l'ILA

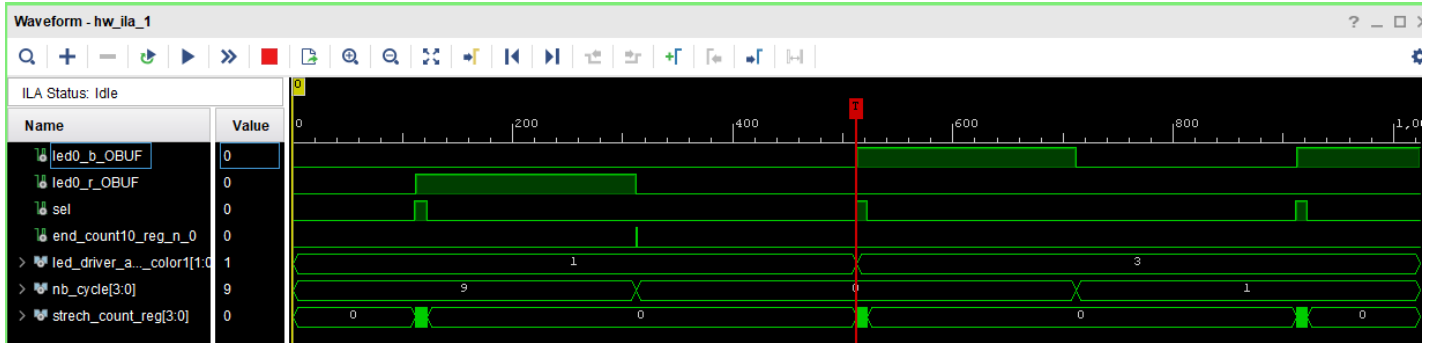
Liste des signaux sur le domaine clka

led0_r_obuf
led0_b_obuf
sel correspond au signal end_cycle_long
end_count10

Liste des signaux sur le domaine clkb

led1_r_obuf
led1_r_obuf
led_color[0]
led_color[1]

Afin de pouvoir mesurer le comportement du circuit, la synthèse a été refaite avec une cible = 200

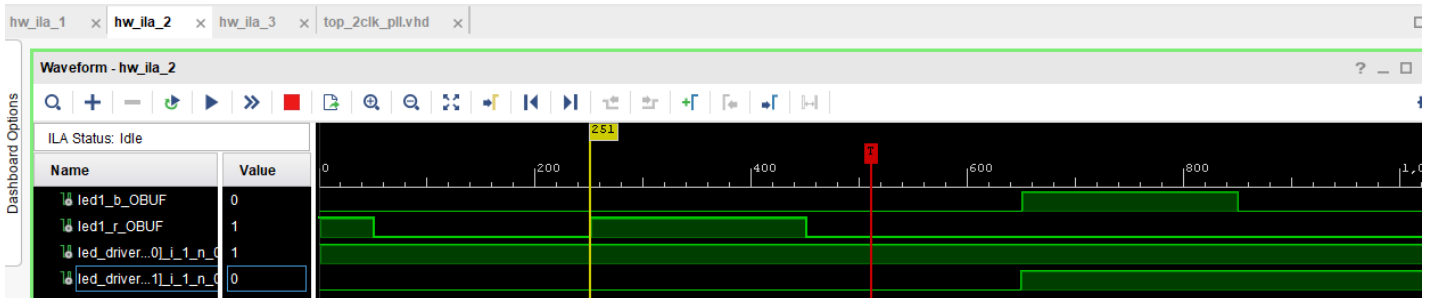


end_count10 passe à 1 quand nb_cycle = 9

Le signal sel qui représente end_cycle_long a une durée rallongée (grâce au décompteur strech_count) pour permettre à la led1 de se mettre à jour

La couleur de la led0 change de rouge à bleu sur l'update suivant

La mesure de la led1 sur fait sur une autre ila qui fonctionne à 50Mhz



La led1 change bien de couleur de rouge à bleu

- Validation sur la carte: mesures à l'oscilloscope

Afin de mesurer plus facilement sur l'oscilloscope:

Les sorties led0_r , led0_b , led0_g, led1_r , led1_b , led1_g) sont dupliquées et routées sur des ports user_io. afin de pouvoir mesurer quelques cycles sur la capacité de capture de l'oscilloscope

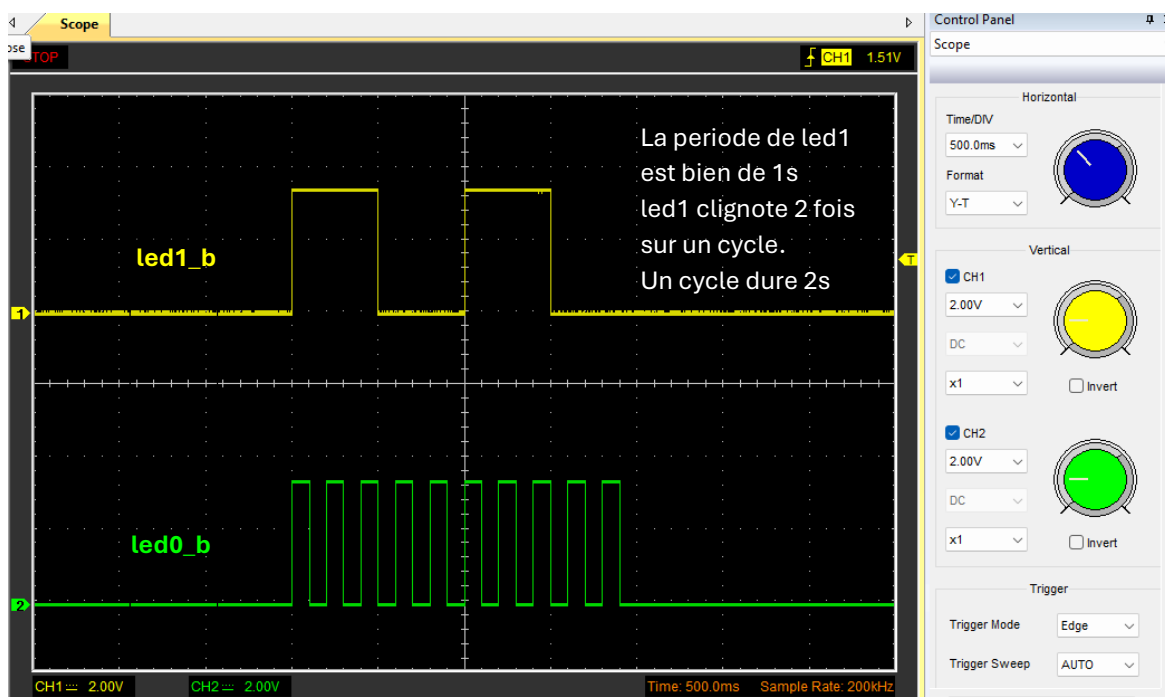
Le cycle de comptage est donc calé à 25 000 000

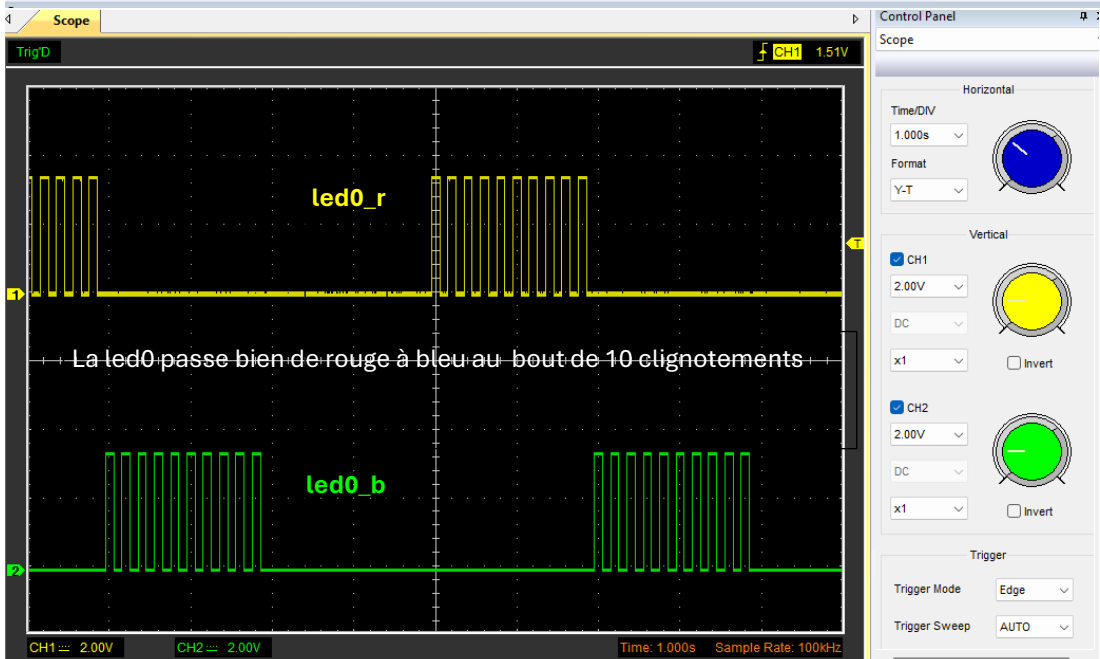
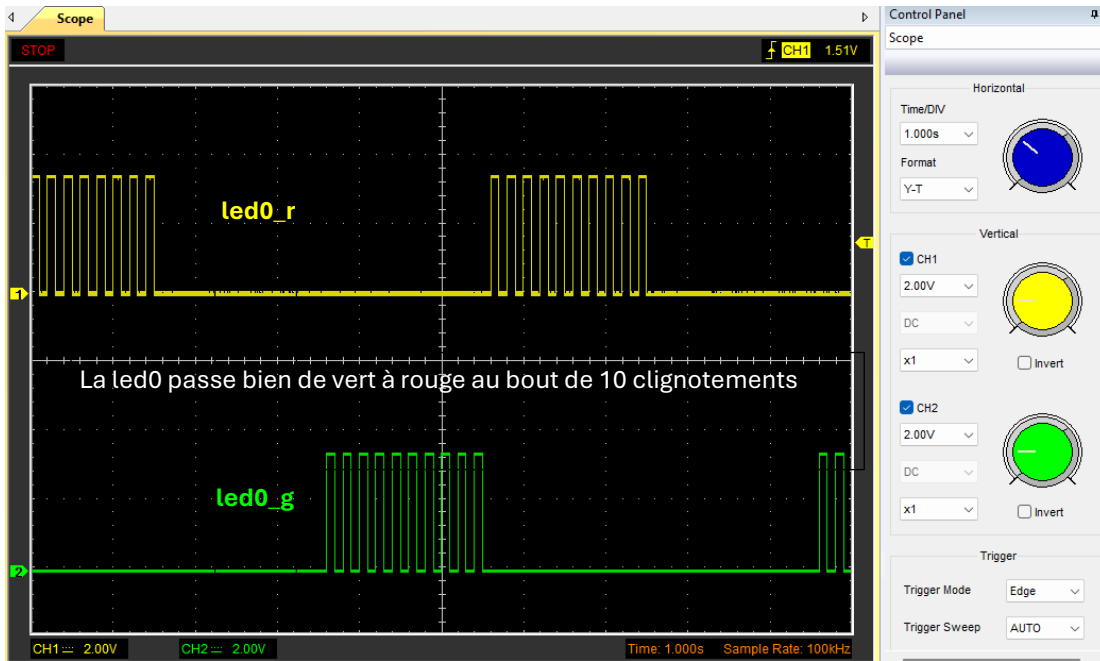
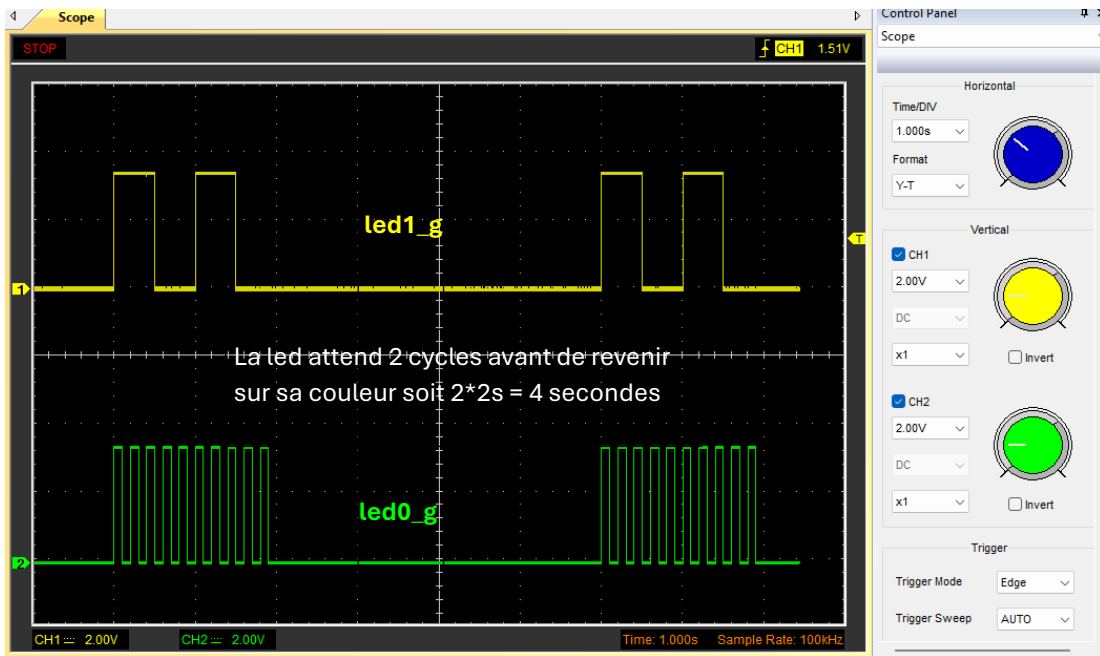
La période de led0 sera de 200ms , 10 cycles dureront 2s

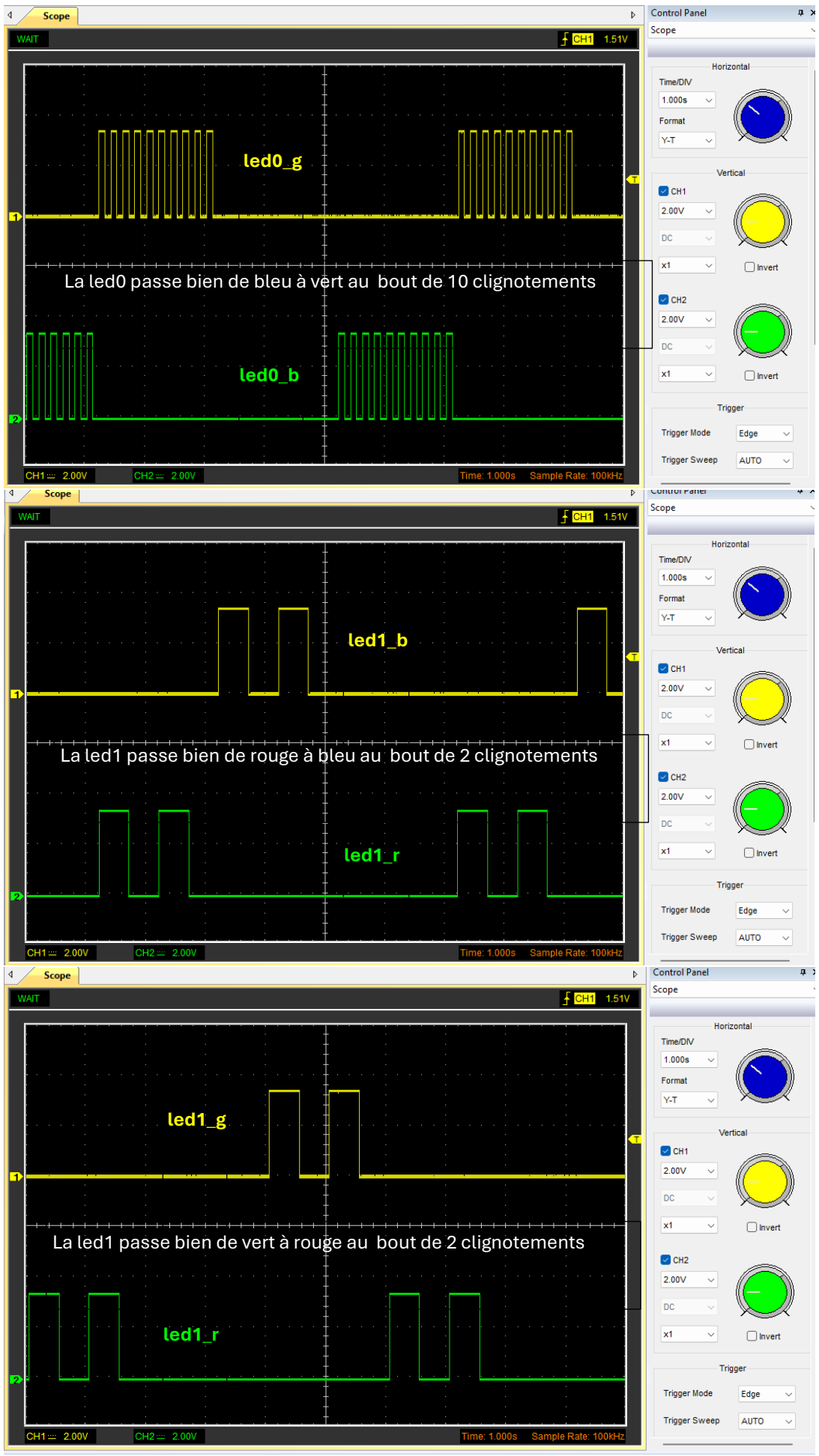
La période de led1 sera de 1s

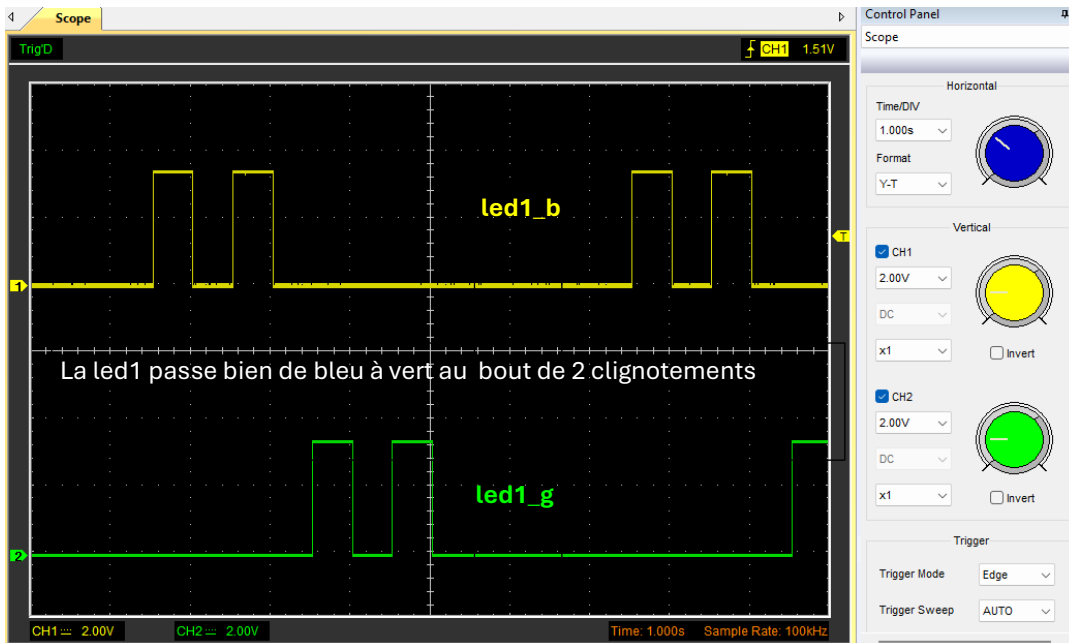
Le nouveau fichier est top_2_clk_pll_test.vhd

L'implémentation est refaite avec ces paramètres.









- Démonstration

Après avoir resetté le circuit, on vérifie le cycle et le nombre de clignotement de chaque led
La démonstration est faite avec un cible = 100000000

Conclusion

Le TP5 a permis de mettre en œuvre et de gérer plusieurs domaines d'horloge au sein d'un même circuit FPGA. L'utilisation d'une PLL a permis de générer deux horloges de fréquences différentes à partir d'une horloge de référence. Ce TP a permis d'étudier les problématiques liées aux changements de domaine d'horloge (Clock Domain Crossing) et de mettre en place des architectures assurant une synchronisation correcte des signaux entre ces différents domaines. La simulation, l'analyse des schémas RTL, des rapports de timing et l'utilisation de l'ILA ont permis de valider le fonctionnement du circuit et d'identifier les chemins critiques afin d'optimiser l'architecture lorsque les contraintes temporelles n'étaient pas respectées.